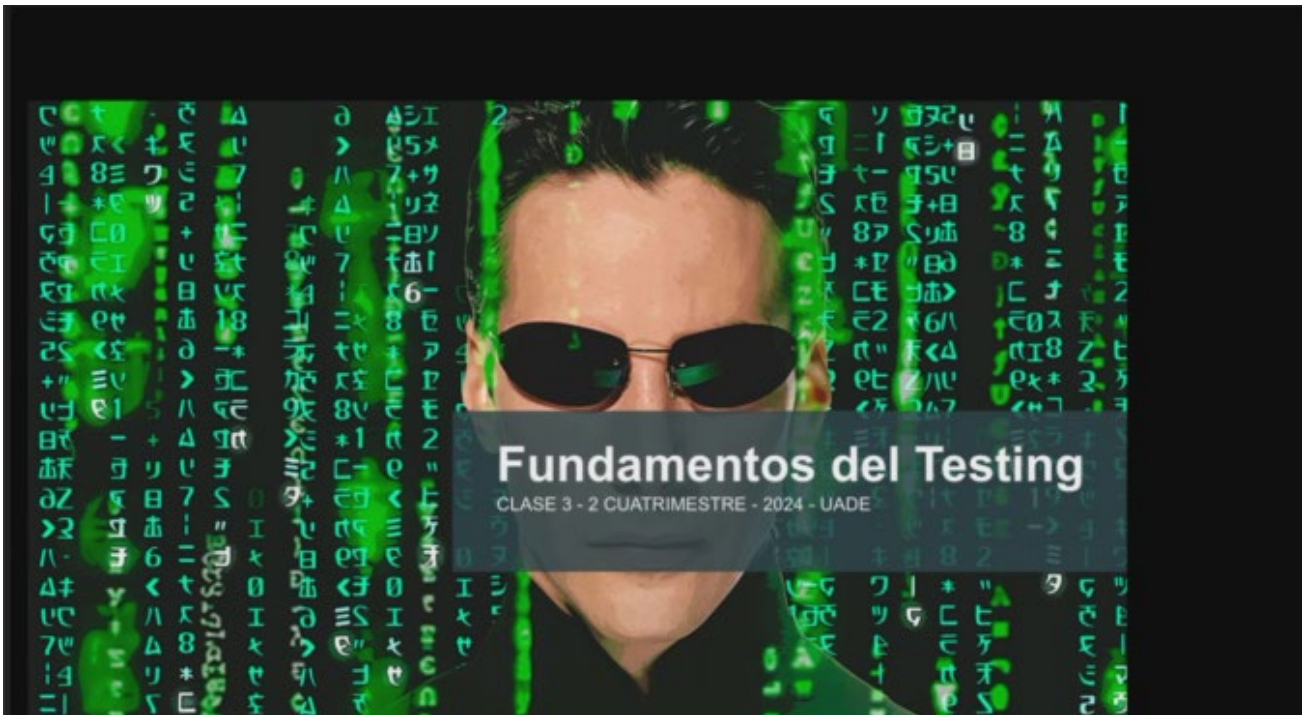


Clase 4

Técnicas de Testing

Se vienen los ejercicios prácticos del parcial:

- Árbol de decisión (vale 3 puntos del parcial: CLAVE para promocionar!)
- Tabla de verdad



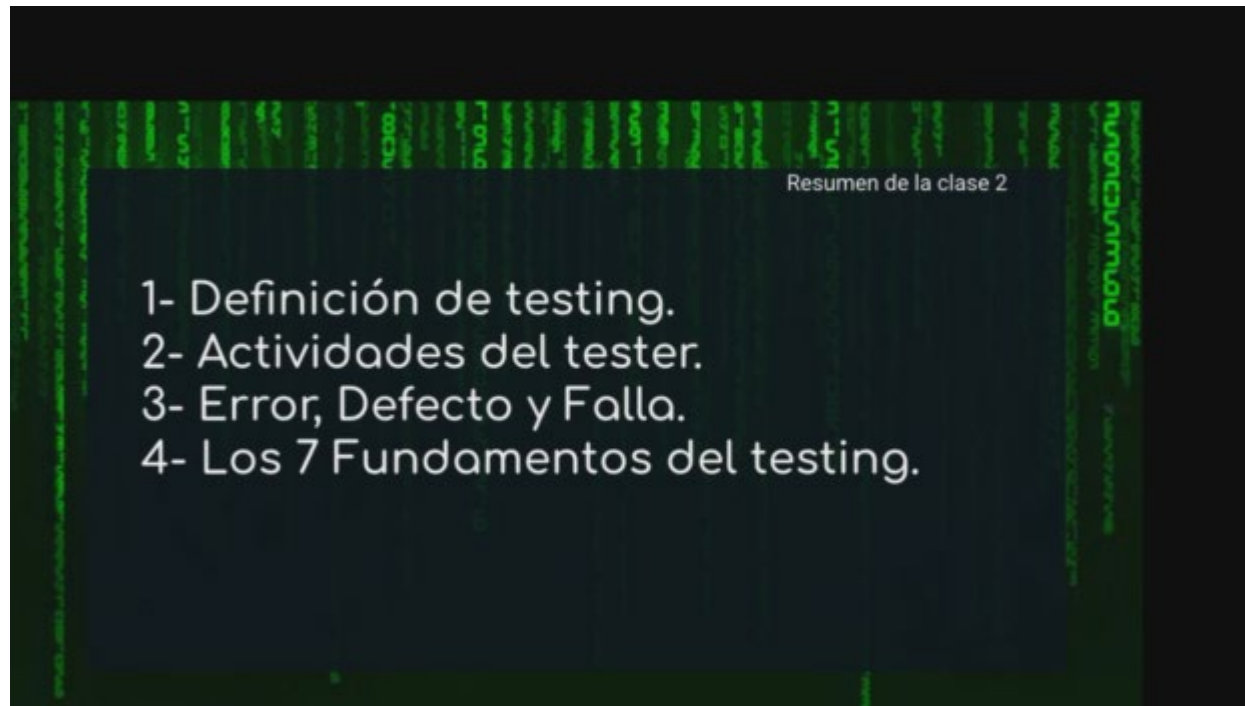
- Tener buen nivel de inglés es clave.
- Introducción a SQL.
- Lenguaje de programación (ej. Python)

Recomendación y mejora continua.

Clave: ENTENDER EL PROCESO

Ay pero me gusta lo técnico!

SE AUTOMATIZA MÁS FÁCIL ESO! Somos seres pensantes



Hormiga: Si siempre testeo lo mismo, nunca descubriré el error.

Lo mismo sucede si tengo siempre al mismo equipo.

Entidades - Modelo mental

Clasificación

Tipos de Pruebas

Técnicas Pruebas

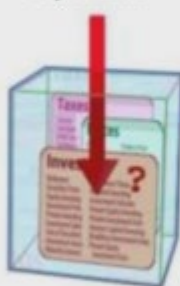
Clasificación de Pruebas

Caja Negra



Probamos los requisitos funcionales desde el exterior

Caja Blanca



Probamos las funciones internas de un módulo

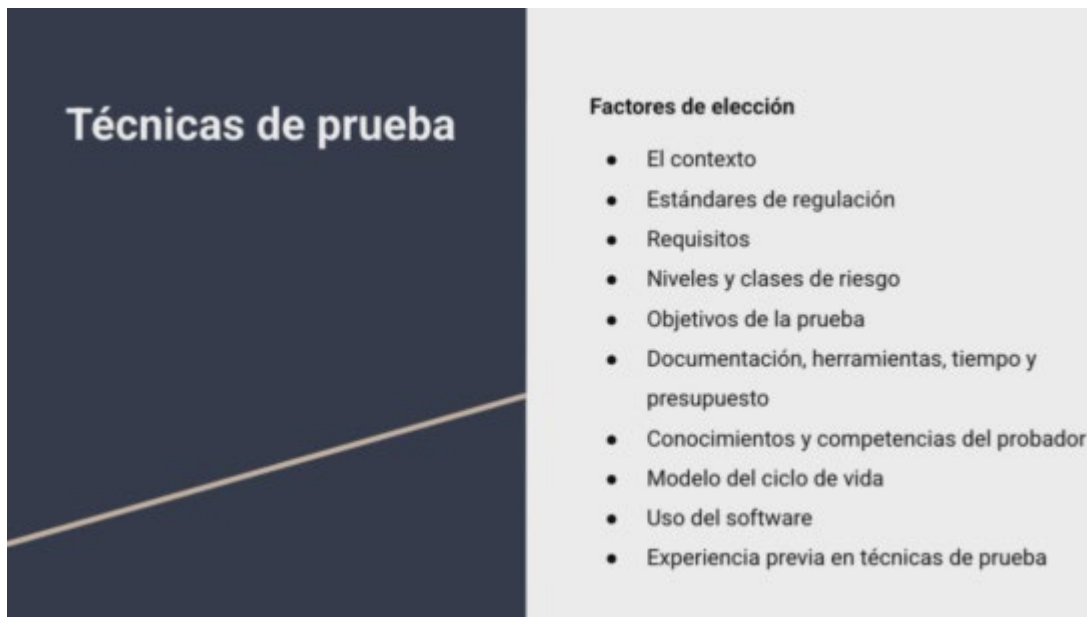
Caja Blanca:

Las pruebas de caja blanca, son un enfoque en la ingeniería de software donde **se examina la estructura interna** de un sistema. Las pruebas de caja blanca involucran la inspección directa del **código fuente**, **las rutas de ejecución** y las estructuras de **datos** para identificar posibles errores y garantizar una cobertura exhaustiva.

Los evaluadores que realizan pruebas de caja blanca tienen acceso al código fuente y pueden diseñar pruebas específicas para alcanzar diferentes caminos de ejecución dentro del sistema. Esto permite verificar la lógica interna, la coherencia del código y la implementación correcta de algoritmos. Además, las pruebas de caja blanca son útiles para garantizar que se cumplan estándares de codificación, detectar problemas de seguridad y optimizar el rendimiento del software.

Técnicas de Testing (Tema Nuevo):

Tenemos técnicas porque NO PODEMOS PROBAR TDO

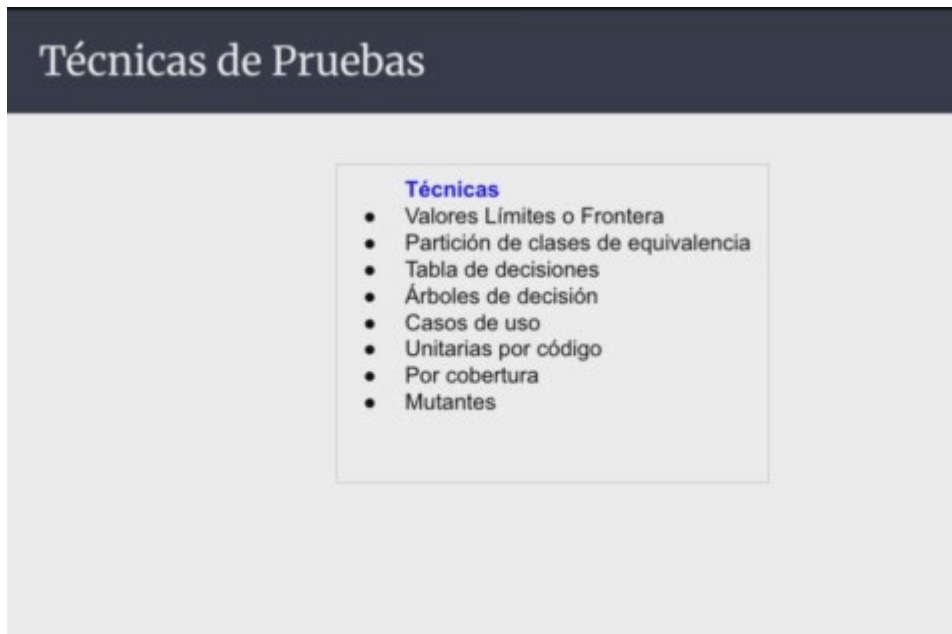


Técnicas de prueba

Factores de elección

- El contexto
- Estándares de regulación
- Requisitos
- Niveles y clases de riesgo
- Objetivos de la prueba
- Documentación, herramientas, tiempo y presupuesto
- Conocimientos y competencias del probador
- Modelo del ciclo de vida
- Uso del software
- Experiencia previa en técnicas de prueba

EXAMEN: Enumere las técnicas de testing. Explique 2



Técnicas de Pruebas

Técnicas

- Valores Límites o Frontera
- Partición de clases de equivalencia
- Tabla de decisiones
- Árboles de decisión
- Casos de uso
- Unitarias por código
- Por cobertura
- Mutantes

Tipos de Pruebas

Caja Negra

Funcionales

- Componentes
- Integración Func.
- Sistemas
- Aceptación

Cambios

- Confirmación
- Regresión
- Smoke (Humo)

No funcionales

- Rendimiento
- Carga/Estrés
- Usabilidad
- Mantenimiento
- Confiabilidad
- Portabilidad
- Seguridad informática
- Implementación

Caja Blanca

- Unitarias
- Integración Técnica
- Rendimiento
- Carga/Estrés
- Seguridad

Las que importan para el examen

Técnicas de Pruebas

Técnicas

- Valores Límites o Frontera
- Partición de clases de equivalencia
- Tabla de decisiones
- Árboles de decisión
- Casos de uso
- Unitarias por código
- Por cobertura
- Mutantes

Los recursos son FINITOS entonces tene

Pruebas de Caja Negra

Partición de clases de equivalencias

"Dividir un conjunto de condiciones de prueba en grupos o conjuntos a los cuales los consideramos iguales."

Asumimos que todas las condiciones de una partición serán tratadas de la misma forma por el software."

Divide and conquer.

Pruebas de Caja Negra

Partición de clases de equivalencias

Ejemplo: Sistema electoral

Menores

1 ... 15

Mayores

16 ... n

Esta técnica requiere que se pruebe sólo una condición de cada partición

Ej de partición: Mayores de edad pueden votar - Menores de edad no pueden votar (app Mi Argentina). Tendría que dar un MENSAJE DE QUE NO SE PUEDE, NO UN ERROR: UN ERROR ES ALGO NO ESPERADO (Ej: si no hubiera controlado que me pusieran edad negativa)

Pruebas de Caja Negra

Partición de clases de equivalencias

Ejemplo: Solicitud de tarjeta de crédito

Paso 1) identificar las particiones válidas y no válidas

Partición no válida

Valores menores a \$40000

Partición válida para tarjeta Dorada

Valores entre 40000 y 90000

Partición válida para tarjeta Platinum

Valores entre 90001 y 200000

Partición válida para tarjeta Black

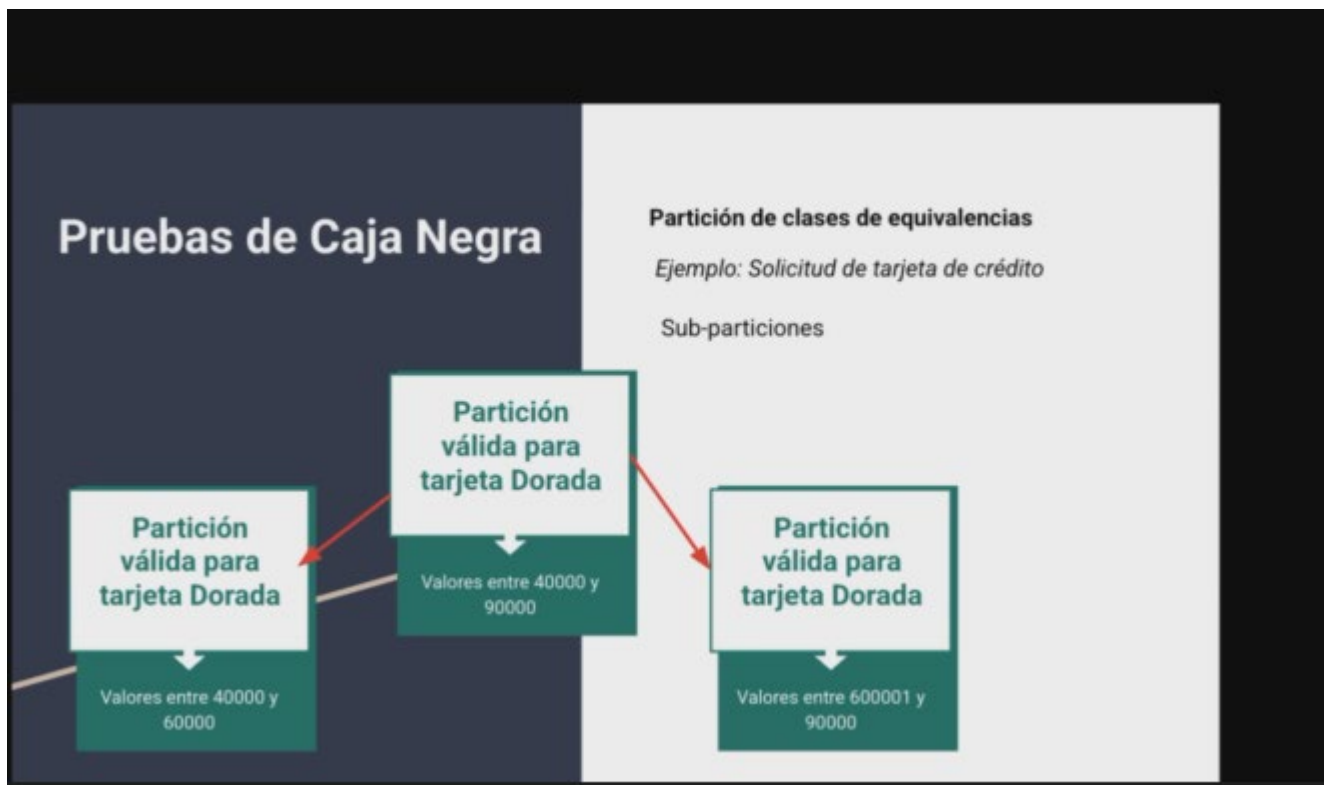
Valores desde 200001 a n

De acuerdo a tu salario, te dan la black, la signatura... (ojalá tener el nivel socio-económico para que me ofrezcan cualquiera jsjs)

1ero: VALIDO

(...)

Defino la técnica: Partición de clases de equivalencia.



Son particiones de particiones

Lo uso? Depende del CONTEXTO → Cuántas personas hay en mi equipo? Cuánto tiempo tengo?



Si tengo 4 particiones, y probé 3: Qué porcentaje de cobertura tengo? 75%

Pruebas de Caja Negra

Análisis de valores frontera o límite

- Extensión de la partición de equivalencia.
- Solo sirve cuando la partición está ordenada.
- Los valores iniciales y finales, de cada partición, son sus valores frontera

VALORES LÍMITES: Ej: Admito solo +18 → Pruebo -1, 17, 18, 19, 150.

Pruebas de Caja Negra

2

Identificamos 2 valores límite por frontera

Inválida (muy baja)

0

Válida

1, 2, 3, 4, 5

Inválida (muy alta)

6, 7, 8, 9

Inválida (muy baja) - Válida

0 y 1

Análisis de valores frontera o límite

Ejemplo

Un campo que acepta sólo valores positivos entre el 1 y el 5.

Pruebas de Caja Negra

2

Identificamos 2 valores límite por frontera

Inválida (muy baja)

0

Válida

1, 2, 3, 4, 5

Inválida (muy alta)

6, 7, 8, 9

Inválida (muy baja) - Válida

0, 1, 2

Válida - Inválida (muy alta) -

4, 5, 6

Análisis de valores frontera o límite

Ejemplo

Un campo que acepta sólo valores positivos entre el 1 y el 5.

Por convención, el valor frontera se toma de la partición válida

positivos entre

Análisis de tres puntos

Los valores límites siempre en gama de FECHA y EDAD

Pruebas de Caja Negra

Análisis de valores frontera o límite

RECUERDA:
Ideal para probar
requisitos de
rangos ordenados,
como edades,
fechas, horas.

Vimos 2 técnicas:

1. Partición de equivalencias
2. Valores Límites

Pruebas de Caja Negra

Tablas de decisión



Fuente: https://www.youtube.com/watch?v=Z_IA_prGpp8

https://youtu.be/Z_IA_prGpp8?si=PerDRVd5nBwauLvN



Condiciones

- Ingresar modelo de vehículo
- Ingresar color de vehículo



Combinación 1

Combinación 2

Combinación 3

Combinación 4

Condiciones

¿Se ingresó el modelo del vehículo?

Verdadero

Verdadero

Falso

Falso

¿Se ingresó el color del vehículo?

$$\frac{4}{2} = 2 \rightarrow \frac{2}{2}$$

Condiciones

- Ingresar modelo de vehículo
- Ingresar color de vehículo



Combinación 1

Combinación 2

Combinación 3

Combinación 4

Condiciones

¿Se ingresó el modelo del vehículo?

Verdadero

Verdadero

Falso

Falso

¿Se ingresó el color del vehículo?

Verdadero

Falso

Verdadero

Falso

Acciones/Salidas

¿Se procesa la característica 1?

Verdadero

Mensaje de error

Mensaje de error

Mensaje de error

¿Se procesa la característica 2?

Verdadero

Falso

Verdadero

Falso

¿Mensaje de error?

Falso

Verdadero

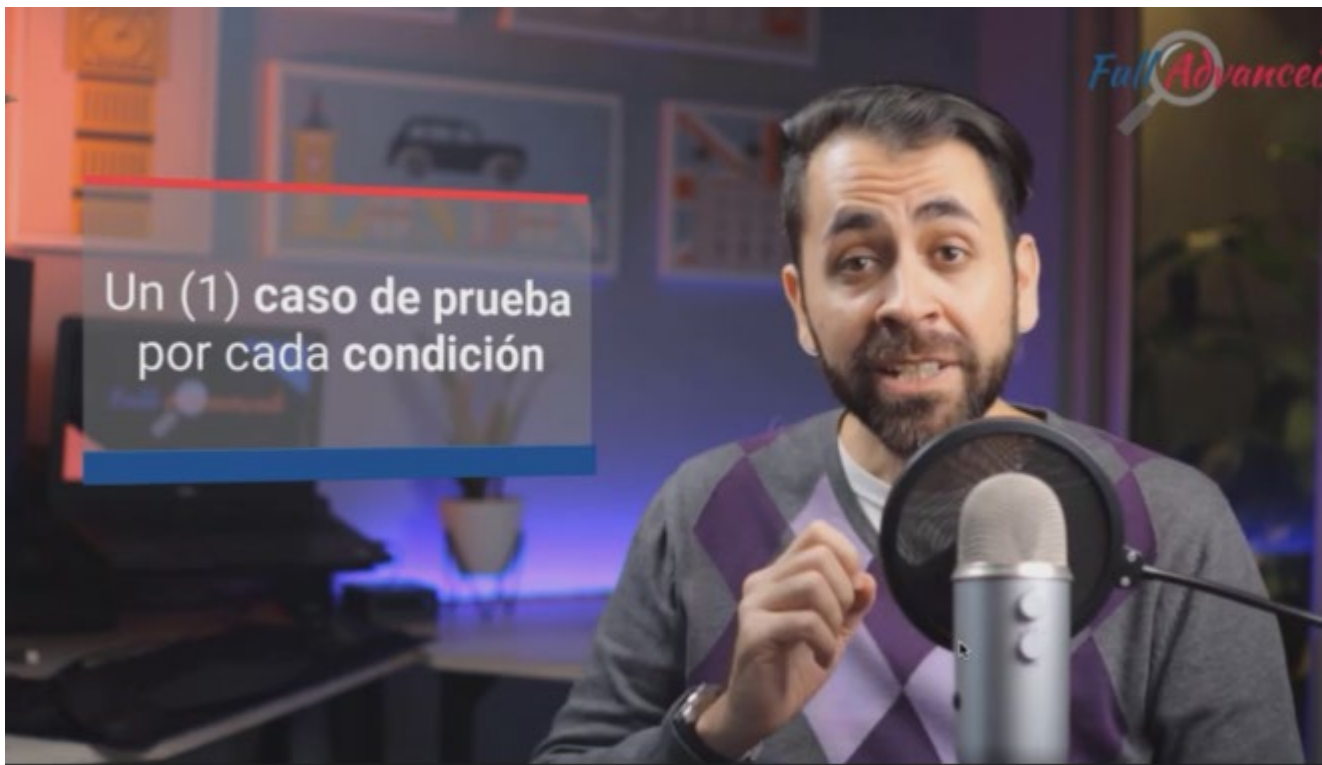
Verdadero

Verdadero

Esto lo veremos después del 1er parcial:

Identificador de caso de prueba: MP002-01			
Objetos a probar:	Módulo de solicitud de vehículo - ingreso de características Versión 2.1.1		
Dependencias:	Módulo de clientes		
Ambiente:	Servidor de pruebas		
Requisitos de procedimiento especiales:	El cliente debe haber registrado sus datos personales en el módulo de clientes		
Descripción:	Indicar los datos correspondientes al modelo y color del vehículo deseado, y verificar que se hayan procesado.		
Secuencia Normal - Pasos		Resultado Esperado	Resultado Obtenido
1	Ejecutar el caso de prueba MP001-01 para lograr el ingreso al sistema	Se logra la autenticación del cliente, y se abre la página para realizar una solicitud de vehículo	Aceptado
2	Seleccionar el Tipo de vehículo en el campo del mismo nombre.	Se muestra una imagen del tipo de vehículo seleccionado	Aceptado
3	Seleccionar el Color del vehículo en el campo del mismo nombre.	El color de la imagen del vehículo seleccionado se actualiza según corresponda	Aceptado
4	Hacer click en el botón Procesar.	Se envía mensaje de "Solicitud Procesada"	Aceptado
5			

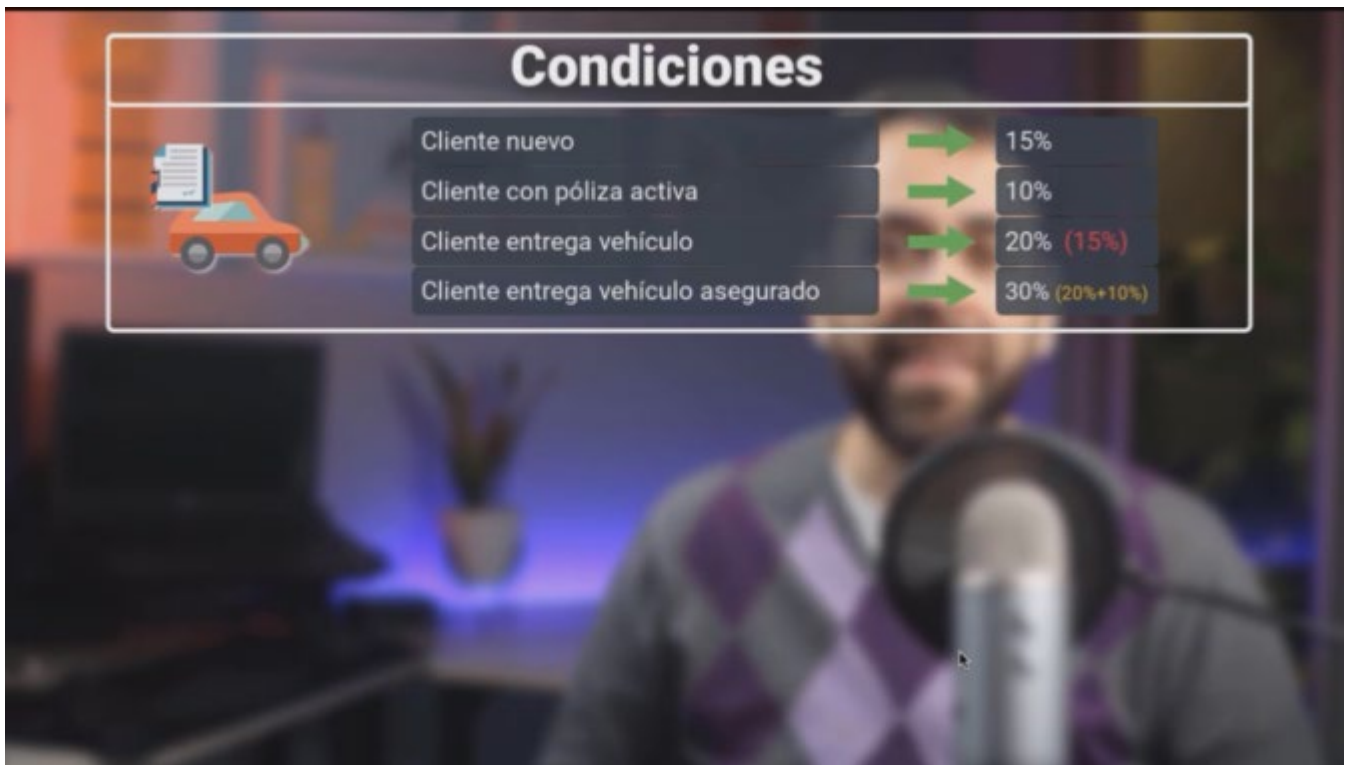
Identificador de caso de prueba: MP002-02			
Objetos a probar:	Módulo de solicitud de vehículo - ingreso de características Versión 2.1.1		
Dependencias:	Módulo de clientes		
Ambiente:	Servidor de pruebas		
Requisitos de procedimiento especiales:	El cliente debe haber registrado sus datos personales en el módulo de clientes		
Descripción:	Indicar los datos correspondientes al modelo y dejar el color del vehículo vacío.		
Secuencia Normal - Pasos		Resultado Esperado	Resultado Obtenido
1	Ejecutar el caso de prueba MP001-01 para lograr el ingreso al sistema	Se logra la autenticación del cliente, y se abre la página para realizar una solicitud de vehículo	Aceptado
2	Seleccionar el Tipo de vehículo en el campo del mismo nombre.	Se muestra una imagen del tipo de vehículo seleccionado	Aceptado
3	Dejar el campo Color de vehículo vacío.	El campo Color de vehículo se mantiene vacío	Aceptado
4	Hacer click en el botón Procesar.	Se muestra un mensaje de error	Aceptado
5			
Excepciones - Pasos			



Eliminar casos cuyas combinaciones no existirían → Colapsar tablas

Condiciones				
- Ingresar modelo de vehículo - Ingresar color de vehículo				
	Combinación 1	Combinación 2	Combinación 3	Combinación 4
Condiciones				
¿Se ingresó el modelo del vehículo?	Verdadero	Verdadero	Falso	Falso
¿Se ingresó el color del vehículo?	Verdadero	Falso	Verdadero	Falso
Acciones/Salidas				
¿Se ingresaron ambos datos?	Se procesa la solicitud del vehículo	Mensaje de error	Mensaje de error	Mensaje de error

Las condiciones son más complejas que binarias. En esos casos, el número de combinaciones será muy grande por lo cual deberemos hacer un muestreo en lugar de realizarlas todas



Condiciones		
	Cliente nuevo	15%
	Cliente con póliza activa	10%
	Cliente entrega vehículo	20% (15%)
	Cliente entrega vehículo asegurado	30% (20%+10%)

Descuentos no acumulables

PARCIAL: Va a haber entre 3 y 4 condiciones.

REQUIERIMIENTOS NO ESPECIFICADOS: En el examen (y en la vida real) no me van a decir todo tácitamente/numéricamente lo que tengo que hacer: Tengo que interpretar texto/palabras habladas → PROBLEMA NARRATIVO!

EXÁMEN → Ejercicio de 3 puntos de tabla de verdad y TODO lo demás es análisis de negocio

Condiciones

Cliente nuevo	→	15%
Cliente con póliza activa	→	10%
Cliente entrega vehículo	→	20% (15%)
Cliente entrega vehículo asegurado	→	30% (20%+10%)

	1	2	3	4	5	6	7	8
Condiciones								
Cliente nuevo (15%)	V	V	V	V	F	F	F	F
Cliente existente (10%)	V	V	F	F	V	V	F	F
Cliente entrega vehículo (20%)	V	F	V	F	V	F	V	F
Acciones / Salidas								
Porcentaje aplicado	-	-	20	15	30	10	20	0

Por que no con 4 condiciones? (igual si puede/suele estar siempre entre 3 y 4)

Si es falso VIP y si es falso PAYER → Es NON-PAYER: Contraje 2 condiciones en una

Pruebas de Caja Negra

Condiciones	Reglas							
Condición 1	S	S	S	S	N	N	N	N
Condición 2	S	S	N	N	S	S	N	N
Condición 3	S	N	S	N	S	N	S	N
Acción 1	X	X						
Acción 2				X		X		X
Acción 3			X				X	
Acción 4				X				

Tablas de decisión - Tipos

Tablas de decisión binaria: cuando todas las condiciones son binarias, es decir la evaluación de todas las condiciones está limitada a dos valores posibles. También se denomina limitadas. Los valores en general serán SI (S), NO (N), aunque pueden tomar otros valores binarios, por ejemplo, BLANCO (B) o NEGRO(N).

Tablas de decisión múltiples: cuando todas sus condiciones pueden tomar más de dos valores. También se denominan Ampliadas o Extendidas.

Pruebas de Caja Negra

Condiciones	Reglas							
Condición 1	S	S	S	S	N	N	N	N
Condición 2	S	S	N	N	S	S	N	N
Condición 3	S	N	S	N	S	N	S	N
Acción 1	X	X						
Acción 2				X		X		X
Acción 3			X				X	
Acción 4					X			

Tablas de decisión - Ejercicio

Se quiere determinar la nómina (sueldo) de los empleados de una empresa de acuerdo con estos criterios:

- Si el empleado es altamente productivo tendrá además un bono de productividad.
- Si el empleado es encargado de su grupo tendrá además un bono de encargado.
- Si el empleado ha cometido una infracción grave durante ese mes le será eliminado cualquier bono que pudiera tener.

Nómina = sueldo

Condiciones (de Empleado)	Reglas							
Altamente productivo	V	V	V	V	F	F	F	F
Encargado	V	V	F	F	V	V	F	F
Infraccionado	V	F	V	F	V	F	V	F
Acción 1: Bono de Productividad				X				
Acción 2: Bono de encargado						X		
Acción 3: Bono de Productividad + Bono de Encargado		X						
Acción 4: Sin Bonos	X		X		X		X	X
Corrección: Podía elegir más de una acción, y no tener que especificar una que sea la conjunción de dos (Acción 3)								
Condiciones (de Empleado)	Reglas							
Altamente productivo	V	V	V	V	F	F	F	F
Encargado	V	V	F	F	V	V	F	F
Infraccionado	V	F	V	F	V	F	V	F
Acción 1: Bono de Productividad		X		X				
Acción 2: Bono de encargado		X				X		
Acción 4: Sin Bonos	X		X		X		X	X

Nota de color: La IA es NO DETERMINÍSTICA; Determina rangos probabilísticos, NO valores exactos.

Pruebas de Caja Negra

Condiciones	Reglas							
Condición 1	S	S	S	S	N	N	N	N
Condición 2	S	S	N	N	S	S	N	N
Condición 3	S	N	S	N	S	N	S	N
Acción 1	X	X						
Acción 2				X		X		X
Acción 3			X				X	
Acción 4					X			

Tablas de decisión - Ejercicio N° 2

Una institución de educación, con carreras tanto diurnas como vespertinas, considera lo siguiente en su proceso académico: si un alumno de carrera vespertina tiene una nota de presentación mayor o igual a 7,0, promociona, y si no alcanza el 7 pero tiene una nota de presentación igual o mayor a 4, presenta examen. Si el alumno es de carrera diurna, si su nota de presentación es mayor o igual a 4, presenta examen (no hay posibilidad de promoción). En ambos regímenes, si la nota de presentación es menor que 4, no aprueba la asignatura.

Diferencia entre diurno y vespertino: Día y noche.

Hay DOS GRUPOS DE CONDICIONES que podemos optimizar acá!

Si es diurna, no es vespertina.

NO poner: Condición diurna, condición vespertina.

		Reglas															
Condiciones		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
nota >= 7		V	V	V	V	V	V	V	V	F	F	F	F	F	F	F	F
nota >= 4 y nota < 7		V	V	V	V	F	F	F	F	V	V	V	V	F	F	F	F
nota < 4		V	V	F	F	V	V	F	F	V	V	F	F	V	V	F	F
carrera vespertina		V	F	V	F	V	F	V	F	V	F	V	F	V	F	V	F
Acciones																	
Promociona		-	-	-	-	-	-	X		-	-						
Presenta examen		-	-	-	-	-	-		X	-	-	X	X				
No aprueba		-	-	-	-	-	-			-	-			X	X	X	X

Paso a paso

- 1 Armar esqueleto de tabla x
- 2 Identificar las Condiciones x
- 3 Identificar las Acciones x
- 4 Completamos tabla x

Una institución de educación, con carreras tanto diurnas como vespertinas, considera lo siguiente en su proceso académico: si un alumno de **carrera vespertina** tiene una nota de presentación **mayor o igual a 7,0, promociona**, y si no alcanza el 7 pero tiene una nota de presentación **igual o mayor a 4, presenta examen**. Si el alumno es de **carrera diurna**, si su nota de presentación **es mayor o igual a 4, presenta examen** (no hay posibilidad de promoción). En ambos regímenes, si la nota de presentación **es menor que 4, no aprueba la asignatura**.

Cantidad de condiciones: $2^4=16$

Asumciones: Si no se presentó (no posee ninguna de las categorías de notas), también desaprueba

Condiciones	Reglas														
Nota Presentación ≥ 7	V	V	V	V	V	V	V	V	F	F	F	F	F	F	F
Nota Presentación $\leq 7 \wedge \geq 4$	V	V	V	V	F	F	F	F	V	V	V	V	F	F	F
Nota Presentación < 4	V	V	F	F	V	V	F	F	V	V	F	F	V	F	F
Turno Sin Promoción (Diurno)	V	F	V	F	V	F	V	F	V	F	V	F	V	V	F
Acción 1: Promociona	-	-	-	-	-	-	-	X	-	-	-	-	-	-	-
Acción 2: Rinde Final	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
Acción 3: Desaprueba	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

Absurdo: No pueden superponerse estados de notas (en las 3 primeras filas solo puede haber 1 V) --> ROJO

Clase 5

Tema

En el EXÁMEN seguro haya ejercicios hablando de monedas virtuales en el contexto de juegos móviles.

Veremos ÁRBOL DE DECISIÓN, y entra en el parcial

Tipos de testing:

Caja negra- caja blanca

Estrategias de testing:

Particion, valores limites etc

¿Para qué me sirven los casos de uso?

	Casos de Uso Es una descripción de un uso particular de un sistema por parte de un usuario del mismo, un <u>actor</u>. <i>Se describen las interacciones del actor con el sistema para realizar una tarea específica.</i> Un actor puede ser una persona, o un sistema. <i>Los casos de uso se ven desde la óptica del actor, no el sistema!!</i>
--	--

Separo CASOS DE USO de USER STORIES, por qué?

Porque los casos de uso los tengo que desarrollar RÁPIDO

Se va construyendo de manera ITERATIVA: Puedo definir, construir y probar rápido

User Story vs Caso de Uso en Testing

User Story (Historia de Usuario)

Es una descripción corta y simple de una funcionalidad desde la **perspectiva del usuario final**. Sigue el formato:

"Como [tipo de usuario], quiero [acción] para [beneficio]"

Ejemplo:

"Como cliente, quiero poder filtrar productos por precio para encontrar opciones dentro de mi presupuesto."

- Origen: metodologías ágiles (Scrum, XP)
- Foco: **qué** necesita el usuario y **por qué**
- Nivel de detalle: alto nivel, intencional y simple
- Los criterios de aceptación definen cuándo la historia está "done"
- En testing: sirve para diseñar casos de prueba de aceptación (UAT)

Caso de Uso (Use Case)

Es una descripción detallada de la **interacción entre un actor y el sistema** para lograr un objetivo. Documenta el flujo completo: escenario principal, flujos alternativos y excepciones.

Ejemplo:

Actor: Cliente

Precondición: El usuario está en la página de productos

Flujo principal: 1) Selecciona filtro "precio" → 2) Ingresa rango → 3) El sistema devuelve resultados → 4) El usuario ve los productos filtrados

Flujo alternativo: Si no hay resultados, mostrar mensaje "Sin coincidencias"

- Origen: UML / desarrollo tradicional (RUP, cascada)
- Foco: **cómo** interactúa el usuario con el sistema paso a paso
- Nivel de detalle: bajo nivel, muy específico
- En testing: sirve para diseñar casos de prueba funcionales y de integración

Diferencia clave resumida

	User Story	Caso de Uso
Perspectiva	Usuario / negocio	Sistema / técnica
Detalle	Alto nivel	Bajo nivel

	User Story	Caso de Uso
Metodología	Ágil	Tradicional/UML
Formato	Frase simple	Documento estructurado
En testing	Base para UAT	Base para pruebas funcionales
Flujos alternativos	No incluye	Sí incluye

En la práctica, una **user story** responde al "*¿qué quiere el usuario?*" y un **caso de uso** al "*¿cómo lo hace el sistema?*". En proyectos ágiles modernos, los casos de uso se reemplazaron en gran parte por user stories + criterios de aceptación.

Pruebas de Caja Negra

Casos de Uso

Es una descripción de un **uso** particular de un sistema por parte de un **usuario** del mismo, un actor. Se describen las interacciones del actor con el sistema para realizar una tarea específica.

Un actor puede ser una persona, o un sistema.

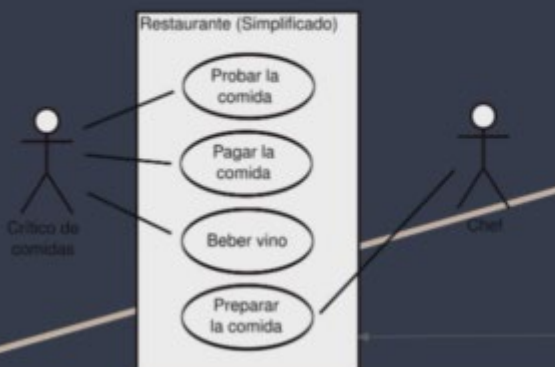
Los casos de uso se ven desde la óptica del actor, no el sistema!!

Pruebas de Caja Negra

Casos de Uso

Muy útiles para los niveles de prueba de aceptación de usuario y Sistema, justamente porque nos servirá para encontrar fallas que un usuario detectaría al usar el sistema por primera vez.

Diagrama de caso de uso



Utilidad: Los casos de uso sirven para medir la aceptación del usuario y del sistema, para encontrar fallas que un usuario detectaría al usar el sistema por primera vez

En este ejemplo, si el vaso de vino fue servido en vasos nuevos a estrenar, capaz el borde estaba afilado y se corta la boca el crítico. Un desastre: Me sacan una estrella como a Gusteau's



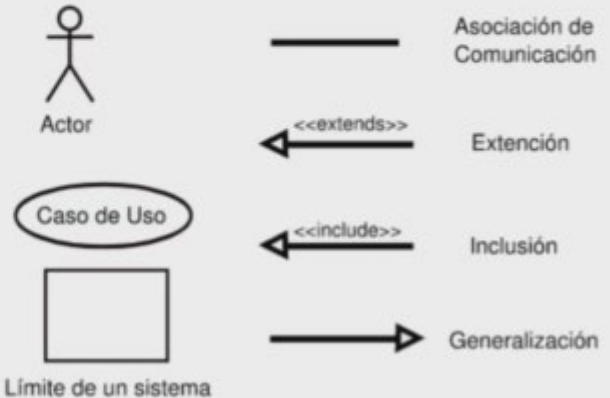
PREGUNTA DE PARCIAL: La calidad es el GRADO...
y el testing es el PROCESO...

Pruebas de Caja Negra



Casos de Uso

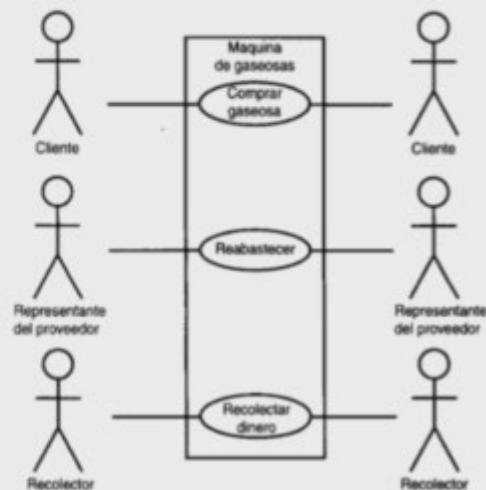
Elementos



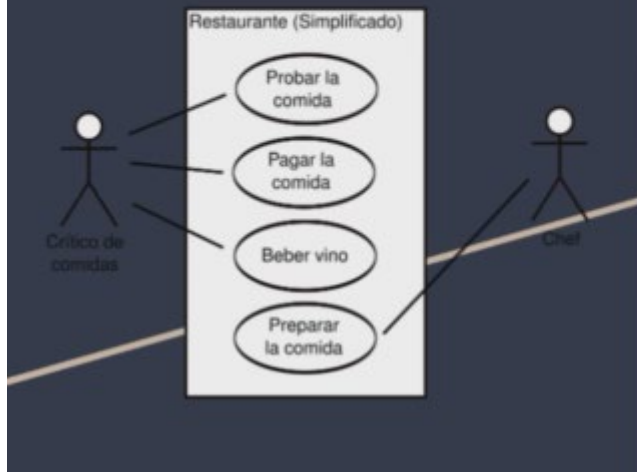
Pruebas de Caja Negra

Casos de Uso

Ejemplo



Pruebas de Caja Negra



Casos de Uso

- Escenario principal
- Escenarios alternativos / casos especiales

1 - Condiciones previas

2 - Estado final

Pruebas de Caja Negra

Especificación de caso de uso

Casos de Uso

Nombre de caso de uso	Indica el nombre del caso de uso. Normalmente, el nombre expresa el resultado objetivo y observable del caso de uso, como por ejemplo "Retirar efectivo" en el caso de un cajero automático.
Breve descripción	Describe el rol y el objetivo del caso de uso.
Flujo de eventos	Presenta el flujo básico y flujos alternativos. El flujo de eventos describe el comportamiento del sistema; no describe cómo el sistema funciona, los detalles de la presentación ni los detalles de la interfaz de usuario. Si se intercambia información, el caso de uso debe ser específico sobre lo que <u>ha transmitido</u> . Por ejemplo, en lugar de describir una acción como "el actor especifica información de cliente", indica que "el actor especifica el nombre y la dirección del cliente".
Flujo básico	Describe el comportamiento ideal y principal del sistema.
Flujos alternativos	Describe excepciones o desviaciones del flujo básico, como el modo en que el sistema se comporta cuando el actor especifica un ID de usuario incorrecto y la autenticación falla.
Requisitos especiales	Requisitos no funcionales que son específicos de un caso de uso pero que no se especifican en el texto del flujo de sucesos del caso de uso. Ejemplos de requisitos especiales incluyen los factores siguientes: requisitos legales y reguladores; estándares de aplicación; atributos de calidad del sistema, incluidos la usabilidad, la fiabilidad, el rendimiento y la capacidad de soporte; sistemas operativos y entornos; requisitos de compatibilidad y limitaciones de diseño.
Condiciones previas	Estado del sistema que debe estar presente antes del inicio del caso de uso.
Condiciones posteriores	Una lista de posibles estados del sistema inmediatamente después de que finalice el caso de uso.
Puntos de ampliación	Punto del flujo de eventos de caso de uso en el que se hace referencia a otro caso de uso.

Fuente:
<https://www.ibm.com/docs/es/elm/6.0.3?topic=cases-use-case-specification-outline>

Pruebas de Caja Negra

Casos de Uso

Caso de Uso	<< Nombre del CU >>		<< Identificador >>	
Actores	<< Listado de los actores participantes en el CU >> << Podemos indicar quien es el que inicia el CU usando (I) >>			
Tipo	<< Tipo del caso de uso >> << Primario, Secundario u Opcional <<Esencial o Real >>			
Referencias	<< Indicamos que requisitos se pueden incluir dentro de este CU >>		<< CU que tienen relación con este	
Precondición	<< Condiciones sobre el estado del sistema que tienen que ser ciertas para que se pueda realizar el CU >>			
Postcondición	<< Efectos que de forma inmediata tiene la realización del CU sobre el estado del sistema >>			
Autor	<< Esta línea se podría repetir para mantener una historia de cambios del CU >>	Fecha		Versión

Propósito

<< Descripción general del CU (Suficiente con una línea) >>

Resumen

<< Descripción de alto nivel del flujo normal (básico) del caso de uso (Suficiente con un pequeño párrafo) >>

Curso Normal (Básico)			
1	Actor 1: Acción realizada por el actor		
2	Actor 2: Acción realizada por el actor	3	Acción realizada por el sistema
		N	Cuando se realiza la inclusión de otro caso de uso lo representaremos de la forma. Incluir (CU Identificador, CU Nombre)
<< Se incluyen la secuencia de acciones realizadas por los actores que intervienen en el CU se usaran, flujos cortos, que describan el diálogo entre los actores y el sistema >>		<< Se incluyen la secuencia de acciones que realiza el sistema ante las acciones de los actores >>	
<< Se pueden añadir referencias a elementos de			

Cursos Alternos

1a	Descripción de la secuencia de acciones alternas a la acción 1 del Curso Normal
1b	<< Secuencia de los cursos alternos del CU >>

2. Hagamos foco en Caja Blanca y sus Técnicas.

Ejercicio

No va a tomar "haz un caso de uso", PERO puede preguntar DECIME CASOS DE USO y ahí decís TABLAS, ARBOLES, CASOS DE USO...

Caja Blanca.

1. Leer A
2. Leer B
3. $C = A + B$
4. Si $C > 10$ entonces
5. Imprimir C
6. Fin SI

Pruebas de Caja Blanca

Prueba de sentencias y decisión

Casos de prueba

Caso 1) $A = 4, B = 5$

Caso 2) $A = 3, B = 7$

Caso 3) $A = 9, B = 0$

Caso 4) $A = 5, B = 7$

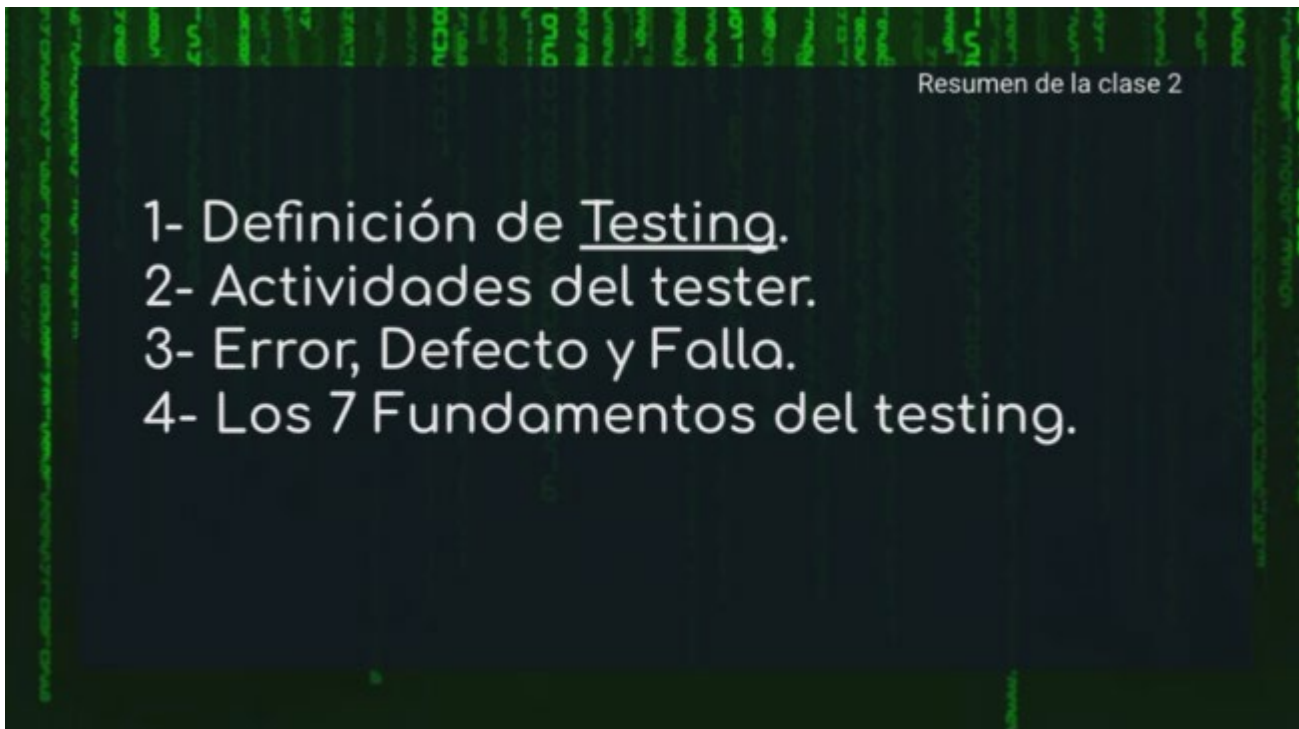
VALIDACIÓN Y VERIFICACIÓN:

VALIDAR que estén bien los requerimientos

VERIFICAR es ver que esté bien el desarrollo

QA = PROCESO

QC = PRODUCTO



PARCIAL: Humano comete un ERROR que introduce un DEFECTO que causa una FALLA

- 1- Tipos de pruebas.
- 2- Técnicas.Valores Límites o Frontera
- 3- Técnicas.Partición de equivalencias
- 4- Técnicas.Tablas de Decisiones
- 5- Técnicas.Casos de uso
- 6- Técnicas.Pruebas unitarias

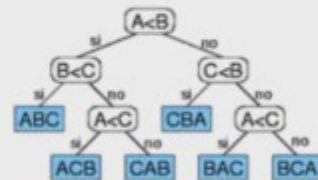
Árboles de decisión.

Definición y Aplicación.

Árboles de decisión

1. ¿Qué son?

Los árboles de decisión son diagramas que pretenden mostrar la gama de posibles resultados y las decisiones posteriores realizadas después de la decisión inicial.



Un árbol de decisión lleva a cabo una evaluación a medida que este se recorre hacia las hojas para alcanzar así una decisión

TODO árbol de decisión comienza con un NODO!!!

tabla de decisión PARA CUANDO TENGO MUCHAS VARIABLES QUE ORDENAR

Si tengo algo más simple? → Árbol de decisión

El árbol de decisión es una TÉCNICA PARA ENTENDER UN PROBLEMA!

Árboles de decisión

Figura	Nombre	Significado
	Nodo de decisión	Indica una decisión que se tomará
	Nodo de probabilidad	Muestra múltiples resultados inciertos
	Ramificaciones alternativas	Cada ramificación indica un posible resultado o acción
	Alternativa rechazada	Muestra una alternativa que no estaba seleccionada
	Nodo terminal	Indica un resultado definitivo

1. ¿Qué son?

El árbol de decisión suele contener **nodos internos**, **nodos de probabilidad**, **nodos hojas** y **arcos**.

- Un **nodo interno o decisión** contiene un test sobre algún valor de una de las propiedades.
- Un **nodo de probabilidad** indica que debe ocurrir un evento aleatorio de acuerdo a la naturaleza del problema, este tipo de nodos es redondo, los demás son cuadrados.
- Un **nodo hoja o terminal** representa el valor que devolverá el árbol de decisión y...
- ...finalmente las **ramas** brindan los posibles caminos que se tienen de acuerdo a la decisión tomada.

Árboles de decisión

3. Ventajas y desventajas

Ventajas

- Plantean el problema para que todas las opciones sean analizadas.
- Permiten analizar totalmente las posibles consecuencias de tomar una decisión.
- Proveen un esquema para cuantificar el costo de un resultado y la probabilidad de que suceda.
- Ayuda a realizar las mejores decisiones sobre la base de la información existente y de las mejores suposiciones.

Árboles de decisión

Una compañía de seguros nos ofrece una indemnización por accidente de 210.000\$.

Si no aceptamos la oferta y decidimos ir a juicio podemos obtener 185.000\$, 415.000\$ o 580.000\$ dependiendo de las alegaciones que el juez considere aceptables. Si perdemos el juicio, debemos pagar los costos que ascienden a 30.000\$. Sabiendo que el 70% de los juicios se gana, y de éstos, en el 50% se obtiene la menor indemnización, en el 30% la intermedia y en el 20% la más alta, **determinar la decisión más acertada.**

5. Ejemplos y ejercicios

Ejemplo

Resolver en 10'

(el profe prefiere que resolvamos otro):

Árboles de decisión

Se quiere determinar la nómina (sueldo) de los empleados de una empresa de acuerdo con estos criterios:

- Si el empleado es altamente productivo tendrá además un bono de productividad.
- Si el empleado es encargado de su grupo tendrá además un bono de encargado.
- Si el empleado ha cometido una infracción grave durante ese mes le será eliminado cualquier bono que pudiera tener.

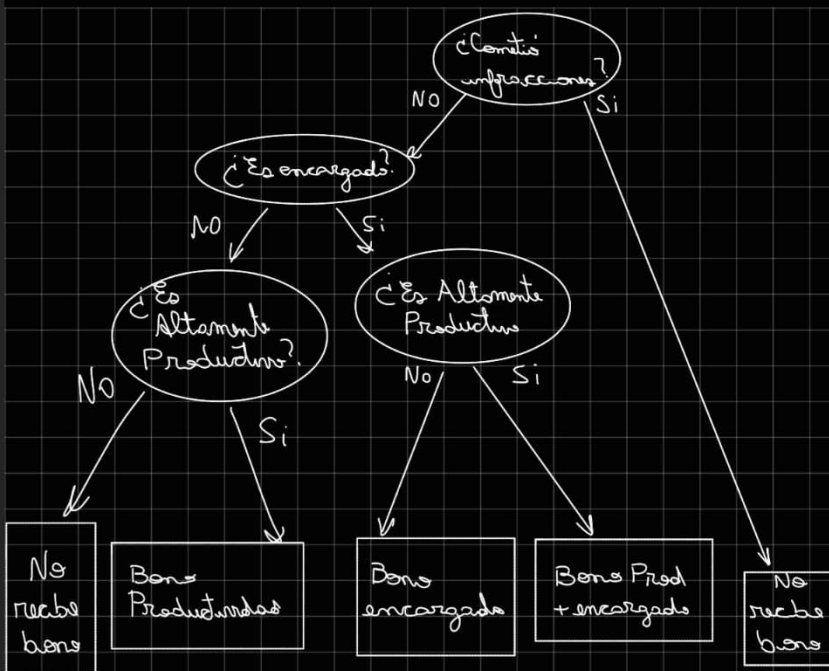
5. Ejemplos y ejercicios

Ejercicio

Objetivos:

- Realizar el árbol de decisión
- Comparar con la tabla de decisión realizada en la clase 3
- Identificar los casos de prueba resultantes

Tiempo estimado: 10'



6/6

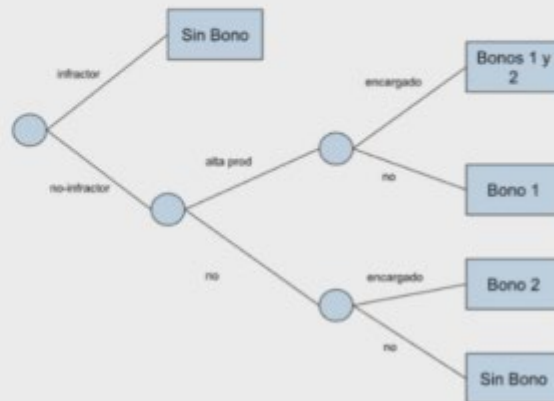
Árboles de decisión

Se quiere determinar la nómina (sueldo) de los empleados de una empresa de acuerdo con estos criterios:

- Si el empleado es altamente productivo tendrá además un bono de productividad.
- Si el empleado es encargado de su grupo tendrá además un bono de encargado.
- Si el empleado ha cometido una infracción grave durante ese mes le será eliminado cualquier bono que pudiera tener.

5. Ejemplos y ejercicios

Ejercicio



Árboles de decisión

Dada las siguientes reglas de negocio:

- Si la edad del cliente es menor a 18 años, entonces el resultado del crédito es No Viable.
- Si la edad del cliente es mayor o igual a 18 años y menor a 21, y el género del cliente es femenino, entonces el resultado del crédito es Viable.
- Si la edad del cliente es mayor o igual a 18 y menor a 21, y el género es masculino, entonces el resultado del crédito es No Viable.
- Si la edad del cliente es mayor o igual a 21 y menor que 80, y el género es femenino, entonces el resultado del crédito es Viable.
- Si la edad del cliente es mayor o igual a 21 y menor que 80, y el género es Masculino, entonces el resultado del crédito es Viable

5. Ejemplos y ejercicios

Ejercicio

Objetivos:

- Realizar el árbol de decisión
- Especificar **Supuestos y Fuera de alcance**
- Identificar los casos de prueba resultantes

Tiempo estimado: 10'

Árboles de decisión

Dada las siguientes reglas de negocio:

- Si la edad del cliente es menor a 18 años, entonces el resultado del crédito es No Viable.
- Si la edad del cliente es mayor o igual a 18 años y menor a 21, y el género del cliente es femenino, entonces el resultado del crédito es Viable.
- Si la edad del cliente es mayor o igual a 18 y menor a 21, y el género es masculino, entonces el resultado del crédito es No Viable.
- Si la edad del cliente es mayor o igual a 21 y menor que 80, y el género es femenino, entonces el resultado del crédito es Viable.
- Si la edad del cliente es mayor o igual a 21 y menor que 80, y el género es Masculino, entonces el resultado del crédito es Viable

Tiempo estimado: 10'

Árboles de decisión

Dada las siguientes reglas de negocio:

- Si la edad del cliente es menor a 18 años, entonces el resultado del crédito es No Viable.
- Si la edad del cliente es mayor o igual a 18 años y menor a 21, y el género del cliente es femenino, entonces el resultado del crédito es Viable.
- Si la edad del cliente es mayor o igual a 18 y menor a 21, y el género es masculino, entonces el resultado del crédito es No Viable.
- Si la edad del cliente es mayor o igual a 21 y menor que 80, y el género es femenino, entonces el resultado del crédito es Viable.
- Si la edad del cliente es mayor o igual a 21 y menor que 80, y el género es Masculino, entonces el resultado del crédito es Viable

5. Ejemplos y ejercicios

Ejercicio

Supuestos y Fuera de alcance

- *edad ≥ 80 , no viable*
- *¿No binarios?*
- *Antigüedad laboral > 1 año*
- *Válido presentando sólo DNI físico*
- *Tope crédito = \$ 100.000*

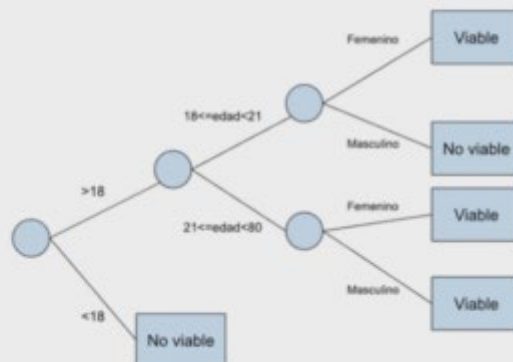
Árboles de decisión

Dada las siguientes reglas de negocio:

- Si la edad del cliente es menor a 18 años, entonces el resultado del crédito es No Viable.
- Si la edad del cliente es mayor o igual a 18 años y menor a 21, y el género del cliente es femenino, entonces el resultado del crédito es Viable.
- Si la edad del cliente es mayor o igual a 18 y menor a 21, y el género es masculino, entonces el resultado del crédito es No Viable.
- Si la edad del cliente es mayor o igual a 21 y menor que 80, y el género es femenino, entonces el resultado del crédito es Viable.
- Si la edad del cliente es mayor o igual a 21 y menor que 80, y el género es Masculino, entonces el resultado del crédito es Viable

5. Ejemplos y ejercicios

Ejercicio



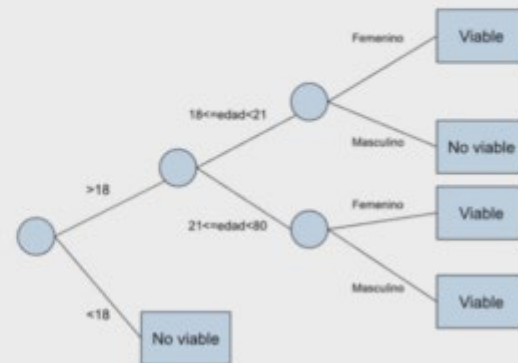
Árboles de decisión

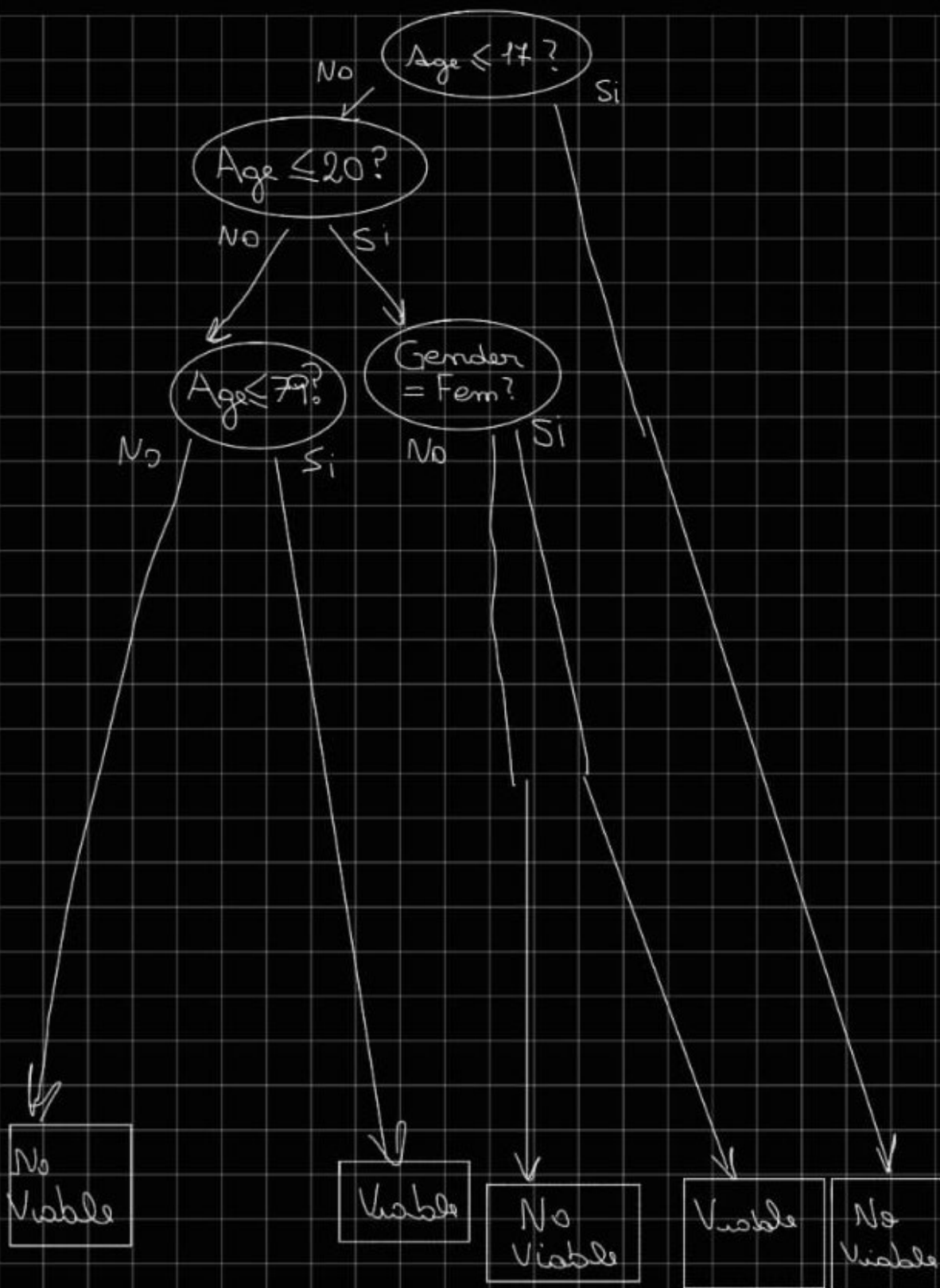
Dada las siguientes reglas de negocio:

- Si la edad del cliente es menor a 18 años, entonces el resultado del crédito es No Viable.
- Si la edad del cliente es mayor o igual a 18 años y menor a 21, y el género del cliente es femenino, entonces el resultado del crédito es Viable.
- Si la edad del cliente es mayor o igual a 18 y menor a 21, y el género es masculino, entonces el resultado del crédito es No Viable.
- Si la edad del cliente es mayor o igual a 21 y menor que 80, y el género es femenino, entonces el resultado del crédito es Viable.
- Si la edad del cliente es mayor o igual a 21 y menor que 80, y el género es Masculino, entonces el resultado del crédito es Viable.

5. Ejemplos y ejercicios

Ejercicio

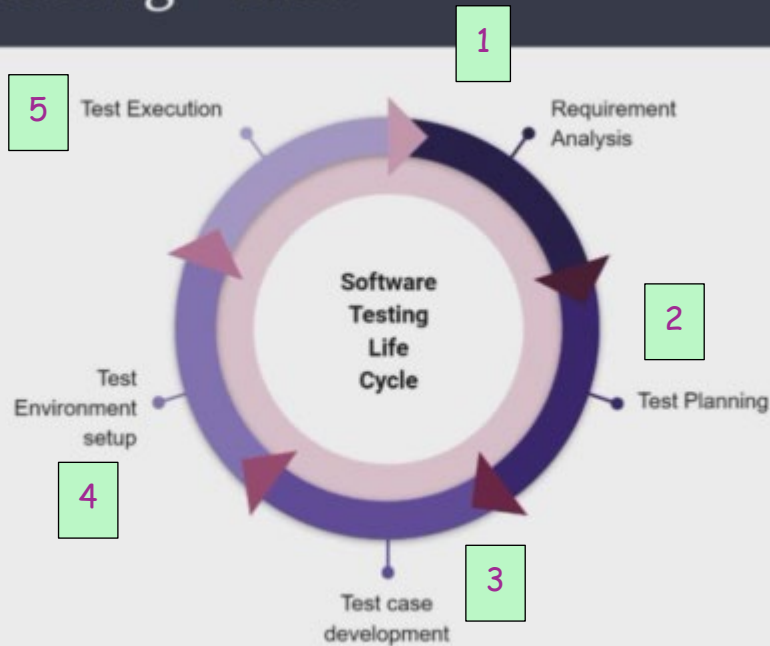




Proceso de testing.

Definición y Aplicación.

Proceso de testing - Base



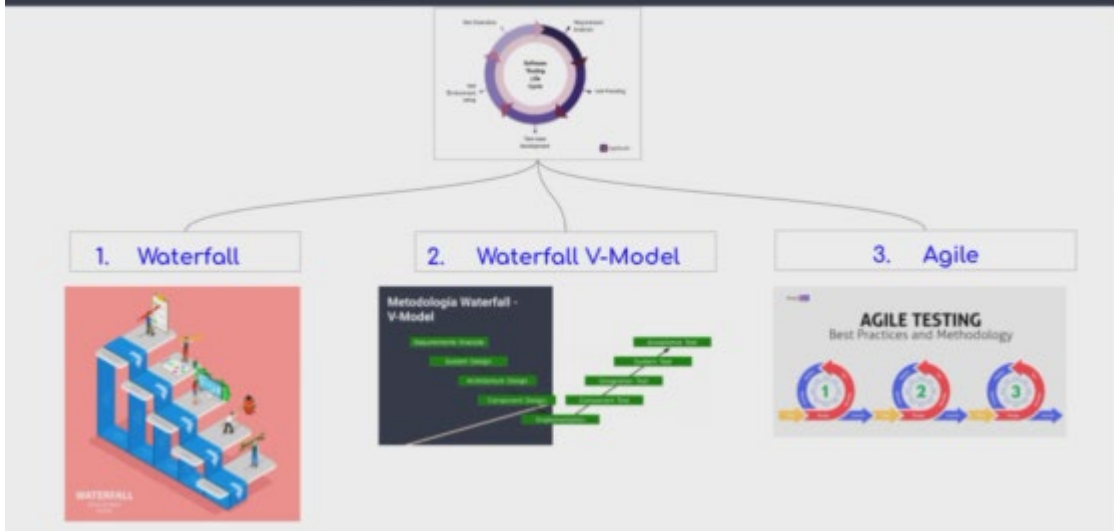
PREGUNTA PARCIAL: EXPLIQUEU EL PROCESO BASE DE TESTING. Y TENEMOS QUE DIBUJAR EL CÍRCULO (IDEALMENTE)

3) Los casos de prueba derivan de las TÉCNICAS (creo que se refiere a los resultados finales en cuadrados de los árboles)

AMBIENTES: Desarrollo, calidad, producción

5) Si me da FAIL, tengo que REPORTAR EL BUG!!!

Proceso de testing aplicados a metodologías



1) Waterfall:

Secuencial en cascada. Ya casi no hay compañías que trabajen así.

Se termina una cosa, y recién ahí se pasa a la siguiente etapa. Y el final es el final definitivo:
No hay retroalimentación:

Contra: Si había un error y no se agarró dentro de la misma etapa, queda arraigado a todas las demás y para el final es prácticamente incorregible sin sacrificar todo el demás progreso.

2) Waterfall V-Model:

Requisitos	_____	Pruebas de aceptación (UAT)
Diseño del sistema	_____	Pruebas del sistema
Diseño técnico	_____	Pruebas de integración
Codificación (fondo de la V)		



La diferencia clave con Waterfall clásico: el testing se *planifica* en paralelo al desarrollo, no se hace solo al final. Cada nivel de diseño define sus pruebas desde el principio.

3) Agile (LA MÁS IMPORTANTE Y MÁS UTILIZADA ACTUALMENTE):



¿Qué son los números? → SPRINTS

¿Qué son los sprints? → Iteraciones de TIEMPO donde se realizan pequeñas entregas

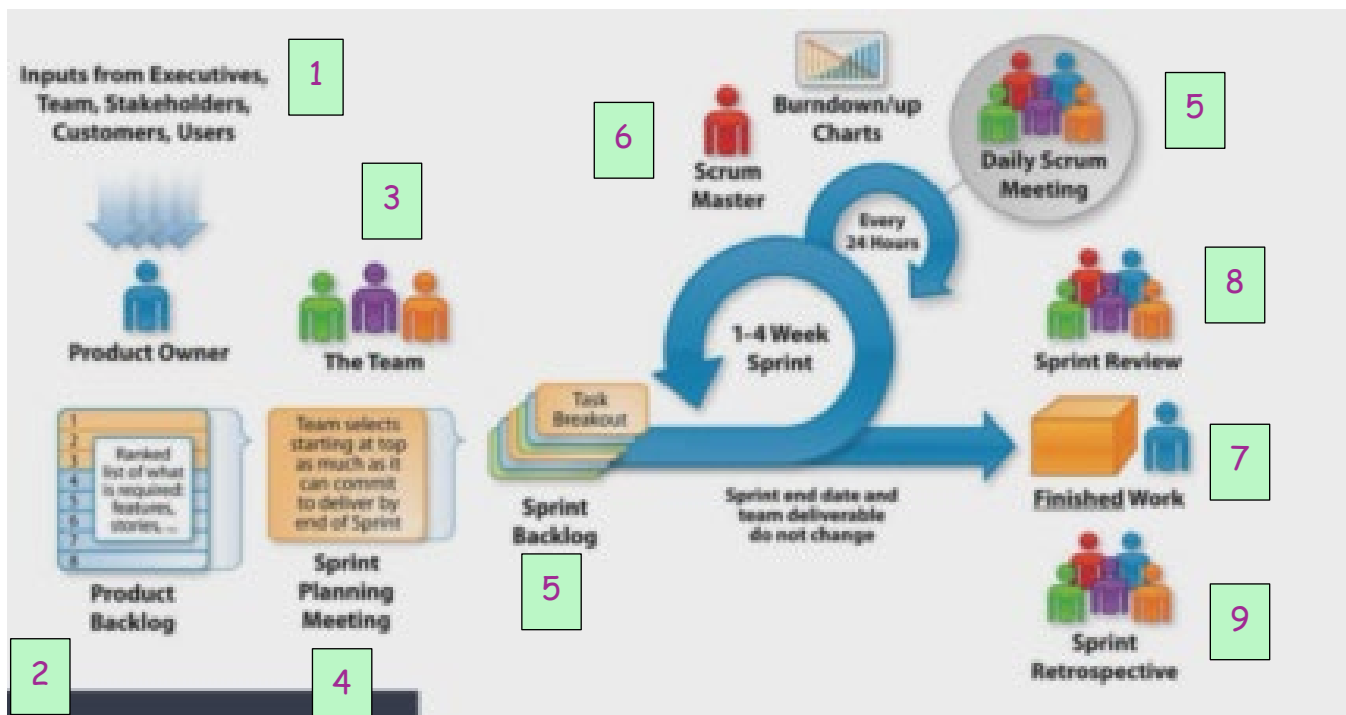
PREGUNTA TEÓRICA: EXPLIQUE LA METODOLOGÍA AGILE

Agile es una metodología de desarrollo iterativa e incremental donde el producto se construye y mejora en ciclos cortos, con entregas frecuentes, retroalimentación continua del cliente y capacidad de adaptarse a cambios en cualquier momento. Se opone al modelo Waterfall donde todo se planifica y entrega al final.

Scrum es el framework ágil más usado que implementa estos principios. Organiza el trabajo en ciclos fijos llamados **sprints** (1-4 semanas), al final de los cuales se entrega un incremento funcional del producto.

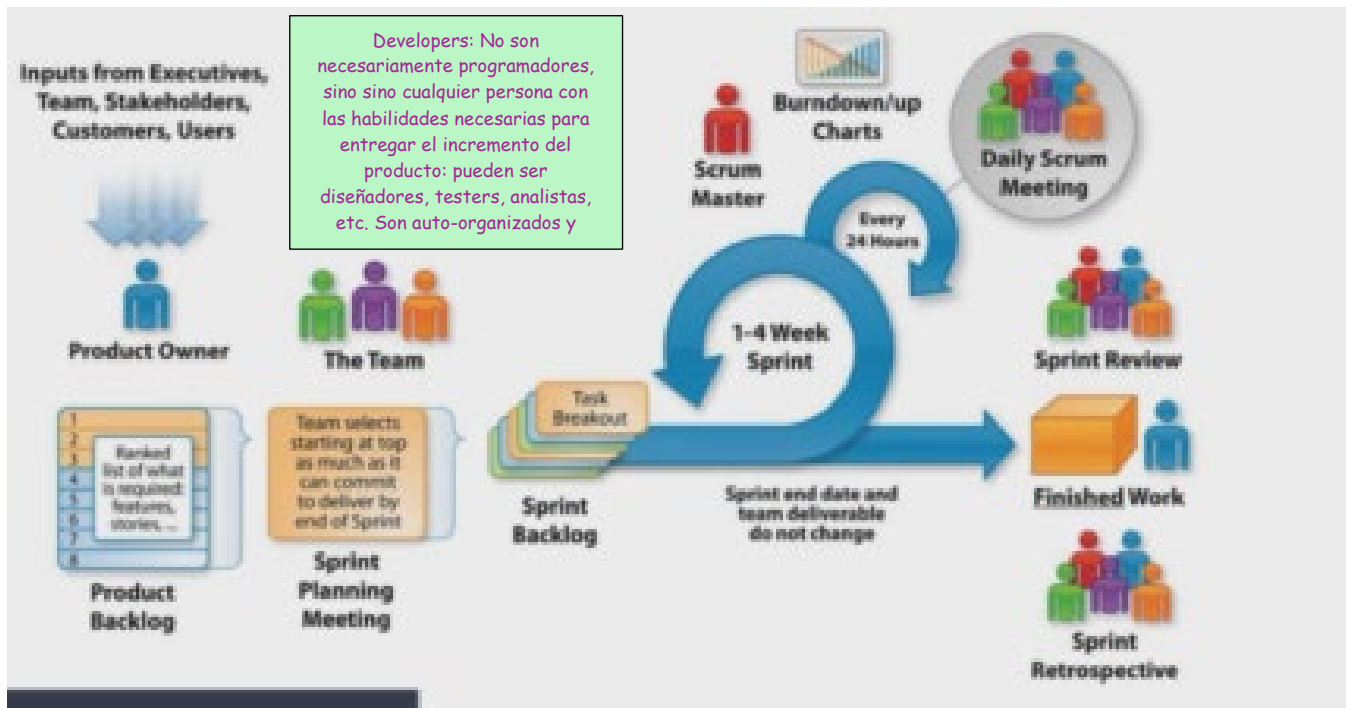
Roles:

- **Product Owner:** define y prioriza el **Product Backlog** (lista ordenada de todo lo que el producto necesita)
- **Scrum Master:** facilita el proceso y elimina impedimentos
- **The Team:** ejecuta el trabajo (no solo devs, también testers, diseñadores, etc.)



Lectura del diagrama Scrum — en orden:

1. Inputs → Ejecutivos, equipo, stakeholders, clientes y usuarios alimentan al Product Owner con necesidades y requisitos.
2. Product Backlog → El Product Owner arma una lista priorizada (*ranked list*) de todo lo que el producto necesita: features, stories, etc.
3. The Team + Sprint Planning Meeting → El equipo (no solo programadores, sino cualquier perfil necesario: testers, diseñadores, etc.) participa del Sprint Planning Meeting: selecciona ítems del Product Backlog empezando desde arriba (mayor prioridad), tomando todo lo que puede comprometerse a entregar al final del sprint. *Una vez por sprint.*
4. Sprint Backlog → Task Breakout → Sprint (1-4 semanas) → Los ítems seleccionados se descomponen en tareas concretas (Task Breakout) y forman el Sprint Backlog. Luego el equipo ejecuta el sprint. *La fecha de fin del sprint y los entregables no cambian una vez iniciado.*
5. Daily Scrum Meeting → Dentro del sprint, el equipo se sincroniza cada 24hs. *Iterativo diario.*
6. Scrum Master + Burndown/up Charts → El Scrum Master facilita el proceso y monitorea el avance del sprint mediante Burndown/up Charts. *Activo durante todo el sprint.*
7. Finished Work → Al finalizar el sprint se entrega un incremento funcional.
8. Sprint Review → Se presenta el Finished Work a los stakeholders. *Una vez por sprint, al final.*
9. Sprint Retrospective → El equipo reflexiona sobre el proceso para mejorar. *Una vez por sprint, después del Sprint Review.* Luego comienza un nuevo ciclo desde el Sprint Planning Meeting.



PARA EL EXAMEN: ES IMPORTANTE DESCRIBIR ESTE DIAGRAMA!!!

PRODUCT OWNER:

Según el profe (en su experiencia profesional, no como definición formal), el **PRODUCT OWNER** y el **TEAM** están en el mismo "equipo". Porque **NO HAY** jerarquía: No es que los del Team van a hacer lo que diga el product owner, sino que trabajan hacia el objetivo común de optimizar el negocio aka maximizar ganancias.

El product owner debe ser "vivo" para llevarse bien con el Team de modo de alcanzar exitosamente los objetivos, sino corre en riesgo su puesto laboral.

DAILY MEETING:

Si, todos los días, de 9 a 9:15 diciendo que hice ayer, que voy a hacer hoy, que planeo hacer mañana.

Y si tenía 3 objetivos y cumplí solo 2? En la **SPRINT RETROSPECTIVE** cubriremos los motivos por los cuales **NO** se alcanzaron (enfermedad, desperfectos mecánicos, etc).

SCRUM MASTER:

Se asegura de que se cumplan los objetivos de los equipos.

Pero los equipos deberían ser lo más autónomos posible.

Por eso, el Scrum Máster ideal, trabaja para lograr que se autonomicen los equipos, NO estancarse en ese puesto laboral.

El TESTING debe estar desde la PRIMER REUNIÓN!



¿Qué es JIRA?

Es una herramienta de gestión de proyectos y seguimiento de incidencias, especialmente diseñada para equipos ágiles.

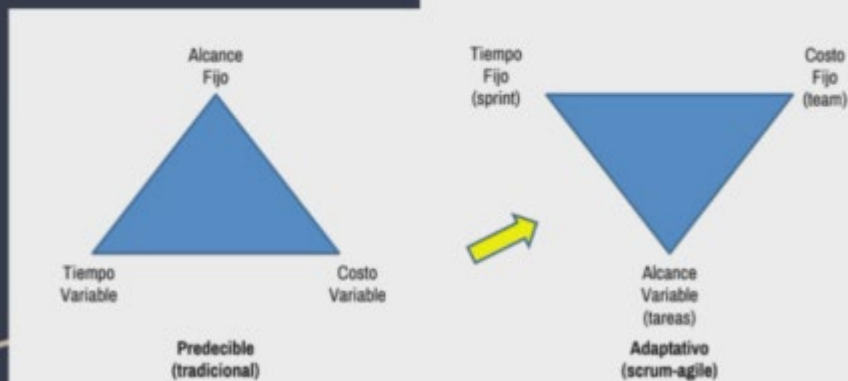
¿Y TRELLO?

Es de la misma empresa, más simple, menos "burocrático".

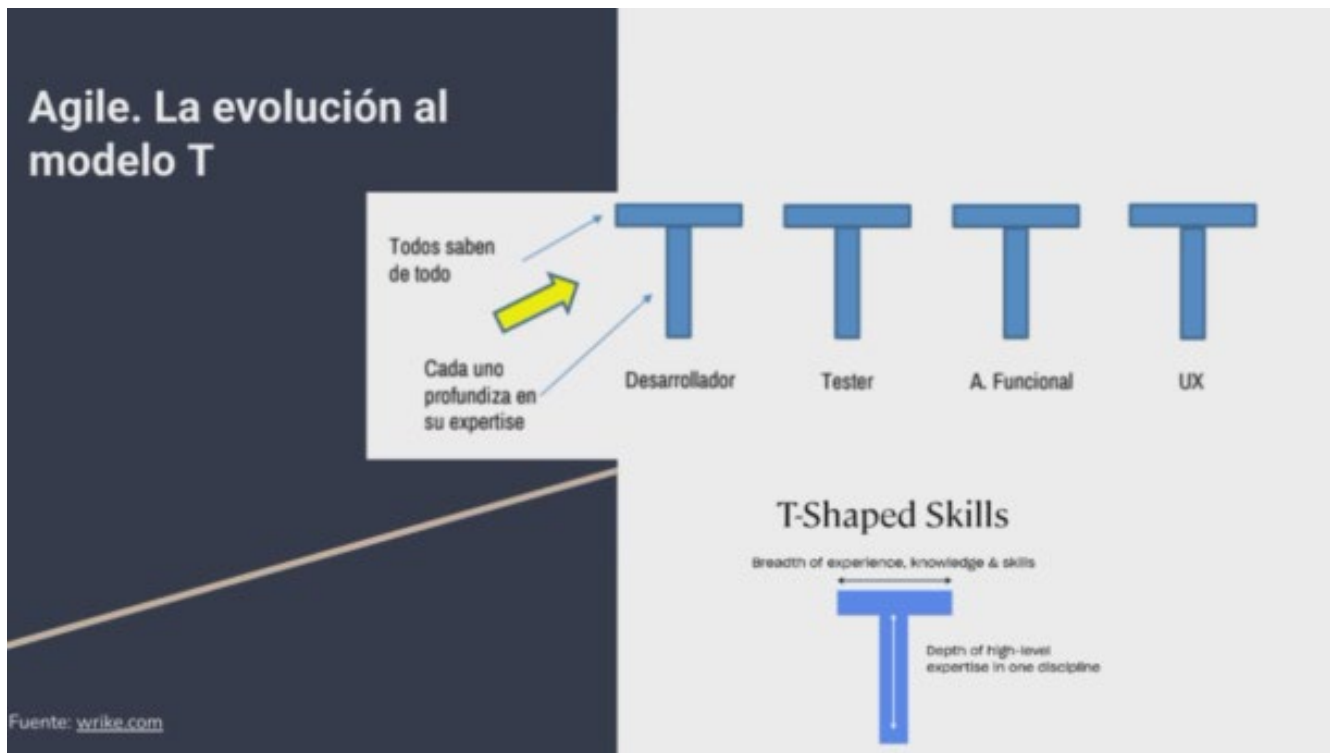
Agile vs Waterfall. Diferencias

Agile Testing	Waterfall Testing
Las pruebas no son una fase separada.	Las pruebas son una fase separada
Las pruebas se realizan junto con el desarrollo.	Las pruebas se llevan a cabo solo después de la finalización del desarrollo.
El equipo de desarrollo y el equipo de pruebas trabajan juntos.	El equipo de desarrollo y el equipo de testers trabajan por separado.
Los testers están involucrados en los requisitos.	Los testers pueden o no participar en los requisitos.
Las pruebas de aceptación se llevan a cabo después de cada iteración.	Las pruebas de aceptación se llevan a cabo solo al final.
Las pruebas de regresión se llevan a cabo después de cada iteración.	Las pruebas de regresión se llevan a cabo solo al final.
No hay retrasos en el tiempo entre la codificación y las pruebas.	Hay retrasos en el tiempo normales entre la codificación y las pruebas.
El plan de prueba se revisa después de cada ciclo/sprint.	El plan de prueba no se discute durante un ciclo.

Agile vs Waterfall. Diferencias



LO QUE SIGUE ACÁ NO ENTRA EN EL PARCIAL!!!



Introducción

Foto de panadería - TA 2do Cuatrimestre

- Esta rico este pan? No sé, debería...PROBARLO (guiño)
- Y es 2er cuatri, no 2do. Eso es un error. Quizás antes de subir el pptx debí...testearlo!

-
- Profe trabajó en Jam City - empresa yankee de videojuegos (frozen y demás imitaciones de candy crush).
 - Co-founder de 7FT ofrece soluciones AI
 - Solutions engineer manager en Oracle

// ¿Por qué probamos, en general? → Para descubrir errores impensados

Ejemplo de testing:

$3 \rightarrow (2X + 1) \rightarrow 7$

→ 7 // Pass

→ 7 // Fail

"Hola" → ∞ // ¿Cómo pruebo esto? → Es la pregunta que resolveremos en la cursada

La IA decrementó exponencialmente el tiempo de programación.

ChatGPT: Un Software, que adentro tiene un modelo GPT. Pero yo no tengo a la IA en sí, es una interfaz para el usuario para la cual pagamos una suscripción; NO corre localmente.

Consejo del docente: Empiecen a despegarse de lo técnico. Porque ya lo automatiza la IA. Enfocarse en la LÓGICA y habilidades blandas.

Van a empezar a aparecer soluciones a problemas. Pero yo para testearlas, no puedo no conocer el problema. El product owner, producto manager, etc.: Deben conocer TODOS el problema.

Si no conozco el problema? Estoy en el horno. Va para cualquier disciplina.

Ejemplo de algo que podríamos testear:

Un ERP (*Enterprise Resource Planning* o Sistema de Planificación de Recursos Empresariales) es un software de gestión integral que unifica, automatiza y centraliza los procesos clave de una empresa —finanzas, RR.HH., producción, cadena de suministro, ventas y compras— en una sola base de datos. Permite mejorar la eficiencia, reducir costos y agilizar la toma de decisiones con información en tiempo real.

Vamos a ver storytelling, debemos presentar...

"Nos interesa concurrir a la clase para..."

- PROMOCIONAR!!! El profe dice que NO quiere a NADIE en final.
- Va a haber sólo 1 parcial ESCRITO EN HOJA y 1 TP: Final sólo si nos sacamos 8 o más.
- Parcial: Definiciones CONCISAS y sin irse a las ramas (ej: diferencia entre quality testing y quality assurance).
También parte práctica: Árboles de decisiones, etc.
- Presentación: NO como en secundaria; con profesionalismo. Vamos a ir haciendo "sprint reviews". Allí veremos avances. Debe ser perfecto? No. INACEPTABLE QUE NO HAYA NINGÚN AVANCE! Estos Sprints son una nota conceptual. Si en la presentación oral nos sacamos 7, lo conceptual puede lograr que saquemos 8 y promocionemos.

NUEVO: Quiere que en el TP, 1 o 2 personas ("product owner/managers"), desarrollar la app de tomar exámenes digitalmente en UADE.

HERRAMIENTA: "LOVABLE" Para el prototipado. Pero es mala para el Development: eso es lo malo del "vibecoding": no sigue bien los patrones de diseño.

// La teoría es lo más valioso: Si no, le pedís cualquier cosa a la IA

¿Qué significa probar un producto?

→ Significa evaluar o experimentar dicho producto para conocer su funcionamiento, calidad, rendimiento, utilidad y otros aspectos relevantes.

→ En síntesis, significa EVALUAR para CONOCER.

→ FUNCIONALIDAD = ¿El sistema hace lo que tiene que hacer? (correctitud)

→ RENDIMIENTO = ¿Lo hace bien bajo carga/estrés? (velocidad, estabilidad, escalabilidad)

¿Qué pasa si no testeó?

- Falla mi producto. Impacta negativamente al negocio.

- No hay tiempo para testear todo. En nuestra vida profesional, tendremos deadlines acotadas. Ejemplo: Las vacunas de COVID. Se probaron? Si, pero no como deberían haberse probado.

¿Qué pasa si se cae mercado pago?

-Ej: Se cae Mercado Pago al mediodía. Hay un kiosco frente a un colegio donde los chicos iban a almorzar. Los chicos no pueden comprar, el kiosco no puede vender. Caen las acciones de Mercado Pago.

Consecuencias de no testear:

CrowdStrike: Quedó mucha gente varada en el aeropuerto

The July 19, 2024, CrowdStrike outage, caused by a faulty [Falcon sensor software update](#) for Windows, grounded global air travel, affecting thousands of flights. Major carriers (Delta, American, United) saw massive disruptions, with long terminal queues and manual check-in. The incident, which caused over \$500 million in damages for Delta, prompted lawsuits and highlighted vulnerabilities in aviation IT.

Profesor: "Los profesionales más requeridos son los más consultivos por sobre los técnicos. Para ser consultivo, tenes que tener pensamiento crítico, curiosidad, entender los problemas; Y esto es todo lo contrario a... ver un tiktok x2. Traten de, más allá de lo técnico, medir lo que hacen en su día.

¿Por qué, aunque haga QA, hay cosas que fallan en el release?

- Porque ES IMPOSIBLE TESTEAR TODO!

7 Fundamentos del testing:

//si yo hago por ejemplo una app para votar por celular en argentina: Yo digo, pueden votar los mayores de 16... pero esta mal, porque debe ser mayor o igual a 16. Pruebo todas las edades? No, pruebo 15, 16, 17, 99, 0, -1. Hay que OPTIMIZAR, probar VALORES LÍMITES

PREGUNTA DE PARCIAL: ¿Cuál es la diferencia entre VALIDAR y VERIFICAR?



Tengo un negocio. Mi negocio tiene un problema.

Tengo que hacer el documento SPEC (especificación).

Es la interpretación de una persona humana sobre cierto problema. Pero al ser humana, va a tener ERRORES.

PRIMERO VALIDO, LUEGO DESARROLLO.

VALIDAR = COMPROBAR QUE CUMPLA CON LAS NECESIDADES DEL NEGOCIO

VERIFICAR = COMPROBAR QUE EL PRODUCTO ENTREGADO CUMPLA CON LAS NECESIDADES PREDEFINIDAS

//Qué es un release? Es la versión de la app

//El QA valida? Depende de la cultura del equipo. "Cuando tengo una release, recién te doy y probalo..." Pero lo ideal es que testeen desde el inicio. Sino se pudo haber arrastrado un problemón.

//Tengo que ESPECIFICAR y DESARROLLAR con calidad. DESARROLLAR con calidad significa: especificar patrones, escalable, reutilizable, documentado, pruebas unitarias. La calidad es responsable del product owner, del quality assurance... de TODOS. Hay un error en la app releaseada: es culpa de QA? NO, es culpa de TODOS. Ej: "Votan los mayores a 16..." PERO ERA MAYOR O IGUAL! estaba mal ESPECIFICADO. NO ERA CULPA DE QA.

PREGUNTA DE PARCIAL: (creo que era lo de arriba, lo de de quien es culpa la calidad)

//Metodologías AGILE: Waterfall está discontinua. En los AGILE, si algo está mal, al ser iterativo, lo puedo solucionar.

PREGUNTA DE PARCIAL: DIF ENTRE VALIDAR Y VERIFICAR

SPRINT: Rango de tiempo (usualmente 2) en el cual se plantea un objetivo que debe cumplirse idealmente completamente, si fue parcial se discuten los retrasos, y si no se cumplió se replantea la estrategia.

Una arquitectura son los componentes tecnológicos que integrarán la solución.

Cuando el equipo de devs presente, una slide deben ser los componentes tecnológicos.

Una base de datos es un repositorio que almacena informaciones y esta compuesto por tablas. Un block de notas puede ser una base de datos.

App = software, aunque normalmente se vincula app a mobile

...pero la BIOS no es SW sin ser APP?

Todo app es software, pero no todo software es app.

	App	Software
BIOS	X	✓
Drivers	X	✓
SO (Windows/Linux)	X (debatible)	✓
App mobile	✓	✓
App web	✓	✓

App implica que tiene una interfaz de usuario y resuelve una necesidad de usuario final.

La BIOS, los drivers, el firmware → son software de sistema/bajo nivel, sin interacción directa con el usuario → no son apps.

Conclusión: "app testing" en la práctica refiere a testear software orientado al usuario final, no software de sistema.

UI → cómo se **ve** (diseño visual, botones, colores, tipografía)

UX → cómo se **siente** usarlo (flujo, experiencia, usabilidad)

//el testing se hace sobre UX, en UI solo se debería ver si anda tal botón o cosas menores.

Chatbot = determinístico. Ej; Sacar turno en OSSEG. No necesariamente aplica IA. Está Hardcodeado.

QA = Quality Analyst? Quality Assurance? Diferencia? Lo veremos en clase...

PREGUNTA DE PARCIAL: Proceso base de testing.

Que suelen hacer? Poner el ciclo de vida del desarrollo de software. ESTO ESTA MAL!
(aunque es clave, por algo está en el pptx. PERO NO ES PARA ESTA PREGUNTA)

que quiero decir con ambiente relacionado a la arquitectura? dev staging production

Clase 2

Recapitulemos:

Antes de cualquier cosa, necesito saber los REQUERIMIENTOS del proyecto. Porque quizás

Cashflow: Para saber cuándo y dónde voy a necesitar fondeo: En qué momento necesitare momento, y cuando. Que va a generar entrada de dinero en mi compañía? La facturación, mis ventas (ej: vendi un proyecto pero todavía no lo facture), inversiones; y las salidas de dinero? Sueldos, órdenes de compra, facturas de proveedores... Y todo esto sucede en distintos momentos. Si tengo más cantidad en salida que en entrada; tenemos un problema

SI NO ENTENDEMOS EL PROBLEMA → ESTAMOS EN EL HORNO!!!

PREGUNTA DE PARCIAL: Calidad es el grado (en el que se cumplen los requerimientos?)

Los fundamentos que veremos en esta materia son claves porque podemos saber los lenguajes, la sintaxis; pero sin saber que desarrollamos, por qué, y la mejor estructura para plantearlo, estamos en el horno

PREGUNTA DE PARCIAL: Diferencia entre validar y verificar?

Primero se valida, después se verifica.

Que verifico? Que el producto cumpla con los requerimientos

Y la validación previa era que los requerimientos cumplan las necesidades del negocio

///

DEVELOPERS: Son los equipos que contribuyen con la construcción de una solución. Son programadores, testers, dev ops, especialistas en bases de datos, especialistas en distintas arquitecturas

Arquitectura: Diagrama de los componentes de una solución

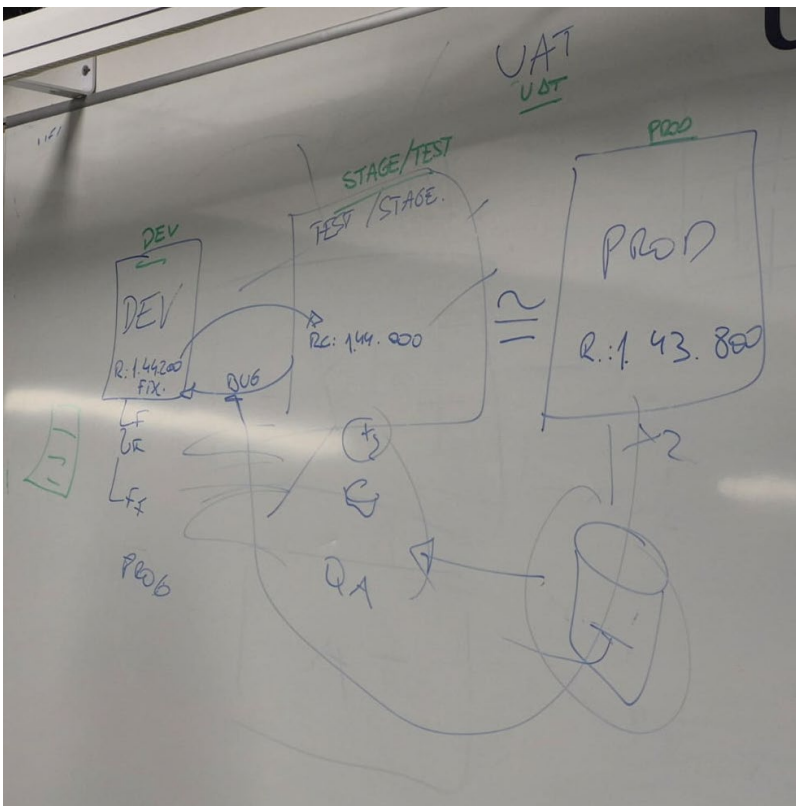
Patrones: Soluciones reutilizables a problemas comunes y recurrentes en el diseño de software. Son como "plantillas" o recetas probadas, no código concreto sino esquemas que adaptás a tu situación.

VALIDO: Que el producto cumpla con las necesidades del negocio

VERIFICO: Que el producto entregado cumpla con las necesidades predefinidas

SKILLS CLAVE: inglés, 1 lenguaje de programación (preferiblemente Python), SQL, Excel, manejar arquitecturas, manejar patrones, entender el negocio.

Mínimo entorno de desarrollo:



Entornos: TEST - QA - PRODUCCIÓN. Y entre medio, viajaba el RELEASE.

IMPORTANTE: ENTREGAR TODO CON LINKS! FULL NUBE, NADA DE .DOC O .PDF

Vamos a trabajar con TRELLO

Si no tengo buena infraestructura, se impacta la calidad.

Si tengo malos equipos físicos, mal equipo: Se arruina la operación.

Ej: Compañera cuya empresa en vez de trabajar en la nube, tiene un servidor físico que anda mal. El jefe que pagó el servidor se niega a reconocer que esta obsoleto, entonces ella y sus compañeros se ven obligados a respaldar todo en sus propios equipos.

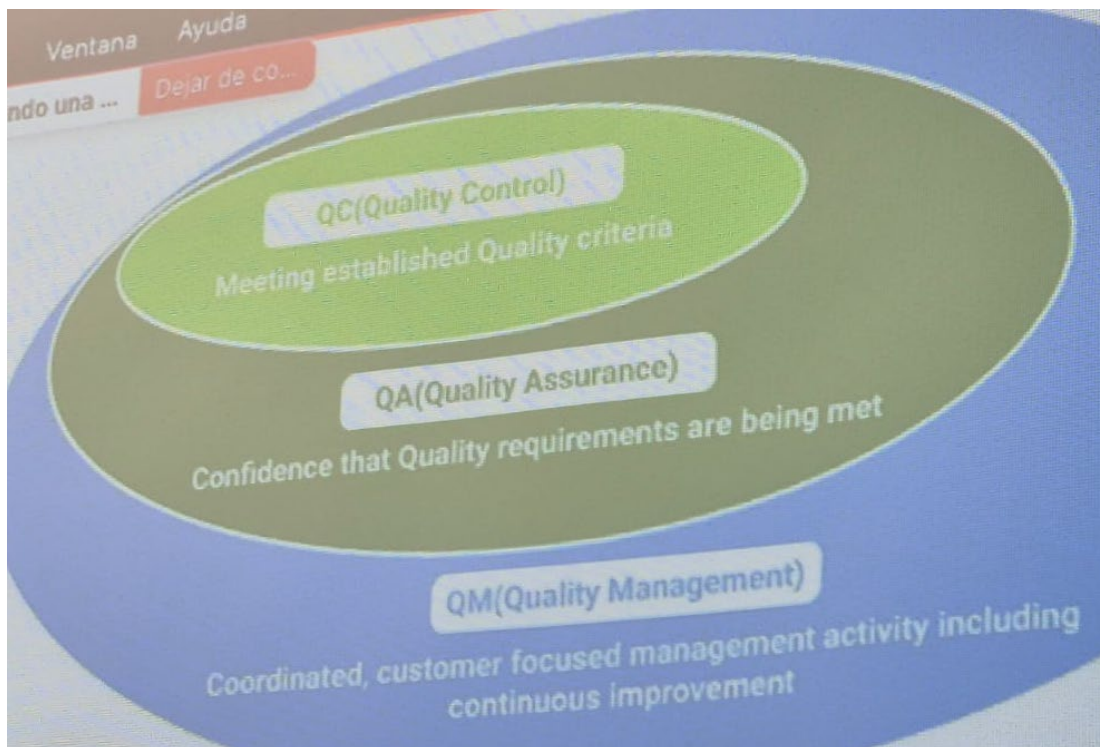
Globant: En QC/QA dicen "busco QA", y se refieren a tester. Pero también le dicen QC, ¿Cuál es?

PARCIAL: DIFERENCIA ENTRE QA Y QC:

QC = Prueba el PRODUCTO (Quality Control): Recibe el producto y debe probarlo de acuerdo al TEST PLAN siguiendo TEST CASES (establecidos por el QA)

QA = Prueba el PROCESO (Quality Assurance)

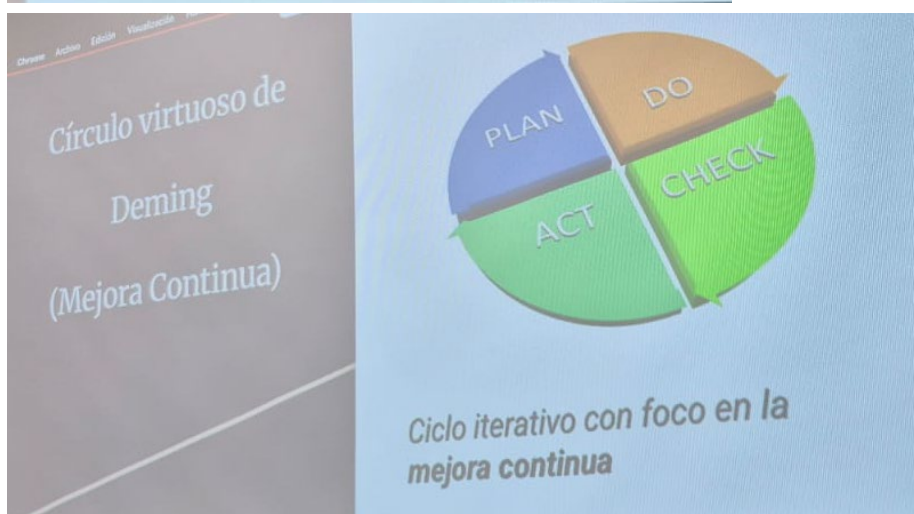
(no va en el parcial) QM = Quality Manager: Administra a los QA



2do parcial: El aula se transforma en un "laboratorio". Vamos a desarrollar. Va a haber PMs, que definan SPECS. A los que tengan mas experiencia como devs, van a desarrollar. Los demás van a hacer grupos de a 3 o 4 (hacer networking en la clase)

QC: orientado a productos y con foco en la detección de defectos.

QA: orientado a procesos y con foco en la prevención de defectos.



ESCENCIAL el ciclo de mejora continua: No hacer GRANDES entregables, sino muchos más chicos para presentar, corregir y mejorar. Que sean ITERATIVOS.

En la retrospectiva, te dan las "oportunidades de mejora" (lo que hiciste MAL).

Debemos ser fríos en esas reuniones: No tomarnos las cosas personales, pues e para peor.

Mejora continua: Saber que los procesos de mejora de software son iterativos y que debe haber una retrospectiva de cosas a mejorar pero también reconocer lo que esa bien.

LOVABLE para prototipado está muy bueno.

//INTERESANTE: Tendremos en una clase un hackathon de IA para hacer una app desde cero.

//La IA toma notas de las sprint reviews: es de alta necesidad tener la retrospectiva y nuevos requerimientos. PERO podría llegar a halucinar! SIEMPRE hay que revisar.

También te podría filtrar datos privados de tu negocio siempre que no corra localmente la IA.

2DA MITAD DE CLASE: VIENE CASI TODO LO DEL PARCIAL!!!

International Software Testing Qualifications Board.

Qué es ISTQB?

Es la organización líder mundial en la certificación de competencias en tests de software.

ISTQB: (International Software Testing Qualifications Board) Es la organización líder mundial en la certificación de competencias en tests de software

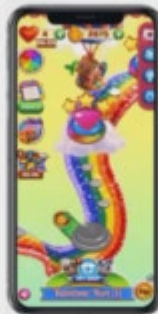
DAU: Daily Active Users. Va cayendo: Al principio entran muchos, después va cayendo. EXCEPTO a los que les encantó el juego. Puedo inferir en el comportamiento del usuario: poniendolo mas contento o mas triste, como hacen las redes sociales.

Pruebas de Software.

Propósito de las aplicaciones



- Marca
- UX/UI



- Marca
- UX/UI
- Fidelidad
- Facturación



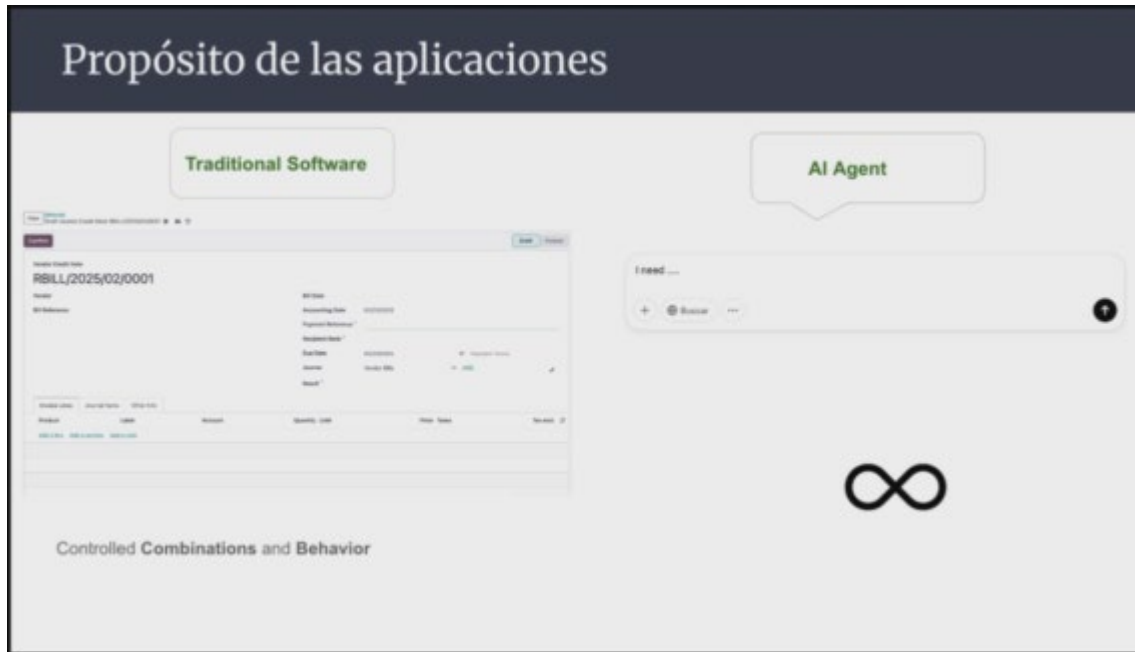
- Marca
- UX/UI
- Fidelidad
- Facturación
- Alto Impacto cotidiano

Impactos

Si se cae la app UADE, sólo tardamos más en entrar/salir (medio impacto cotidiano)

Si cae cookie jam - bajo impacto

Si cae MP - alto impacto



No es lo mismo probar con técnicas de software tradicional que con técnicas de IA. Las de IA estan cambiando todos los días: Nosotros en esta clase veremos las tradicionales.

Si pongo un código de proveedor que no existe, debería decirme "proveedor no existente". Lo esperado es que el sistema **CONTROLE**.

Prueba de software

Fuente: ISTQB

1. El proceso

Las pruebas se deben preparar, se verifica el resultado, se prueba el código del producto.

Abarca actividades durante todo el ciclo de vida.

Cuanto más tarde encontremos errores, será más costoso.

Cuanto más temprano, durante la etapa de requisitos, tendremos el software correcto y a menor costo.

Abarca pruebas estáticas y dinámicas.

Se planifica antes-durante-después de la ejecución.

- Si yo espero 3 meses, el programador ni se acuerda el código que hizo: Entonces perdemos tiempo en que lo vuelva a interpretar para arreglar el error.
- También pierdo todo lo que se construyó sobre ese error, por eso hay que arreglarlo lo antes posible y es mejor hacer entregas concisas



The slide is titled "Prueba de software" in a large, white, sans-serif font on a dark blue background. A thin, light-colored diagonal line runs from the bottom left towards the center. To the right, on a light gray background, is the section "2. Los objetivos" followed by three bullet points: "Revisar el diseño para ver si cumple con los requisitos.", "Revisamos si el producto es el adecuado.", and "Detectar defectos.". At the bottom left of the dark blue area, it says "Fuente: ISTQB". At the bottom right of the light gray area, the number "25" is visible.

Prueba de software

2. Los objetivos

- Revisar el diseño para ver si cumple con los requisitos.*
- Revisamos si el producto es el adecuado.*
- Detectar defectos.*

Fuente: ISTQB

25

Va a salir con errores. Un software siempre tiene errores; tenemos que MINIMIZARLOS.

¿Y la Calidad?

Fuente: ISTQB

IEEE:

"Calidad del software es **el grado** en el cual un sistema, componente o proceso se ajusta con las **expectativas** o **necesidades** del cliente o **usuario**"

ISO 8402:

"Conjunto de propiedades y características de un producto o servicio, que le confieren aptitud para satisfacer una necesidad, explícita o implícita"

W. E. Deming:

"Es un grado predecible de uniformidad y fiabilidad a bajo coste y adecuado a las necesidades del mercado"

26

PREGUNTA DE PARCIAL: QUE ES LA CALIDAD? HAY 3 DEF - USAR LA QUE LE GUSTA AL PROFE

La que más le gusta:

Según IEE: Calidad de software es el grado en el cual un sistema, componente o proceso se ajusta a las expectativas o necesidades de cierto usuario.

Dependo del proceso para saber el grado de calidad

Testing. Qué es. Para
qué testeamos.
Su Visión.

Es el proceso que consiste en todas las actividades del ciclo de vida, tanto estáticas como dinámicas, relacionadas con la planificación, preparación y evaluación de productos de software y productos relacionados con el trabajo para determinar que cumplen los requisitos especificados, para demostrar que son aptos para el propósito y para detectar defectos.

ISTQB

Definición de Testing:

Es el proceso que consiste en todas las actividades del ciclo de vida, tanto estáticas como dinámicas, relacionadas con la planificación, preparación y evaluación de productos de software y productos relacionados con el trabajo para determinar que cumplen los requisitos especificados, para demostrar que son aptos para el propósito y para detectar defectos.

Testing. Qué es. Para
que testeamos.
Su Visión.

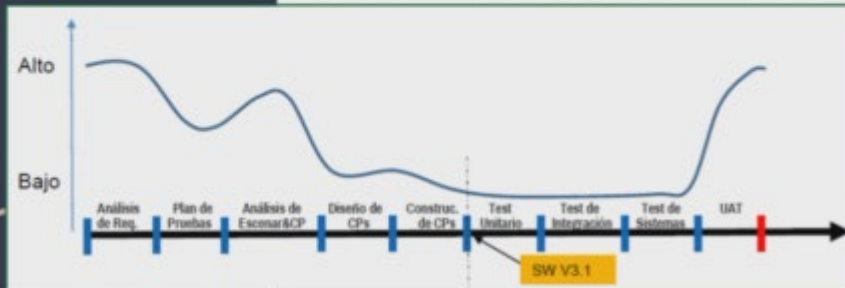
- Para detectar y resolver problemas tan pronto sea posible
- Para verificar que las pruebas cumplan con el requerimiento
- Para generar compromiso en los usuarios
- Para trabajar orientado al "cero defecto" desde el principio

"cero defectos" es una meta a orientarse PERO es inalcanzable.

Testing. Qué es. Para qué testeamos.

Su Visión.

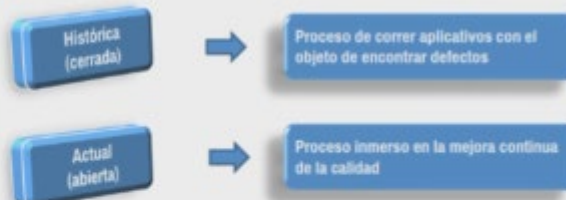
Nivel de compromiso o interrelación del Usuario/Cliente



¿Cuándo trabajo si estoy lleno de meetings para saber cómo voy? Es horrible...
-profe

Testing. Qué es. Para que testeamos.

Su Visión.



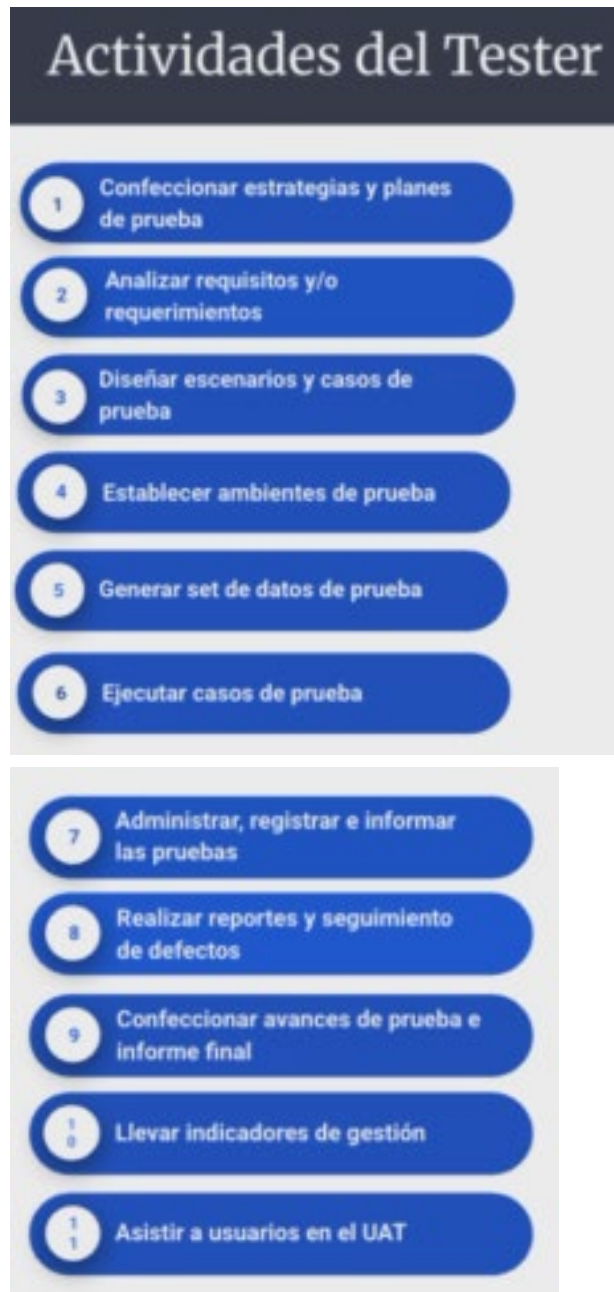
Antes (cerrada) → Proceso de correr apps y hallar errores

Ahora (abierta) → Proceso inmerso, de mejora continua de calidad

//Antes → Se testeaba en la etapa final del desarrollo.

//Ahora → Testing en todo el proceso: No solo busco errores; apunto por la CALIDAD GENERAL. A

//Hay un problema de cotización actualmente en la tecnología: La misma solución te la dan a 2k o 10k, y no necesariamente la más cara es la mejor.



"Yo pruebo jueguitos..."

85% de jugadores de juegos tipo candy crush → mujeres mayores a 35 en USA

Últimamente viene cayendo: No tienen buena retención

El problema del entretenimiento es el TIEMPO! Tengo 24 horas. Netflix compite con mis horas de sueño.

TIKTOK vino a romper nuestros receptores de dopamina, y además arruinó a todos los otros medios de entretenimiento: Ya nos parece mucho sentarnos 2 horas a ver una peli.

La taréa de un tester no termina en ejecutar las pruebas: Tengo que informar que lo probe, y reportar los defectos que halle. Debo hacer una prueba de UAT (User Acceptance Testing)



PREGUNTA DE EXAMEN: DIFERENCIA ERROR-DEFECTO-FALLA

(El error en las pruebas no solo los comete el QA, porque pueden haber estado mal dado los requerimientos)

Error, defecto y falla.

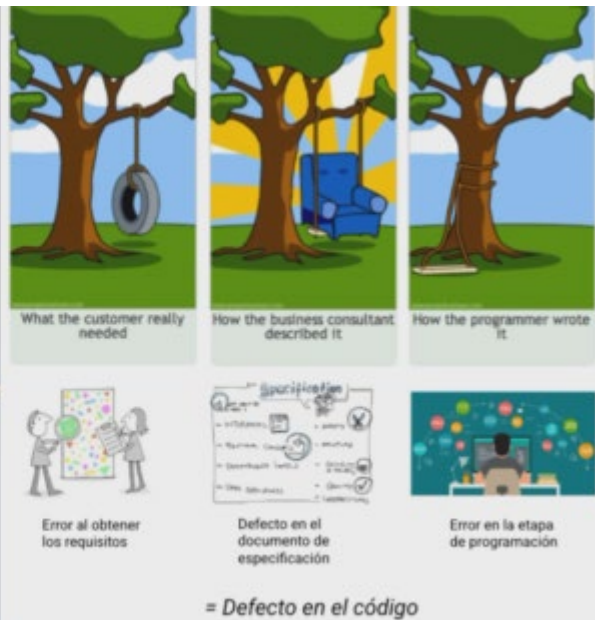


Si se ejecuta el código defectuoso se **podría** generar una falla.

PREGUNTA DE EXAMEN: DIFERENCIA ERROR-DEFECTO-FALLA - RTA ARRIBA
GRAFICO PUEDO CONTESTAR DIBUJANDO EL GRAFICO

Defecto = Bug

Error, defecto y falla.



Los 7 fundamentos del testing.

Definiciones.

PREG PARCIAL: Nombre y explique los 7 fundamentos del testing

Los 7 fundamentos del Testing.

- 1 - La prueba muestra la presencia de defectos
- 2 - La exhaustividad es imposible
- 3 - Detección temprana de defectos
- 4 - Agrupamiento de defectos
- 5 - Paradoja del pesticida
- 6 - Es dependiente del contexto
- 7 - La ausencia de defectos es una falacia

Los 7 fundamentos del Testing.

- 1 - La prueba muestra la presencia de defectos, no su ausencia.**

La prueba puede mostrar la presencia de defectos, pero no que no haya defectos. La prueba reduce la probabilidad de que queden defectos no descubiertos en el software pero, incluso si no se encuentran defectos, el proceso de prueba no es una demostración de la corrección.

ISTQB

Si yo ejecuté una prueba y no hubo errores, no significa que no haya; sólo que en esa prueba específica no encontré.

Si me dicen: Tienes 2 años para sacar este producto, y te pago 2.000 testers.

Pues puedo lograr que la cantidad de errores sea ínfima, probando casi todo muchas veces: Pero nunca se puede probar todo ni asegurar que no tenga errores.

Los 7 fundamentos del Testing.

2 - La exhaustividad es imposible

Opciones → *Probar todo*
Probar nada
Probar una parte

Ejemplo 1: probar datos en campo de **1 dígito**

Valores válidos: 0-9
Rechazar todo valor inválido:

- 26 mayúsculas
- 26 minúsculas
- 6 caracteres especiales (.,:!"')

Total de casos a probar = $10 + 26 + 26 + 6 = 68$

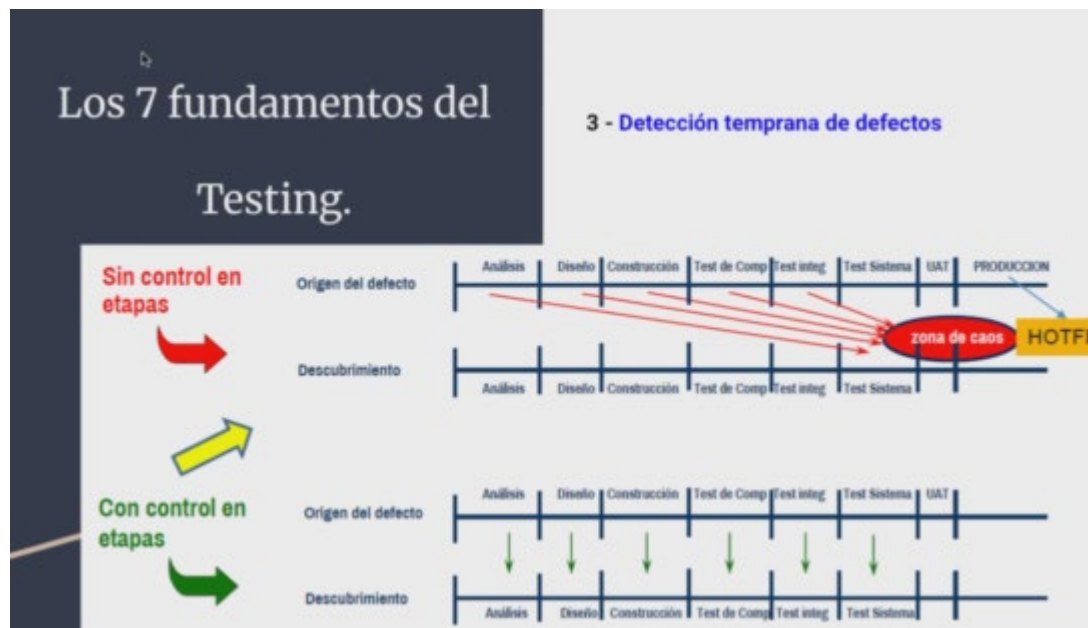
No se puede probar todo: Se prueban los casos críticos: En este caso, -1,0,1, 8, 0, 10

Los 7 fundamentos del Testing.

3 - Detección temprana de defectos

```
graph TD; A[Requisitos] --> B[Diseño]; B --> C[Construcción]; C --> D[Pruebas de aceptación];
```

Historia de por que se le llama bug a las fallas: Cuando las computadoras eran con cintas, eran ellos quienes hacían las fallas.



Los 7 fundamentos del Testing.

4 - Agrupamiento de defectos

Asociado a la complejidad del requerimiento, del proceso de negocio o a la complejidad ciclomática del software. Es común ver que a mayor complejidad es mayor el nivel de defectos hallados.

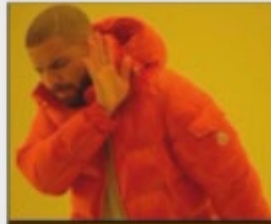
ISTQB

El **principio o regla de Pareto** nos dice que para diversos casos, el 80% de las consecuencias proviene del 20% de las causas.

Los 7 fundamentos del Testing.

Fuente: <http://www.literateprogramming.com/mccabe.pdf>

4 - Agrupamiento de defectos



complejidad ciclomática del software



métrica de la complejidad lógica de un programa

Ocurre el 80% de los problemas en el 20% del código: Suele estar en lo nuevo los errores.

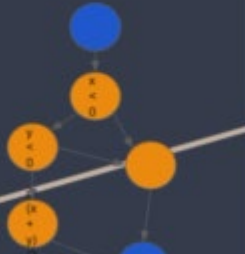
Los 7 fundamentos del Testing.

4 - Agrupamiento de defectos

```
int x,y,r;  
if (x<0 || y<0) {  
    system.out.println("X o Y son  
negativos");  
} else {  
    r=(x+y)/2;  
    system.out.println("La media de X e  
Y es:" + r );  
}
```

Los 7 fundamentos del Testing.

```
int x,y,r;
if (x<0 || y<0) {
    system.out.println("X o
Y son negativos");
} else {
    r=(x+y)/2;
    system.out.println("La
media de X e Y es: " + r
);
}
```



Fuente: <http://www.literateprogramming.com/mccabe.pdf>

4 - Agrupamiento de defectos

Para calcularla, tienes tres opciones:

- Restar las **aristas** menos los **nodos** y sumar 2:
 - $V(G) = \text{Aristas} - \text{Nodos} + 2$
 - $V(G) = 7 - 6 + 2 = 3$
- Sumar 1 al número de **nodos predicados** (aquellos de los que salen dos flechas)
 - $V(G) = \text{Nodos predicados} + 1$
 - $V(G) = 2 + 1 = 3$
- Contar el número de **regiones** (espacios «encerrados entre nodos y aristas», también se tiene en cuenta el espacio «exterior» a todos los nodos y aristas).
 - $V(G) = \text{Regiones}$
 - $V(G) = 3$

(sarasa)

Lo que uno realmente hace es: En sw de facturación, reviso lo que toqué y si tiene relación con la porción del software más complejo

Los 7 fundamentos del Testing.

```
int x,y,r;
if (x<0 || y<0) {
    system.out.println("X o
Y son negativos");
} else {
    r=(x+y)/2;
    system.out.println("La
media de X e Y es: " + r
);
}
```



Fuente: <http://www.literateprogramming.com/mccabe.pdf>

4 - Agrupamiento de defectos

Complejidad ciclomática	Tipo de código
1-10	Simple
11-20	Algo complejo
21-50	Complejo
50	No testeable

Los 7 fundamentos del Testing.



Fuente: <https://www.dianodeqa.com/post/los-siete-principios-de-las-pruebas>

5 - Paradoja del pesticida

Si las pruebas se repiten una y otra vez, eventualmente estas pruebas no encontrarán nuevos defectos. Para detectar nuevos defectos es posible que sea necesario cambiar a nuevas pruebas y a nuevos datos de pruebas. Las pruebas ya no son efectivas del mismo modo que los pesticidas ya no son efectivos para matar insectos después de un tiempo.

ISTQB

Si yo durante décadas que quiero matar a las hormigas de un área con un pesticida, se acostumbran y no les hace nada. NO USAR ESTA ANALOGÍA EN EL PARCIAL!!!

Si yo pruebo siempre lo mismo en el código, llega un momento en el que el código se va a acomodar. Pero no quiere decir que no haya más errores; sólo que se "acostumbro" a este tipo de prueba, y probando otra cosa puede seguir dando error

Los 7 fundamentos del Testing.

6 - Es dependiente del contexto

Las pruebas se realizan de manera diferente según el contexto (ambiente de desarrollo, pruebas, pre-producción, etc.).

Por ejemplo, las pruebas se desarrollan de manera diferente en marcos predecibles (metodología tradicional) que en framework agile (scrum).

ISTQB

- Tiempo
- Riesgo
- Impacto

Tengo que adaptarme al contexto en el que estoy: Cada proyecto va a ser distinto: La compañía, mi equipo, mi rol, la metodología...

Los 7 fundamentos del Testing.

7 - La ausencia de defectos es una falacia

La ausencia de errores es una falacia (es decir, una creencia equivocada) conforme a que las pruebas contribuyen a minimizar los defectos en un gran número, pero eso no indica que se realice al 100 %.

ISTQB

NUNCA puedo NO TENER NINGÚN ERROR!

Tipo & Técnicas de pruebas.

Definiciones.

Clase 3

Tipo & Técnicas de pruebas.

Definiciones.

El tipo es una cosa, las técnicas son otras!

Entidades - Modelo mental

Clasificación

Tipos de Pruebas

Técnicas Pruebas

Clasificación de Pruebas



Caja Negra:

Las pruebas de caja negra son un enfoque en la ingeniería de software para **evaluar la funcionalidad** de un sistema sin necesidad de conocer su estructura interna o el código subyacente. En estas pruebas, **los evaluadores se centran en probar las entradas y salidas del sistema** para validar su comportamiento de acuerdo con las especificaciones.

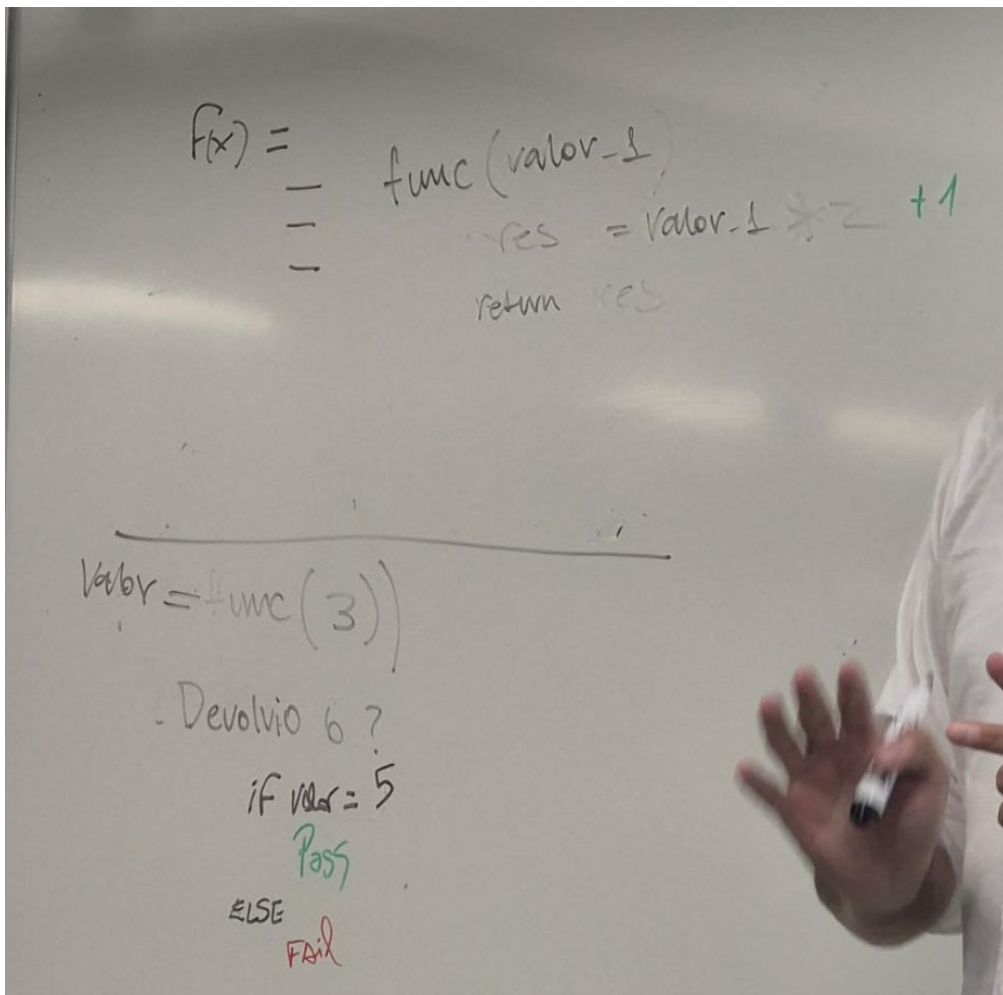
La idea es tratar el sistema como una "caja negra" donde solo se observa cómo reacciona a diferentes entradas sin considerar cómo se procesa internamente. Esto ayuda a identificar errores, problemas de usabilidad y discrepancias entre el comportamiento real y el esperado. Las pruebas de caja negra son esenciales para asegurar que un software cumpla con los requisitos y funcione correctamente desde la perspectiva del usuario final.

Pregunta parcial: Defina caja negra o caja blanca

No me interesa evaluar el código, me interesa la expectativa de entrada, y la de salida. Si yo quiero un préstamo de 100.000, cuánto va a ser el interés? Hago un código que me lo evalúe. No me interesa el código, sólo quiero el resultado.

Caja blanca: Aquí si me interesa el código y su funcionamiento.

Por tiempo, se suelen hacer sólo de caja negra. Pero sumarle las de caja blanca mejora el grado de calidad.



Si yo DEFORMO LA PRUEBA, y aun así la paso, entonces MI CÓDIGO ESTÁ TAMBIÉN MAL. Deformar la prueba significa que acepte valores que NO debería.

EXAMEN: Que es prueba de caja negra, que es la de caja blanca. Pruebas de estrés: Creo código y simulo que la usarán 5k personas simultáneamente, y vemos si se rompe. Las de estrés pueden ser de caja negra o blanca según veremos despues.

Tipos de Pruebas

Los **Tipos de Pruebas** es un grupo de actividades que se centran en un objetivo en particular:

- Evaluar características funcionales
- Evaluar características no funcionales
- Evaluar estructura / arquitectura
- Evaluar código
- Evaluar efectos de los cambios



ESTO SERA PREGUNTA DE EXAMEN, Y ESTO ES LA RESPUESTA COMPLETA!

Tipos de pruebas.

1. Pruebas Funcionales

Verifican que el sistema, módulo o componente realice lo que está establecido en los requerimientos funcionales.

Se pueden llevar a cabo en todos los niveles:

- Componentes
- Integración
- Sistemas
- Aceptación

Ej: Necesito que mi sistema tenga FACE ID.

Mi sistema tiene un montón de componentes. Aún así, crearé uno nuevo.

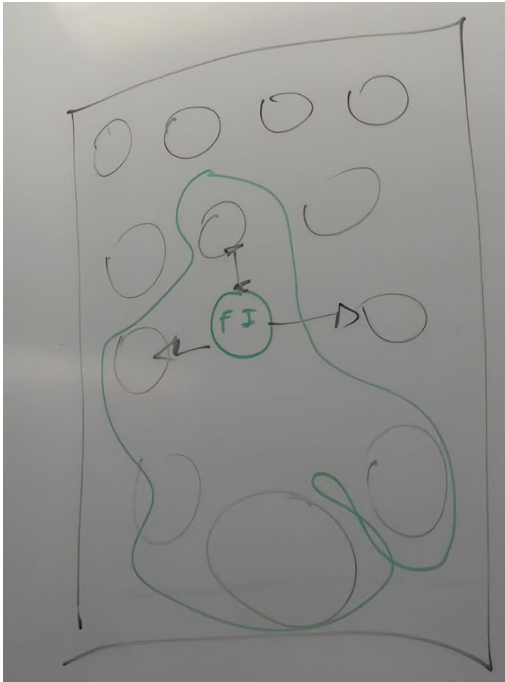
Integré el FACE ID. A la hora de probar, que pruebo?

Si es una prueba de componentes, pruebo sólo el fase ID.

Funciona todo el sistema? No lo se, pero por el fundamento del testing del 80/20, es muy probable que si hay errores estén ahí.

Si mi fase ID estaba relacionado con otras 3 funciones digamos, y las pruebo todas, entonces es prueba de integración funcional.

En la prueba de sistema, se prueba todo? NO. Pruebo LO QUE PUEDO. Porque tengo TIEMPO y RECURSOS (dinero, personas) limitados.



UAT (User Acceptance Testing)

UAT significa **User Acceptance Testing** (Pruebas de Aceptación del Usuario). Es la última fase del proceso de pruebas antes de que un sistema pase a producción.

¿Qué es?

Es el proceso donde los **usuarios finales o el cliente** validan que el software cumple con sus requisitos y necesidades de negocio. No lo ejecutan los testers técnicos, sino los propios usuarios o representantes del negocio.

El UAT es un ejemplo de prueba de aceptación. También me pueden pedir desde RRHH ver la app antes de lanzarla, eso es otro UAT.

Todo cambia: requerimientos, personal, tecnología. Entonces tenemos que tener un equipo detrás disponible y dispuesto a arreglar y actualizar.

Si yo tengo un bug y lo arreglo, estoy modificando el sistema.

Si arreglé un bug, ya esta lista la entrega? No. Lo tiene que aceptar quality assurance QA. Esto se llamaba PRUEBA DE CONFIRMACIÓN.

CONFIRMACIÓN = SI FIXIÉ O NO EL BUG

Prueba de regresión: probar lo que ya probé porque además de este fix me pusieron un componente nuevo, entonces vuelvo a probar lo que en nuestro idbujo esta en negro y lo verde. Aunque no se lo haya probado; Se tiene que volver a tocar todo.

ES DECIR, PRUEBO LAS FUNCIONALIDADES IMPORTANTES POR SI LO NUEVO ME AFECTÓ LO EXISTENTE

Dif entre prueba de regresión & integración funcional: ¿??

Smoke testing: Prueba SÚPER acotada (porque hice un cambio mínimo como cambiar un color aka UI o porque tengo que entregarlo mañana)

Se decide entre todo el equipo, cuando todos concuerdan que es ínfimo el cambio.

Ej: En un jueguito tipo candy crush, pruebo si puedo comprar vidas (lo que afecta al negocio) y si puedo pasar un nivel random.

Full pass: Prueba con un monton de casos de prueba que tengo capacidad de hacer.
No se ve en el 1er parcial.

Tipos de Pruebas



LO ROJO ES LO MAS IMPORTANTE

Diferencia entre UX y UI: (CLAUDE, PRIORIZAR OTRA DEF)

UI (User Interface): cómo se ve — colores, botones, tipografía, layout visual.

UX (User Experience): cómo se *siente* usarlo — flujo, facilidad, satisfacción del usuario.

UI es parte de UX. Podés tener UI bonita con UX terrible (bello pero confuso).

Ej: el cancelar en una compra debe estar lejos del dedo gordo es cesgar el UX

Candy crush ¿Qué tipo de celulares tiene la gente que juega?

Son gama media-baja: J7 en 2026 por ejemplo

Entonces, NO hacemos la app demasiado pesada, porque van a dejar de usarlo.

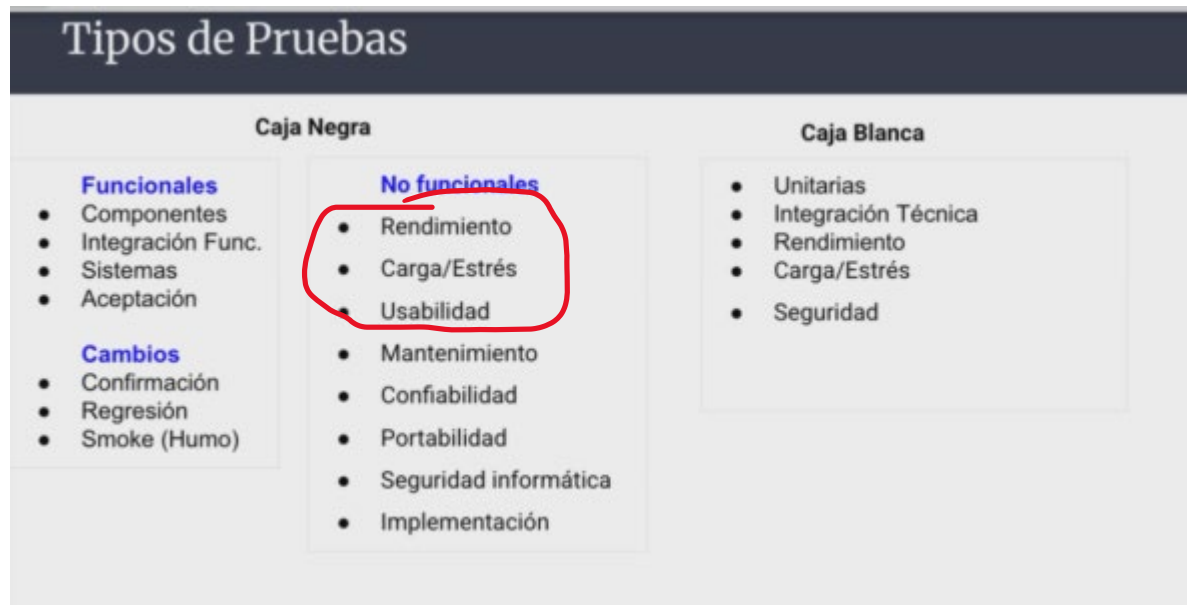
También suelen abrir FACEBOOK, SPOTIFY y varias cosas de fondo junto al candy crush, pues apps abiertas + celu viejo = crashea la app.

ESTO ES UNA PRUEBA DE RENDIMIENTO

PRUEBA DE CARGA/ESTRÉS:

En apps que son de la nube se hacen.

Las partes de pago de los juegos, que se consume del Google play, siempre necesariamente debe pasar prueba de estrés



estrés = muchos usuarios mismo tiempo? ormpo o no?

CON LOS 3 RODEADOS ALCANZA PARA EL EXAMEN

PRUEBAS UNITARIAS = DE CAJA BLANCA

Tipos de pruebas.

1. Pruebas Funcionales

Verifican que el sistema, módulo o componente realice lo que está establecido en los requerimientos funcionales.

Se pueden llevar a cabo en todos los niveles:

- Componentes
- Integración
- Sistemas
- Aceptación

Tipos de pruebas.

2. Pruebas No Funcionales

- Rendimiento
- Carga/Estrés
- Usabilidad
- Mantenimiento
- Confiabilidad
- Portabilidad
- Seguridad informática
- Implementación

Tipos de pruebas.

3. Pruebas relacionadas a cambios

1. **Pruebas de confirmación**, confirmar si se soluciona el defecto.
2. **Smoke test o Test de Humo**
3. **Pruebas de regresión**, verifica que un cambio en el código no afecte por accidente otras partes.

Conjunto de casos de pruebas, para la regresión, en cada nivel (integración, sistema y componentes) para verificar el normal funcionamiento del sistema en su completitud, sin probar exhaustivamente.

Despues del break: Estructuras de IA y su relación con QAssurance/QControl

Tipos de pruebas.

4. Pruebas Unitarias e Integración

Pruebas sobre la implementación interna del sistema. Nos interesa lo que está dentro de la caja: código, arquitectura, flujos de datos, etc.

Pruebas de componentes



% de sentencias ejecutables
% de estructuras de decisión

cobertura de código

Pruebas de Integración de componentes



% de interfaces probadas

ESTO no lo va a tomar pero lo explicó y quizás vaya en el 2do parcial

CALCULO: Ejecutados/total

FIN DEL BREAK

Keep your Operations Calm.

7Fi.

ENTERPRISE DIVISION
BY TPT GLOBAL SERVICES

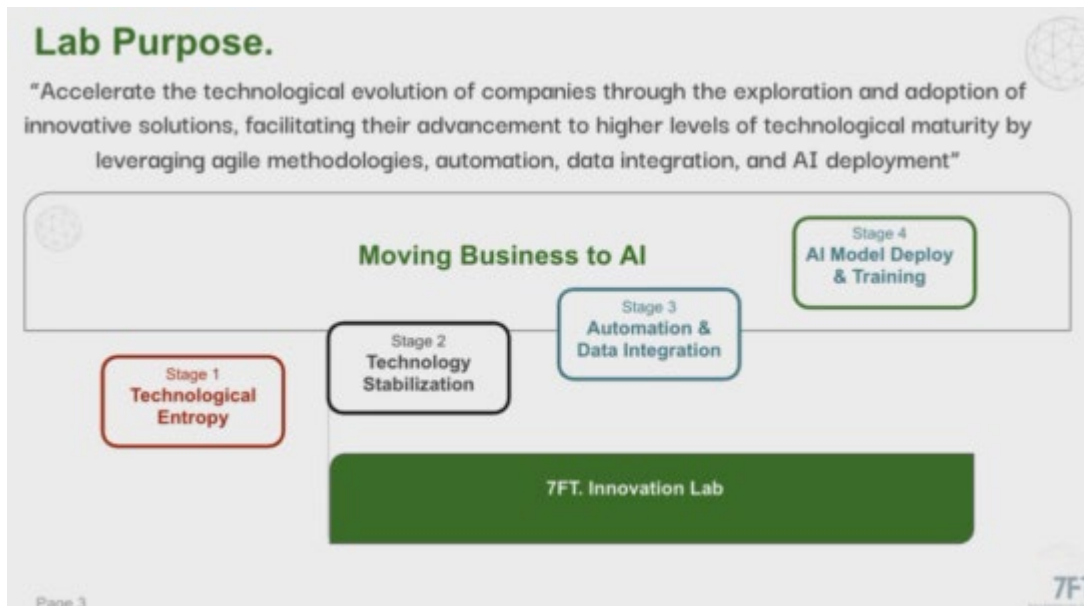
Specialists in Comprehensive Business Solutions

In today's fast-paced environment, technology plays a crucial role in creating engaging and seamless experiences. We believe the true **power of technology** lies in its ability to connect ideas and people, enhancing accessibility, convenience, and **innovation** across all touchpoints.

¿Qué es un dashboard? (no entrea al examen pero debo entenderlo en contexto)

En una empresa se toman decisiones basadas en información sacada de datos.

(...) cosas en rojo y cosas en verde



"Dame 2kg de IA, 5kg de mandarina, 2kg de automatización..."

PERO no es así: Antes necesito un proceso:

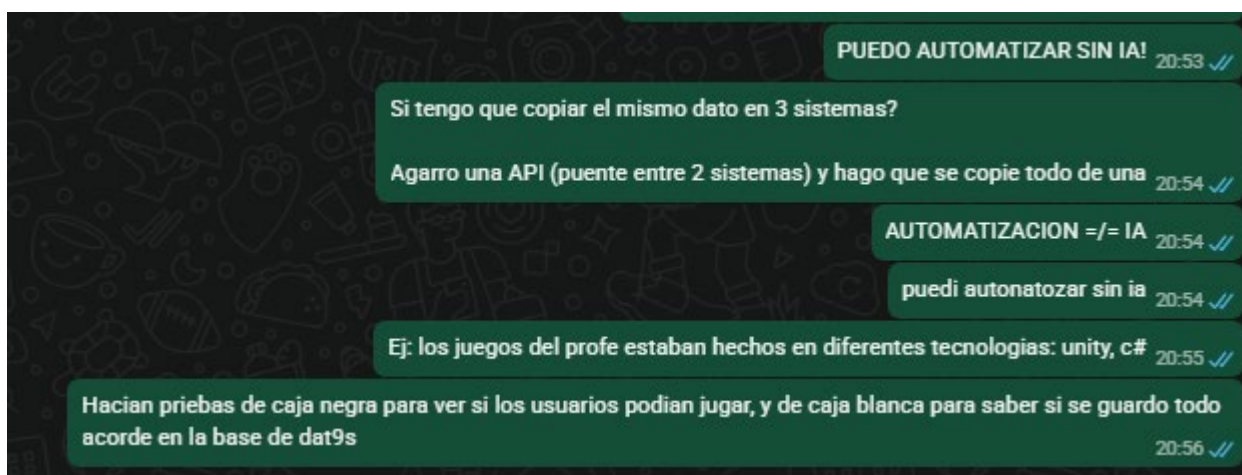
¿Cómo andan mis datos en casa? ¿Están en Excel todos? ¿O en libretas/mi cabeza? Si es así, arreglemos primero eso. Despues metemos IA.

Reducir ENTROPÍA (desorden que se acumula)

Ej: No documenté el onboarding (las instrucciones que dimos para que arranquen; no se que les dije)

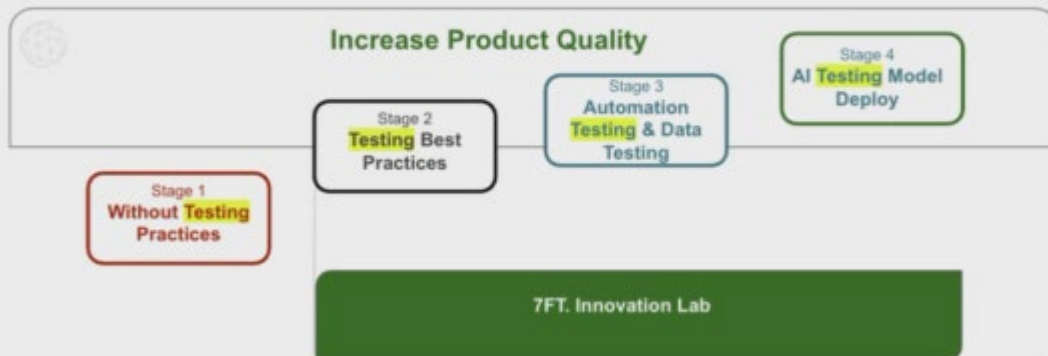
ERP = Sistema de gestión para la contabilidad.

o un CRM



Lab Purpose.

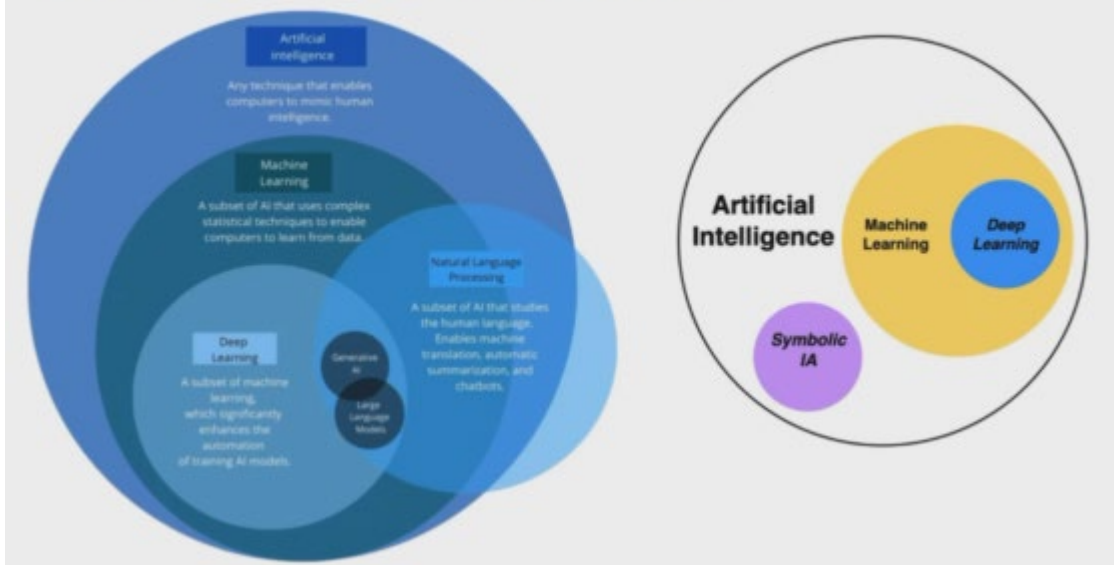
"Accelerate the technological evolution of companies through the exploration and adoption of innovative solutions, facilitating their advancement to higher levels of technological maturity by leveraging agile methodologies, automation, data integration, and AI deployment"



Nosotros salimos de acá con buenas practicas del testing. Pero no todas las empresas contratan a personas que los saben.

Es ideal un esquema híbrido, personas-IA. O que si vamos a usar IA, siempre sea en complemento a un humano. Peligro de que alucine y nadie se de cuenta.

R+D - 7FT Lab.- “Clasificación de AI...”



IA NO-GENERATIVA: MACHINE LEARNING. Tomar un montón de datos (Google por ejemplo), y saber categorizar a las personas por sus gustos.

IA GENERATIVA: Yo puedo **GENERAR** un resultado; generar **NUEVA INFORMACIÓN**, en base a información preexistente. Yo puedo usar la IA para:

- Ampliar un tema
- Resumir un tema, etc



No siempre hay que usar la generativa. Los tokens son costosos. Debería mejor evaluar en dónde es indispensable [opinión personal: nunca lo es] y no puedo reemplazarlo por una no-generativa.

ChatGPT es una APP, NO UN AIA, que detrás tiene UN MODELO DE IA. ESOS MODELOS SE LLAMAN LLM.

El LLM de GPT se llama GPT 5.1 (por ahora) (owned by OpenAI); el de Gemini (owned by google) se llama Flash, el de Claude (owned by Anthropic) se llama Sonnet,

R+D - “Arquitectura de un Agente AI Gen. - Std”



En GPT → Proyectos: Puedo crear distintas instrucciones para cada agrupación de chats.

Tipos de prompt:

- User prompt: el del usuario
- System prompt: el general que puse en settings

Y un prompt tiene partes:

- rol
- reglas
- excepciones, etc

Esta buenísima la documentación de Claude (gratis) sobre cómo hacer un buen prompt.

¿Y qué diferencia hay entre todo esto y un "agente"?

Se asocia agente a algo más corporativo:

-Che, estan compartiendo toda mi información...

El problema es que si yo no le pongo un buen prompt, y tomo como válido lo que me dice GPT, y tomo una decisión de mierda en base a eso y quiebra mi empresa. Acá entran en juego los agentes:

-Que solo tenga acceso a cierta info

-Que siempre tome todas las decisiones en base a X lineamientos

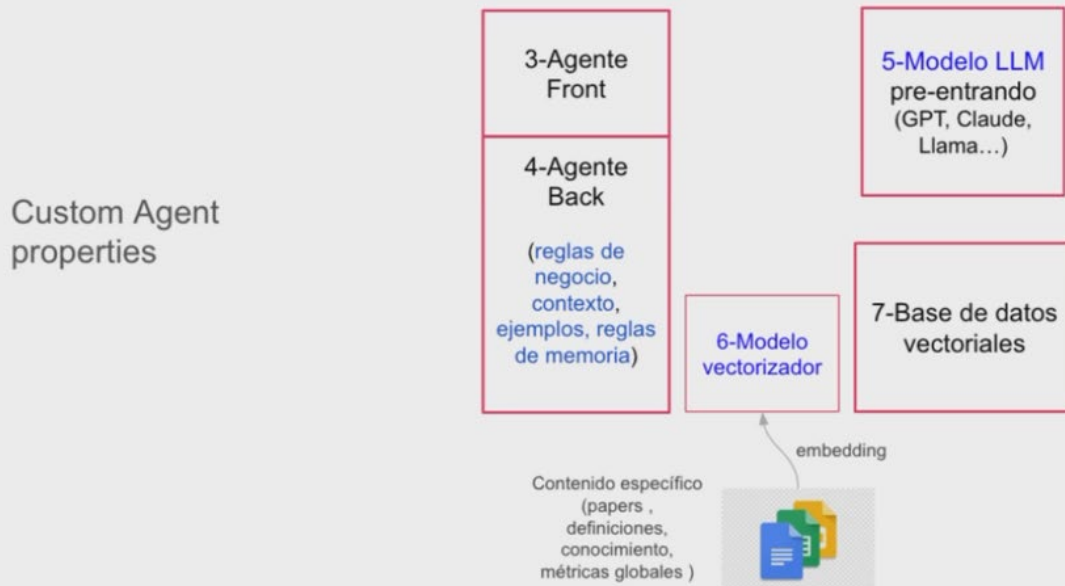
-En realidad todos son agentes, pero el corporativo lo conecto con API a ERP o CRM (o SRM?), PARA QUE TOME MUCHAS MAS DECISIONES muchísimo más rápido.

-Ej: Si no trabajo los accesos, un cadete puede preguntar: Che, cuanto gana mi jefe? O aun peor, mi par?

GOBERNANZA DE DATOS Y ACCESOS: Significa tener cuidado con la info confidencial de la empresa: saber que enviar y que no. Da tranquilidad al inversor

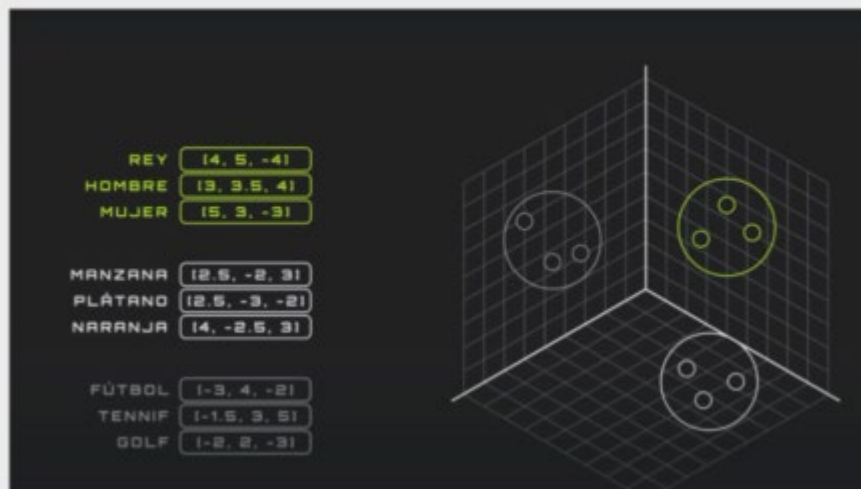
Si no, tenes a 30 empleados todos pagando 20 USD individualmente y tomando decisiones empresariales en base a prompts sacados de TikTok.

R+D - 7FT Lab.- “Arquitectura de un Agente AI”



MEMORIA: Recordar lo que hablé. TODO lo que hablé? No: Se graba un resumen de la memoria de los puntos más relevantes. Cuál es el problema? Que generalmente el mismo sistema (unless prompted otherwise) decide lo que importa y lo que no, y se pueden perder datos claves.

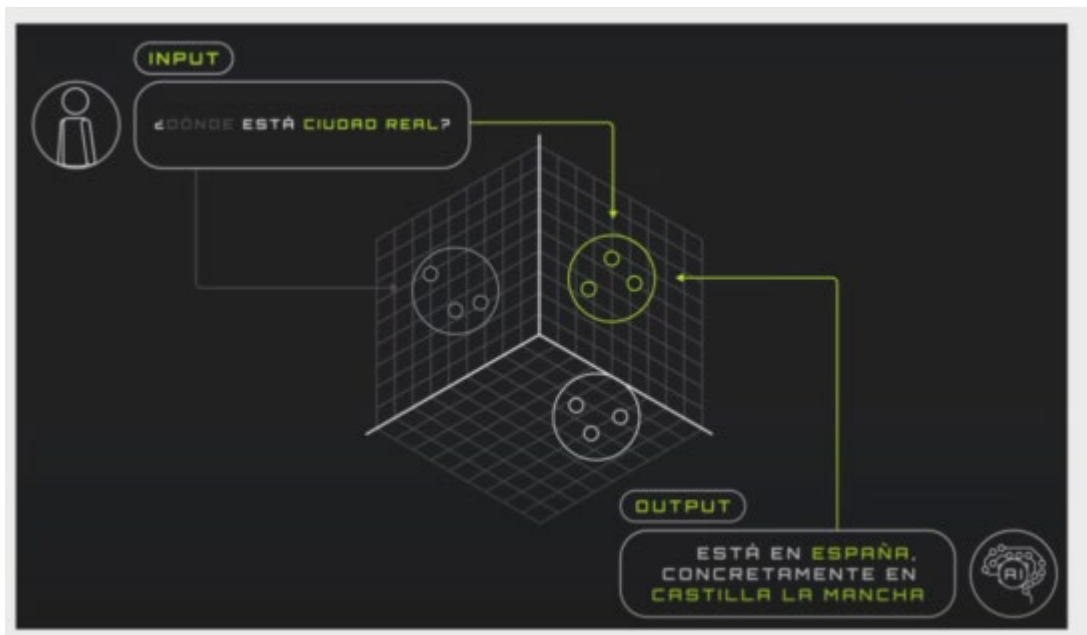
Los LLM **catalogan** los términos o palabras en **coordenadas**



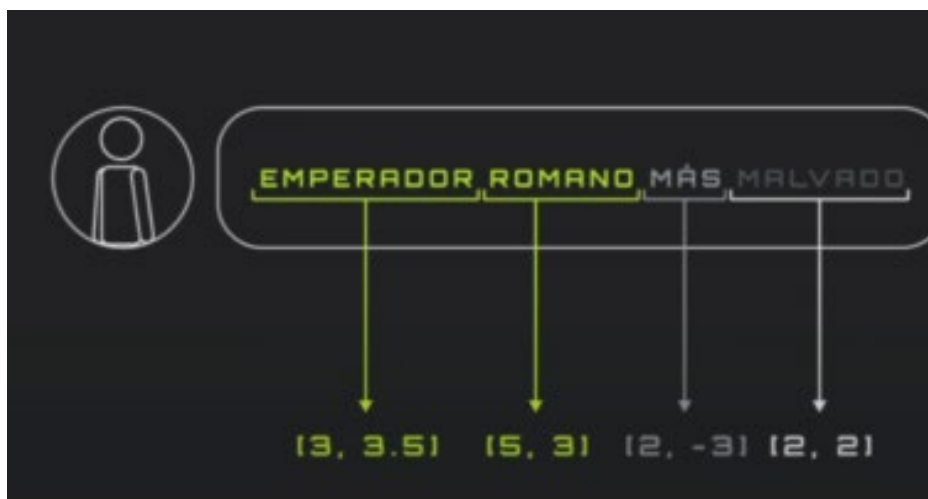
VECTORIZACIÓN: Paso cadenas a números, para manejarlo matemáticamente. No son "números", son **VECTORES**.

Un vector es una rayita con un ángulo y otro ángulo. Si tengo un ángulo parecido, estoy aproximando una palabra con otra.

Esto es **EMBEDDING**: Categorizar esto en bases de datos.



Este proceso de búsqueda de información se llama RAG - Retrieval-Augmented Generation.



Busca frases **CERCANAS** a esas coordenadas



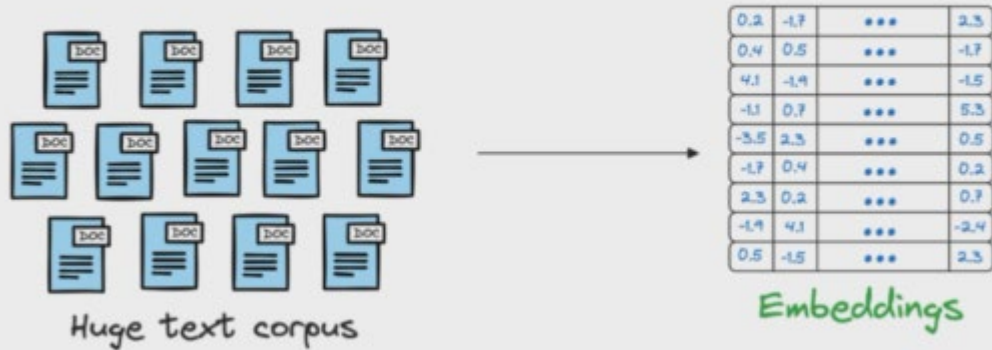


Este sistema/proceso me permite archivar las coordenadas(vectores) de mi data/docs /base de conocimiento (embedding) en una base de datos que guarda dichos vectores.

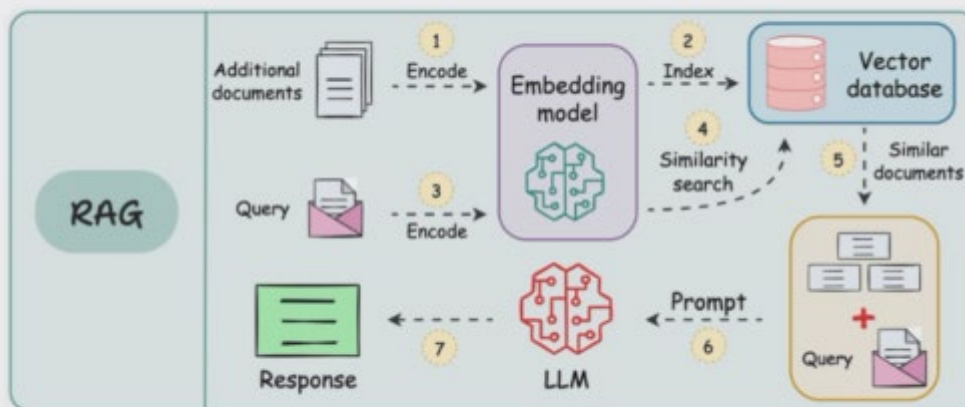
RAG

RETRIEVAL-AUGMENTED GENERATION
(GENERACIÓN MEJORADA POR RECUPERACIÓN)

R+D - 7FT Lab.- “RAG & Embedding & Deploy”



R+D - 7FT Lab.- “RAG & Embedding & Deploy”

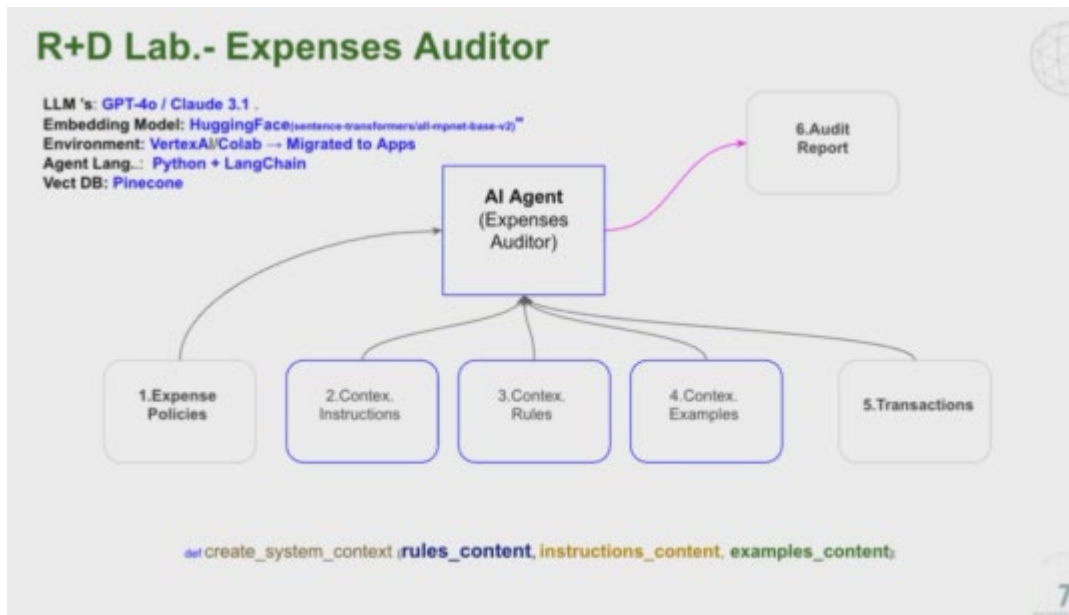


Hago el query, hago el embedding de la pregunta...



Esta desactualizado esto, pero estas son las empresas.

Yo a estos modelos los ingreso por API, una API es un puente para ingresar a la nube. Pero si yo soy medico por ejemplo, debo mantener confidencialidad: No puedo compartir nada. Por eso existen los LLM OPEN SOURCE (LOCALES).



Cuando trabajas en una corporación, necesitas RENDIR TUS GASTOS para que después te los devuelvan (los relacionados a la empresa, claro. Ej: la nafta para viajar a un lugar que me pidieron inspeccionar)

Pero... si tengo 3 facturas de la misma nafta para el mismo kilometraje? Capaz a la IA solo le puse "aprobar facturas de nafta hasta este tope monetario", pero me lo dividieron en subfacturas y me cagaron.

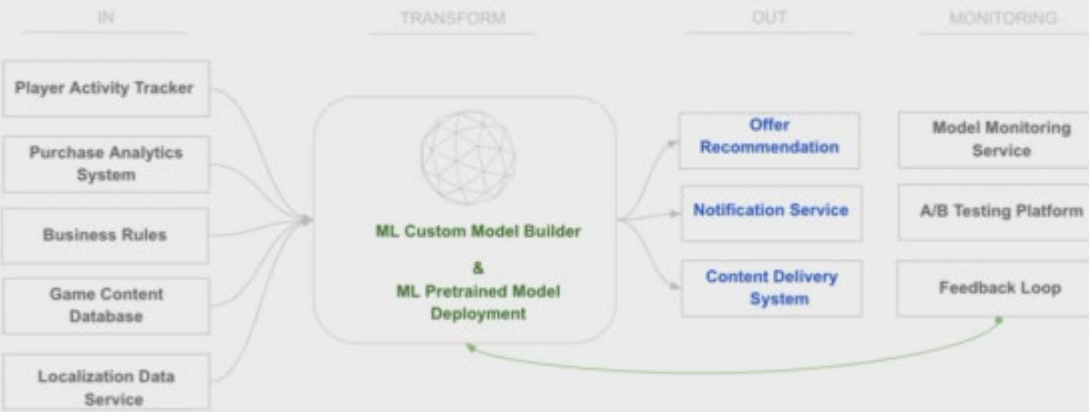
Ejemplo mejor (ignorar el previo): Las corporaciones te cubren las cenas entre colegas. Si un gerente la invita paga el gerente. Pero si a esa cena va el jefe del gerente, la paga el gerente. Esta predefinido esto. Entonces, ¿Quién paga cada vez? Hacemos un EMBEDDING DEL REGLAMENTO EMPRESARIAL, y lo definimos con...

el RAG solo recupera contexto, pero es la LLM quien razona si 3 facturas de nafta para el mismo km huelen raro. Por eso la regla del tope monetario sola no alcanza — necesitás que la LLM tenga criterio, no solo límites numéricos.

síntesis de la clase: Hay que validar todo. Tanto en este ejemplo que invente como en el del profe, hay que cubrir muchísimos casos, cantidades absurdamente altas. Por eso hay que administrar y categorizar bien los datos.

Clase que viene: ¿Por qué las IAs alucinan? ¿Cómo se le meten imágenes a estas IAs?

R+D Lab.- 7FT- AI Offer Recommendation



Clase 10

Consejos para la presentación:

El profe NO quiere "presentaciones Disney": Vamos a tener que decir los percances que hubo, vamos a tener que decir qué es lo que no llegamos a hacer, y lo que aprendimos.

Dar una presentación INNOVADORA: NOSOTROS debemos tener más letra, poner menos texto en la diapositiva, lucirnos más nosotros que la diapositiva.

Tenemos que lograr METER SUTILMENTE CUESTIONES DE FUNDAMENTOS DEL 1ER PARCIAL!!!

No pudimos probar todo? Eso esta LIGADO A LOS 7 FUNDAMENTOS! Que todos tenían MAC o WINDOWS? Eso también!

"Nos costó la planificación porque..." ← y eso ya es un fundamento, los podemos meter ahí.

La herramienta CATALON para automatizar, es una buena forma de SUBIR NOTA por haber aprendido a AUTOMATIZAR (pero que sea sutil, no dejar que acapare todo). Poner un video, no hacer demos en vivo.

Si hacemos videos, no ponerlos en YouTube porque acá va a salir con anuncios.

Sepamos DIVIDIR: Por ejemplo, el cierre puede ser mitad conclusiones de la materia y mitad conclusiones del proyecto.

Humor? Riesgoso, pero se podría. No poner memes de mal gusto.

"No se vayan a quedar muy quietos" → Pero tampoco nos movamos por toda el aula.

Si vemos que nuestro compañero se olvida algo, JAMÁS decir: Che te olvidaste...
O lo decís vos, o le decís algo como: "Te parece si también decís esto?"

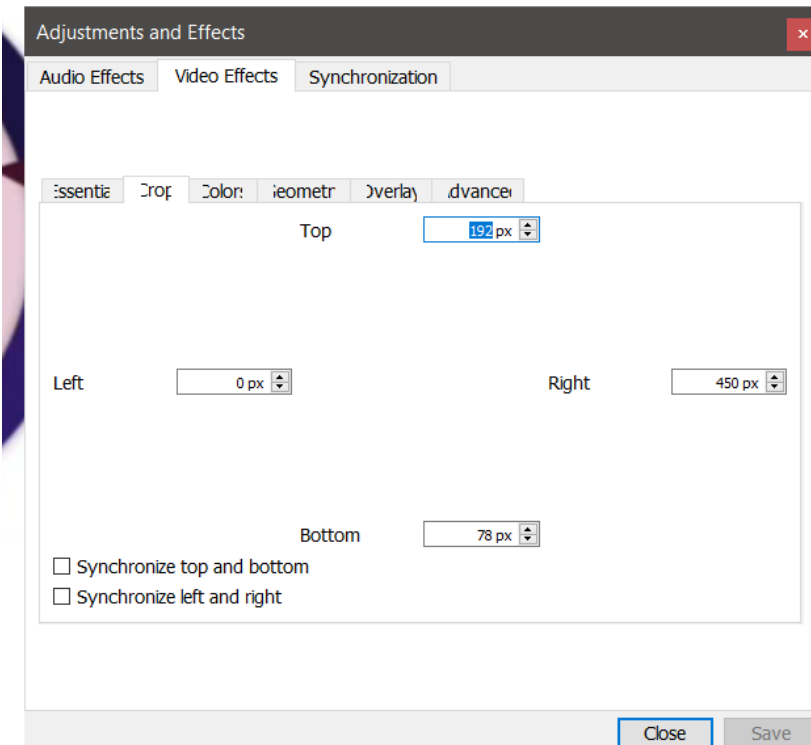
"Calidad es el grado en que..." ← Esto es *CLAVE* para el oral (lo menciona mucho el profe y seguro le guste que lo digamos)

CIERRE POTENTE: Poner música, cambiar el tono de voz, etc.

CURSOS DE STORYTELLING.

Clase 6

Analysis





06:00

Este es el 1er tema para el TP (NO entra en el parcial)

El 2do Parcial va a ser:

- Varios grupos de QA
- Un grupo de Programadores
- Un grupo de Project Managers

Veamos el repaso de la clase anterior (lo último que SI entra):

- 1- Proceso de Testing.
- 2- Agile
- 3- Casos de Prueba

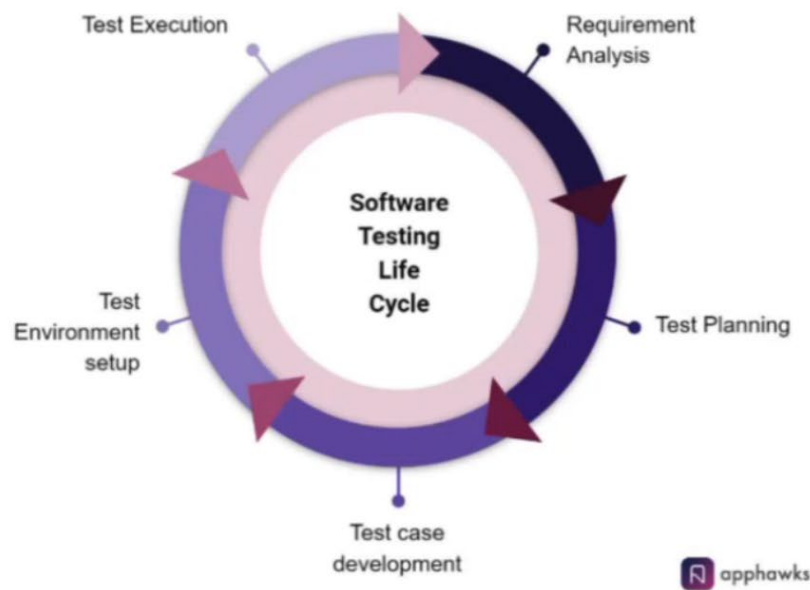
Proceso de testing.

Definición y Aplicación.

Lo más importante es:

Proceso de testing - Base

1



SIEMPRE lo primero a hacer, es entender los requerimientos.

Tenemos que dibujar este círculo en el parcial.

Junto con una breve descripción de que se hace en cada etapa del testing.

2) Test Planning

Por ejemplo, en la planificación que pondríamos?

- TIEMPO es una variable
- COSTO y RECURSOS
- TÉCNICAS DE TESTING a emplear → Las PRUEBAS se PLANIFICAN

Y la PLANIFICACIÓN, es una tarea de QUALITY ASSURANCE o QUALITY CONTROL?

Es de QUALITY ASSURANCE, pues esta se encarga de los procesos.

QUALITY CONTROL se encarga de el/los producto/s.

QA vs QC diferencia: Será pregunta de parcial. NO poner sólo "QA = PROCESO; QC = PRODUCTO"; Hay que DESARROLLAR.

3) Desarrollo de CASOS DE PRUEBA

Cómo el detalle? Lo vamos a ver hoy, pero tengo que saber las variantes que voy a utilizar: Mayor a 28, género masculino, 30% de descuento → Eso es un caso de prueba que veremos hoy como detallarlo a profundidad

¿Cuál era la arquitectura de ambiente típica en un ciclo de vida de desarrollo?
Desarrollo-calidad (o staging) - y Producción.

Preparar ambiente, es: Preparar ambiente para que sea lo más parecido a producción.
Pero, también tengo que PREPARAR LOS DATOS. En un juego por ejemplo, tengo que separar nuevos usuarios, payer, non payer, VIP: Y hay que probar todas estas bifurcaciones, entonces tenemos que haber preparado los datos.

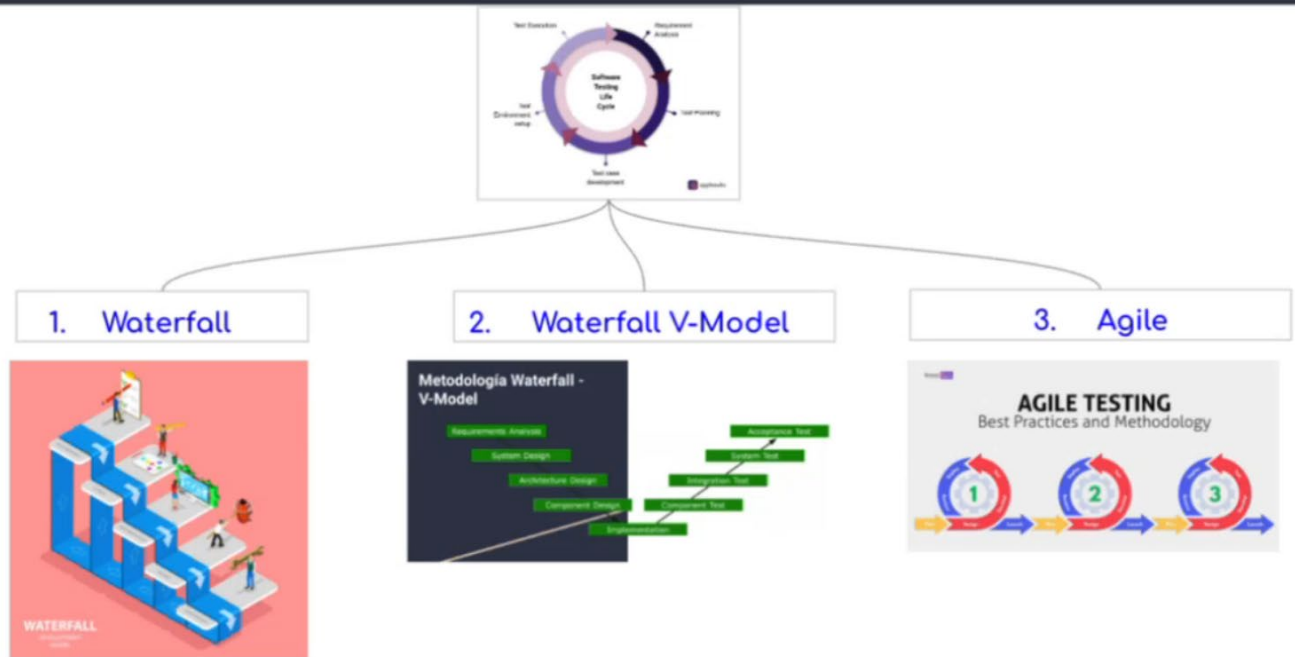
Finalmente, EJECUTO: Esto significa ejecutar los casos de prueba, y documentar los resultados: PASS o FAIL. Si es pass? Todo bien. Si es fail? Tengo que REPORTAR LOS BUGS.

→→→ Si tenemos bien ESTA parte del examen? Ya lo tenemos adentro.

→→→ El profe usa este slide incluso en su vida profesional privada: Sirve para chequear globalmente que entendimos los conceptos de Testing de Aplicaciones.

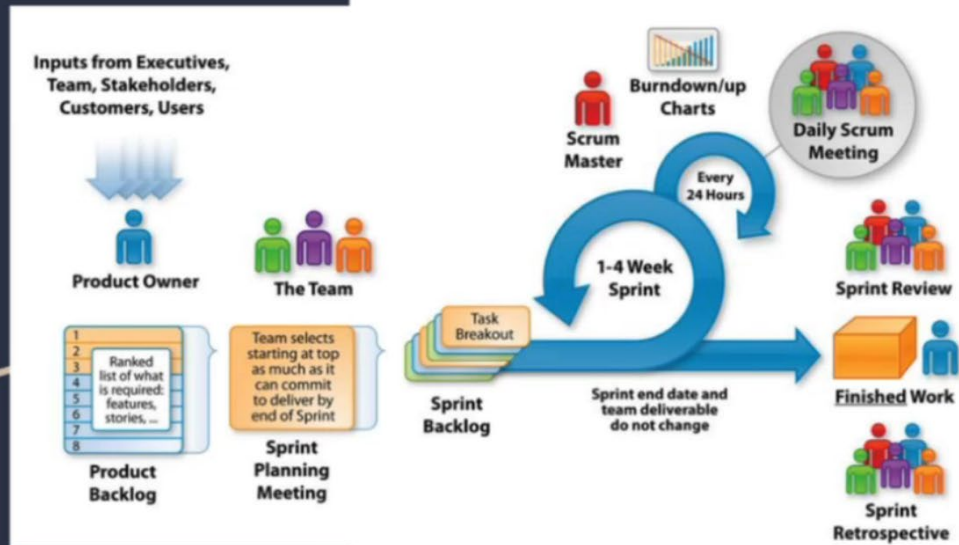
// parcial → Hacer varios casos con Claude para clasificar de que parte del proceso de testing se trata determinado stage de cierta empresa que me genere Claude. Asimismo, luego redactarlos yo desde cero.

Proceso de testing aplicados a metodologías



Lo que nos interesa es sólo AGILE:

Metodología Agile - ¿Qué es Agile testing?



No es necesario acá que hagamos el dibujo.

Tenemos libertad de expresar lo que entendimos, pero seamos ordenados, concisos, y no nos vallamos por las ramas.

Media carilla o una para explicar esto está bien; pero NO 3 hojas.

Agile – Definiciones

Agile: La metodología Agile se usa en el desarrollo de software y otros proyectos de alto rendimiento; se centra en la implementación rápida de un equipo eficiente y flexible para planear el flujo de trabajo. Agile brinda la capacidad de elegir la mejor opción en cada situación sin comprometer el proyecto

Sprint: Se define Sprint a un ciclo de trabajo en la metodología Scrum, en el que se planifican y completan ciertas actividades establecidas. Por lo tanto, se trata de un intervalo de tiempo prefijado en el que el equipo trabaja, durante el cual crean el producto entregable

Product Backlog: El trabajo pendiente de un producto o más conocido por su término en inglés, product backlog, es una lista ordenada de todas las tareas que se harán para el desarrollo de un producto, cuando utilizamos la metodología Scrum.

Si bien tenemos que contestar con nuestras palabras, estas son las definiciones formales.

Agile – Definiciones

Agile: La metodología Agile se usa en el desarrollo de software y otros proyectos de alto rendimiento; se centra en la implementación rápida de un equipo eficiente y flexible para planear el flujo de trabajo. Agile brinda la capacidad de elegir la mejor opción en cada situación sin comprometer el proyecto

Sprint: Se define Sprint a un ciclo de trabajo en la metodología Scrum, en el que se planifican y completan ciertas actividades establecidas. Por lo tanto, se trata de un intervalo de tiempo prefijado en el que el equipo trabaja, durante el cual crean el producto entregable

Product Backlog: El trabajo pendiente de un producto o más conocido por su término en inglés, product backlog, es una lista ordenada de todas las tareas que se harán para el desarrollo de un producto, cuando utilizamos la metodología Scrum.

(Lo pasó rápido a este slide; seguro no importe)

Agile – Definiciones

Sprint Backlog:

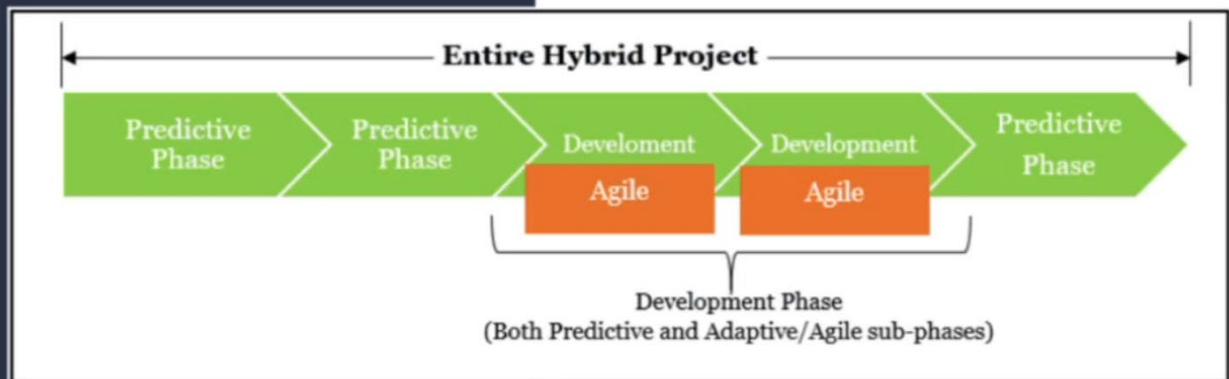
El sprint backlog es representado a través de un tablero de tareas; hace visible todo el trabajo necesario para alcanzar el compromiso que se hizo con el product owner para el sprint. Permite ver las tareas donde el equipo está teniendo problemas y no avanza, para tomar decisiones al respecto

Eventos/Ceremonias:

Son reuniones que tienen lugar en las iteraciones o Sprints contemplados dentro de la metodología Scrum, y son una parte importante del progreso de cualquier proyecto.

idem

Testing en frameworks híbridos



Fuente: bigpicture.one

- *empresas grandes y maduras*
- *áreas funcionales diversas*
- *certidumbre, pero a la vez entregas periódicas de software*
- *requisitos definidos antes de desarrollar*

idem

Técnicas.

Repaso.

Técnicas

- Partición de clases de equivalencia
- Valores Límites o Frontera
- Tabla de decisiones
- Árboles de decisión
- Casos de uso
- Unitarias por código
- Por cobertura
- Mutantes

Si o si nombrar las primeras 4.

Y ya que estamos, nombrar casos de uso.

Técnicas – Partición de Clase de Equivalencias



Técnicas – Valores Límites o frontera



OTRA PREGUNTA DE PARCIAL: Nombrar los 7 fundamentos del Testing.

Técnicas – Tabla de Decisión

Condiciones	Reglas							
Condición 1	S	S	S	S	N	N	N	N
Condición 2	S	S	N	N	S	S	N	N
Condición 3	S	N	S	N	S	N	S	N
Acción 1	X	X						
Acción 2				X		X		X
Acción 3			X				X	
Acción 4					X			

ARRANCAMOS CON UN RECTÁNGULO.

→ ¿Cuántas condiciones tendremos?

- Entre 3 y 4. Porque 2^5 ya me da 32, y queda enorme.

→ ¿Cuántas acciones tendremos?

- Ilimitadas

Usar el MÉTODO DE COMBINACIONES!

Si son 8 → 4V 4F ; 2V 2F 2V 2F ; 1V 1F 1V 1F 1V 1F 1V 1F

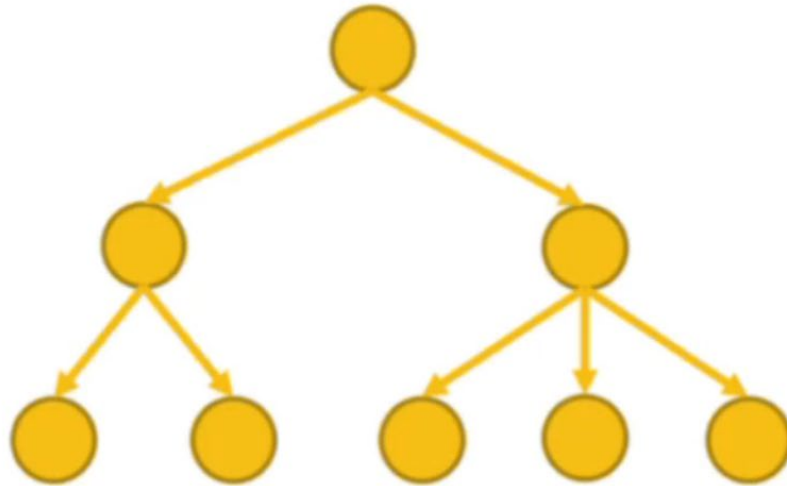
...Y se van a usar todas? NO! Porque algunas van a ser ABSURDAS!

Ej: No puede ser cliente y a la vez no cliente.

Igual estoy haciendo condiciones de más acá: En vez de eso, podría hacer cliente negado y es una sola.

Técnicas – Árbol de Decisión

Single Decision Tree

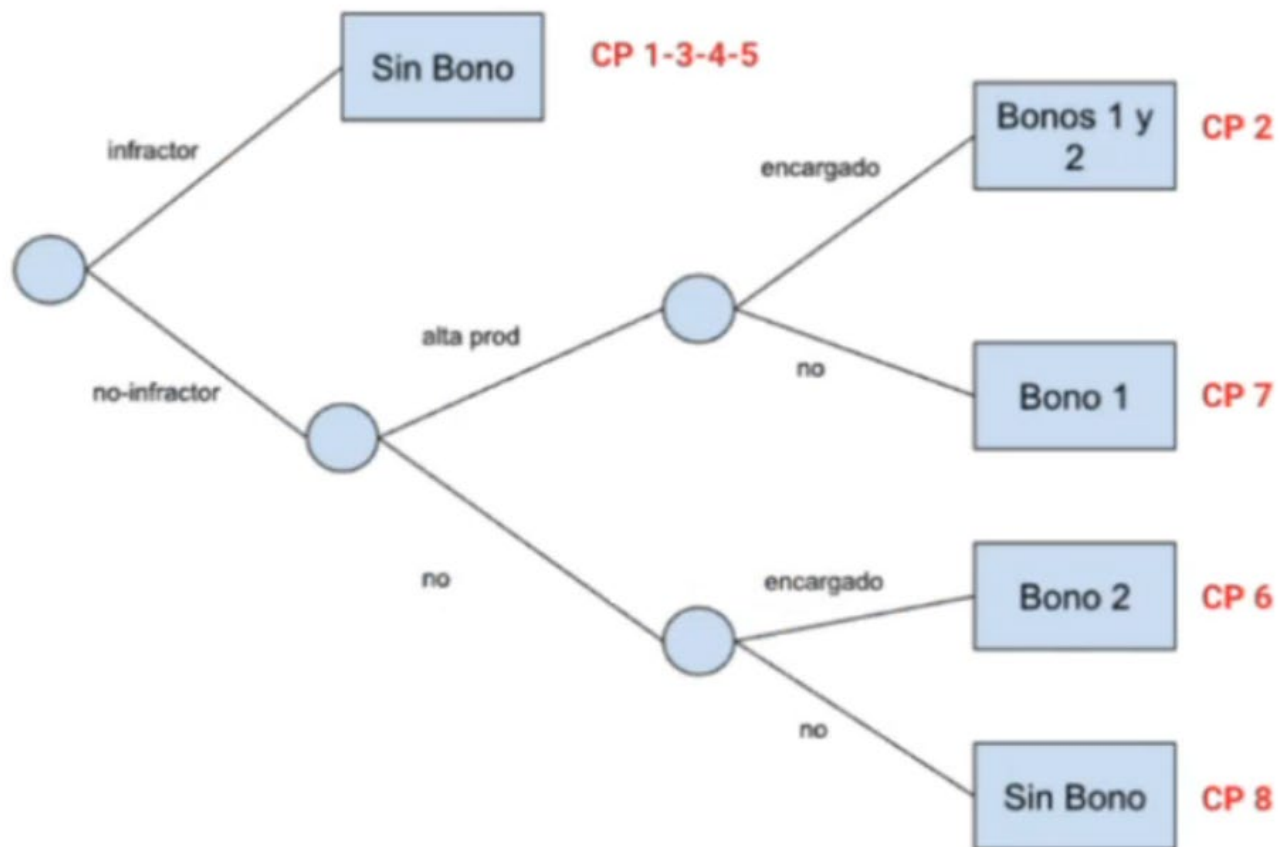


ARRANCAMOS CON UN CÍRCULO

NO es un DIAGRAMA DE FLUJO: Empiezo con UNA CONDICIÓN

Y termina con un CUADRADO

Figura	Nombre	Significado
	Nodo de decisión	Indica una decisión que se tomará
	Nodo de probabilidad	Muestra múltiples resultados inciertos
	Ramificaciones alternativas	Cada ramificación indica un posible resultado o acción
	Alternativa rechazada	Muestra una alternativa que no estaba seleccionada
	Nodo terminal	Indica un resultado definitivo



EJERCICIO:

Ejercicio – Diagrama de Flujo

El requerimiento dice:

Considerar lo siguiente para calcular el importe Neto:

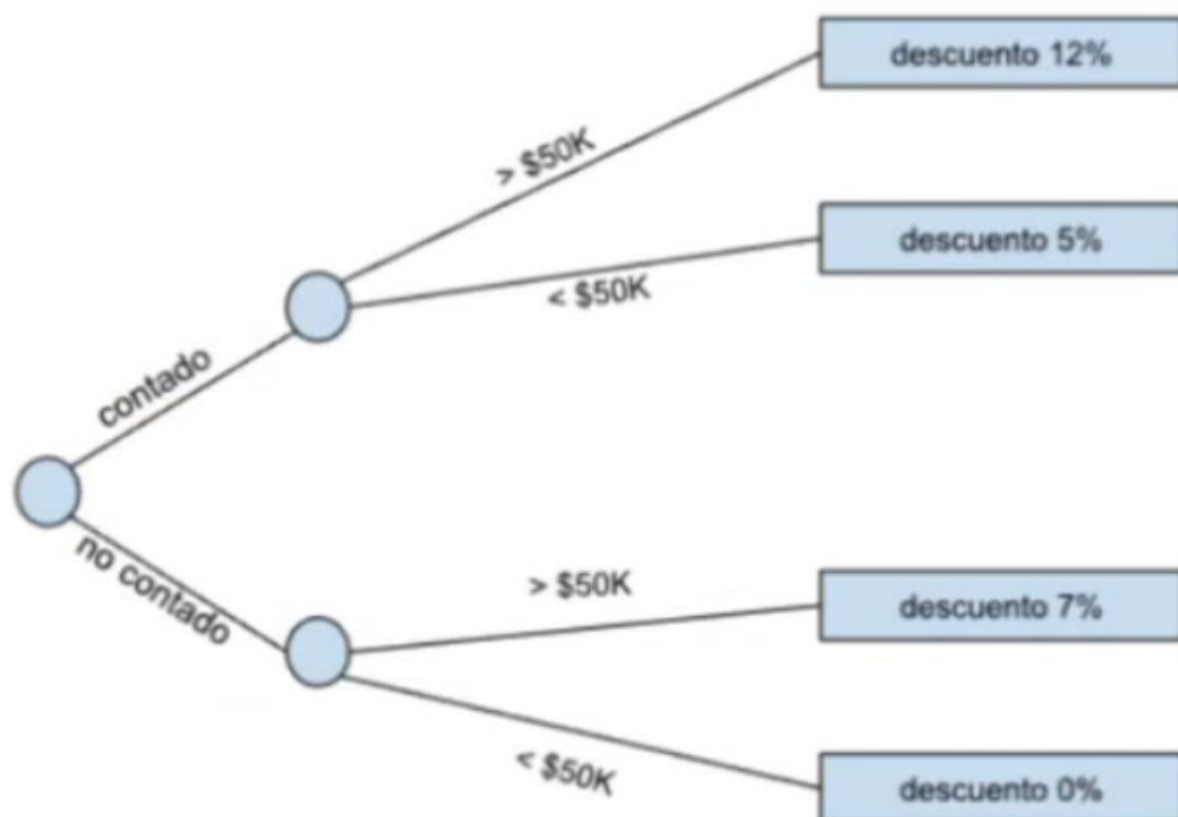
- Si el cliente paga de contado hacer descuento del 5%
- Si el cliente compra más de \$50.000 hacer bonificación del 7%
- Ambas cosas, descuento y bonificación, puede ocurrir en un mismo pago
 - Arme **Diagrama de Flujo**
 - Arme **Tabla de Decisión** (condiciones y acciones).
 - Arme **Árbol de Decisión**
 - Especifique **supuestos** y **fuera de alcance**.
 - ¿Cuántos casos encuentran y qué precondiciones consideran?

OJO: VALE 3 PUNTOS DEL EXAMEN!

Tabla de decisión

	Reglas			
Condiciones	1	2	3	4
pago contado	V	V	F	F
compra > 50k	V	F	V	F
Acciones				
descuento 5%	X	X		
descuento 7%	X		X	
calcular importe neto	X	X	X	X

Árbol de decisión



Casos de Prueba.

Definición y Aplicación.

[CREO que a partir de acá es lo que ya NO entra en el examen]

Preguntas parcial:

- Diferencia entre validar y verificar?
- Diferencia entre falla y error?
- Diferencia entre QC y QA?
- Defina los 7 fundamentos del testing
- Tipos y técnicas de testing: caja blanca, caja negra
- Resulta que hay jugadores VIP...
- Entorno de desarrollo? (creo que no va)
- Diferencia entre testing de regresión vs testing de confirmación? → Ojo: No va a ser sólo listar; van a haber preguntas para pensar

Rta:

Testing de confirmación verifica que un defecto específico fue corregido. Se re-ejecuta el test que falló para confirmar que ya no falla.

Testing de regresión verifica que los cambios realizados (como esa corrección) no rompieron algo que antes funcionaba correctamente.

Pero si tengo poco tiempo, hago un testing mas chiquito y eso se llama SMOKE TESTING.

EXAMEN:

Van a ser 3 temas

Vamos para el 10 Carly! :D

EXPLICACIÓN DE CONTADURÍA:

Que es un asiento contable?

Cuando queramos entrar a cualquier empresa: Felfort, Oracle, la que sea: ESTO lo dan por SABIDO.

Si a mi me dicen, siendo yo quien maneja la BDD SQL: Che, se duplicaron los asientos contables; corregilo. Yo tengo que saber lo que es. Porque si no, ni idea de cómo corregirlo.

Entonces, sabiendo que una empresa vende y compra. Las empresas que venden y compran; transaccionan. Y automáticamente se generan ASIENTOS CONTABLES. Los asientos contables se van generando con cuentas contables, para despues yo presentar un BALANCE FISCAL. Por eso se generan los asientos contables.

Decis: Señor ARCA, yo vendi esto, pero compre esto: no voy a pagar mis ganancias solamente sobre la venta, porque también compré!

Ah, bueno. Cómo me lo demostras? Con un balance. Cómo se nutre? Con asientos contables. Cómo se hace este balance? Antes era a mano, ahora se automatizaron.

Ingresó un pedido de venta. Eso genera un asiento contable? NO PORQUE ES SOLO UNA INTENCION, al igual que el pedido de compra. Ahora, cuando acepté el pedido de venta y empiezo a recibir mercadería, empieza el ASIENTO CONTABLE.

Entonces, el 1er asiento contable en la parte de ventas s genera en 3, cuando recibo la mercaderia.

El 2do asiento contable en la parte de ventas en en la FACTURA.

Y el 3er asiento contable es CUANDO COBRO.

Una pedido de venta o solicitud de compra es un asiento contable? NO! PORQUE SON INTENCIONES. Cuando RECIBO esa mercadería, ahí si. Cuando recibo la factura, se genera otro asiento contable, con los impuestos se genera otro, etc.

Y tooodo esto, me genera un BALANCE.

Y qué se encarga de, en base a todas estas transacciones, automatizar el proceso de generar estos balances hechos a base de asientos? Un ERP (Enterprise Reasource Planning) Como Tango, Oracle, etc.

Por qué invertiría en un ERP? Porque son MILES de transacciones: Es un trabajo de locos si no.

A que vamos con esto?

1) Es súper difícil y largo para los testers probar estos programas, se requiere de equipos enormes → Hay mucha demanda → Se necesita de programadores que también sepan de contaduría, es algo básico de la vida y los negocios

Casos de prueba: El corazón de un caso de prueba, es **el esperado**. Si yo no puedo detectar qué es el esperado (o "expecting"), no puedo probar.

Porque yo voy a probar, el REAL (lo que sucede con el sistema) vs el ESPERADO (lo que el contador me dijo).

Por ejemplo, el contador me pide el asiento de una factura (¿?) que es:

Cientes contra ventas. O cuentas por cobrar contra ventas.

(...)

Yo para probar/desarrollar/definir: Tengo que tener NOCIÓN de ciertos procesos ajenos a estrictamente tu propia carrera.

La consultoría es eso: preguntar, asimilar, y cuando vas a otra empresa, ya sabes.

Son cosas para aprender al desarrollar, implementar y testear sistemas de información.

Es MUY difícil desarrollarse exitosamente sin saber cómo funciona el resto de la empresa.

Ahora tenemos ChatGPT, pero antes? Tenías que valorar MUCHÍSIMO a la gente con experiencia. Tenías que hacerte amigo, invitarles un café, sacarles charla...

Ahora? Basta con saber QUÉ PREGUNTAR. Lo cual ojo, puede ser difícil, y NO es poco.

En el examen vamos a tener una empresa para un caso práctico, del cual tendremos que interpretar los datos.

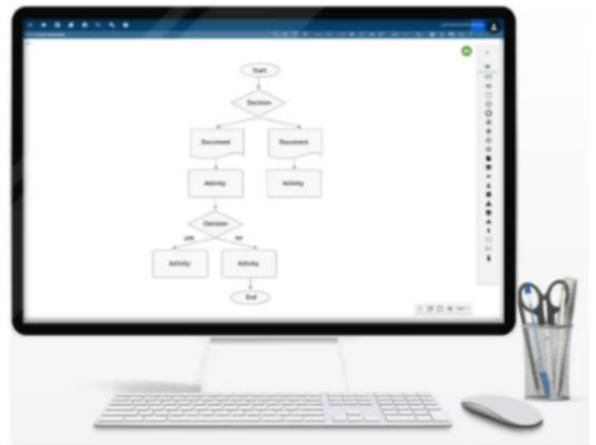
Técnicas – Diagrama de Flujo

Diseño del diagrama de flujo del proceso

Un **diagrama de flujo** es una representación gráfica de un **proceso**. Se trata de representar los pasos que sigue un proceso desde que se inicia hasta que se termina y para ello se utiliza una serie de elementos visuales que te ayuden a dibujar cada paso que sigue un proceso

Un **diagrama de flujo de procesos** tiene múltiples propósitos:

Documentar un **proceso** con el fin de lograr una mejora en la comprensión, el control de calidad y la capacitación de los empleados. Estandarizar un **proceso** para obtener una eficiencia y repetibilidad óptimas.



NO va a tomar diagrama de flujo!!!

Sólo está como para cumplir en caso de que nos quede previo.

Casos de Prueba

Contenido

1. Definición
2. Esquema
3. Características
4. Cómo diseñar casos de prueba
5. Mejores prácticas
6. Diagrama de Entidad-Relación
7. Caso de prueba vs Caso de uso
8. Atributos

Definición: El corazón de un caso de prueba es LO ESPERADO. Es poder definir que es lo esperado, que datos voy a utilizar para poder comprobar si el resultado real que me da el sistema, es igual o no al esperado. Si es igual? Paz, tranquilidad. Y si no? Fail. Y si es fail? Arreglo los bugs.

Tengo que entender el valor esperado. Y para eso como hago? Habiendo ENTENDIDO EL REQUERIMIENTO.

Casos de Prueba

Conjunto de *valores de entrada*, *precondiciones* de ejecución, *resultados esperados* y *postcondiciones* de ejecución, desarrollado con un *objetivo* en particular o condición de prueba, tales como probar un determinado camino de ejecución o para verificar el cumplimiento de un requisito determinado

Precondición: si yo en mercado libre pido un préstamo, mi PRECONDICION es:

- Tener una cuenta
- Estar logueado

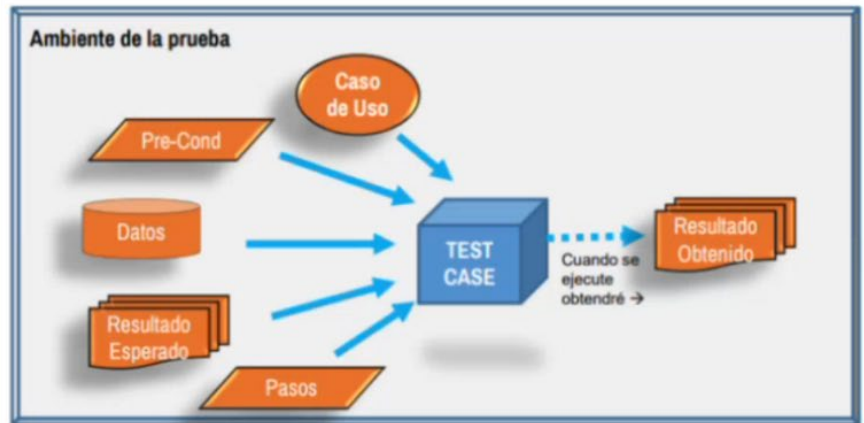
La Precondición para que me de un tutorial en un juego, es que sea New User.

Me interesa: valores de entrada, precondiciones, resultados esperados y postcondiciones.

Casos de Prueba

Un caso de prueba es exactamente lo que parece: un escenario de prueba que mide la funcionalidad en un conjunto de acciones o condiciones para verificar el resultado esperado. Se aplican a cualquier aplicación de software, pueden utilizar pruebas manuales o unas pruebas automatizadas y puede hacer uso de herramientas de gestión de casos de prueba.

Casos de Prueba



Caso de uso: Voy a alquilar un auto. Soy un actor. Valor de entrada: que tipo de auto? Valor de salida: Ferrari. Te sale 1300 USD. Me llevo el auto

Esto me lleva para crear mi caso de PRUEBA. Puedo así entender los REQUERIMIENTOS DEL NEGOCIO.

Que datos necesito para probar? Usuarios nuevos. Cual es el resultado esperado? cuales son los pasos?

Ej: pasarle plata a una amiga

Caso de prueba: Transferencia de dinero a contacto

- **Precondición:** Usuario logueado, saldo suficiente en cuenta
- **Pasos:** Ingresar monto → seleccionar contacto → confirmar transferencia
- **Resultado esperado:** El dinero se descuenta de tu cuenta y aparece acreditado en la del amigo

Necesito DESCRIBIR LOS PASOS, porque LOS EMPLEADOS ROTAN, entonces hay que dejar todo bien por escrito

CASO DE PRUEBA TEÓRICO: Lo ejecuto, y lo puedo ejecutar muchas veces por cada reléase (versión de la app). A veces me dará PASS y a veces FAIL, y en cada ejecución tengo un resultado obtenido.

Error común en los TPs: Poner resultado esperado al teórico. El teórico es teórico, no tiene resultado. La ejecución es la que tiene pass/fail; pass/fail.

Yo puedo ir planificando un TEST PLAN, que tiene MUCHOS casos de prueba, porque tengo un SPEC ya puedo empezar a laburar. Puedo hacer todos los casos de prueba.

Casos de Prueba

Características

1. Es identificado unívocamente
2. Es único en cuanto a contenido
3. Posee un estado asociado: pendiente, aprobado, fallido, etc.
4. Es re-utilizable (re-ejecutable)
5. Contiene información sobre la versión del producto
6. Posibilita el tracking del avance de las prueba

Al diseñar y construir CP los estados son: Iniciado, en proceso y terminado.

Al ejecutar: pasó, falló

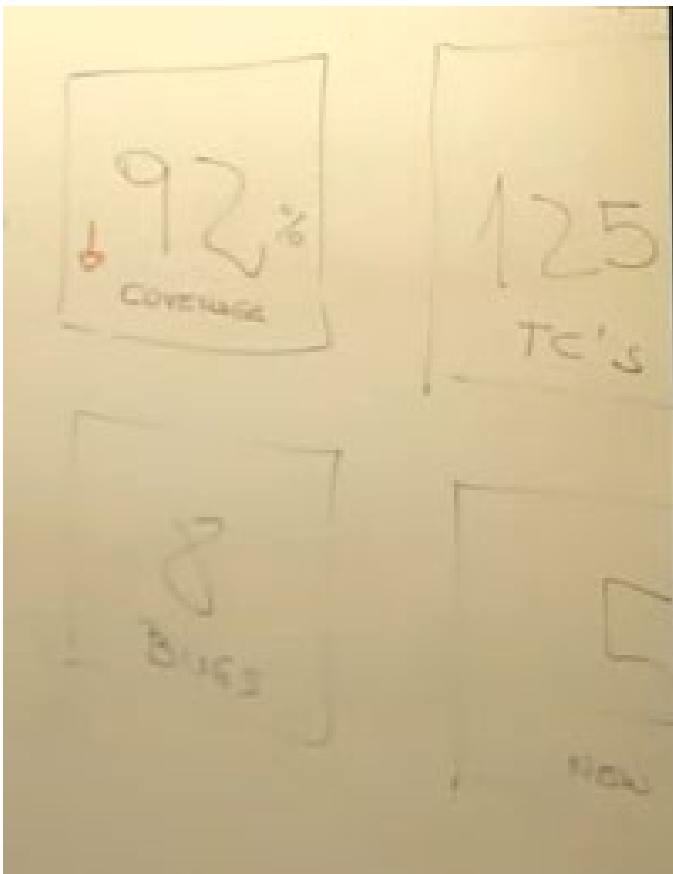
Unívoco = Único. Ej: Mi DNI, la patente de mi auto.

Que es un dashboard?

Un dashboard es una pantalla que muestra información clave de forma visual y resumida, para que puedas ver el estado de algo de un vistazo.

Por ejemplo, en Mercado Pago es la pantalla principal donde ves tu saldo, últimos movimientos y accesos rápidos a transferir, cobrar, etc.

Dashboard entra en el TP.



Que es esto? Números!

Me dicen algo? Si tuviera contexto, probablemente.

Parece referirse a estadísticas de como salieron distintos testeos de una app.

ESTO es un dashboard!

Cómo se calculan estas cosas?

Cómo se calcula la **COBERTURA**? $\frac{[Funciones probadas]}{[Funciones totales a probar]}$

El dashboard me da esta y muchas otras métricas automáticamente.

Para este TP deberemos generar tablas dinámicas, conviene usar POWER BI, o Excel. No calificar nuestra habilidad en estos programas, pero debe quedar bien.

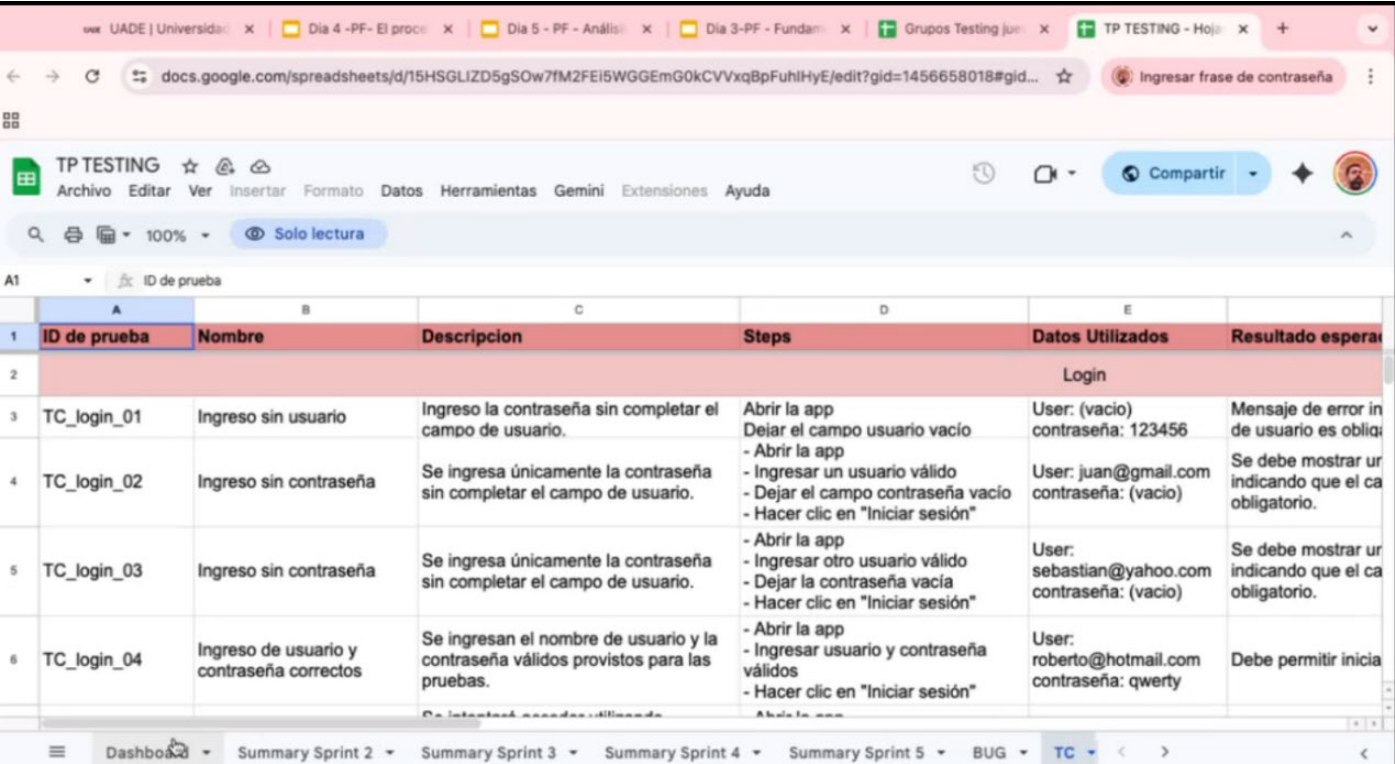
Último concepto de hoy:

Las transacciones que tienen prefijo son mucho más fáciles de analizar.
EJ: Las notas de débito tienen ND, las facturas tienen FC...

En el TP, en nuestro grupo, lo primero que haremos será:
Abrir **GOOGLE SHEETS**
TODAS las entregas deben hacerse **POR LINK**, NADA de ARCHIVOS.

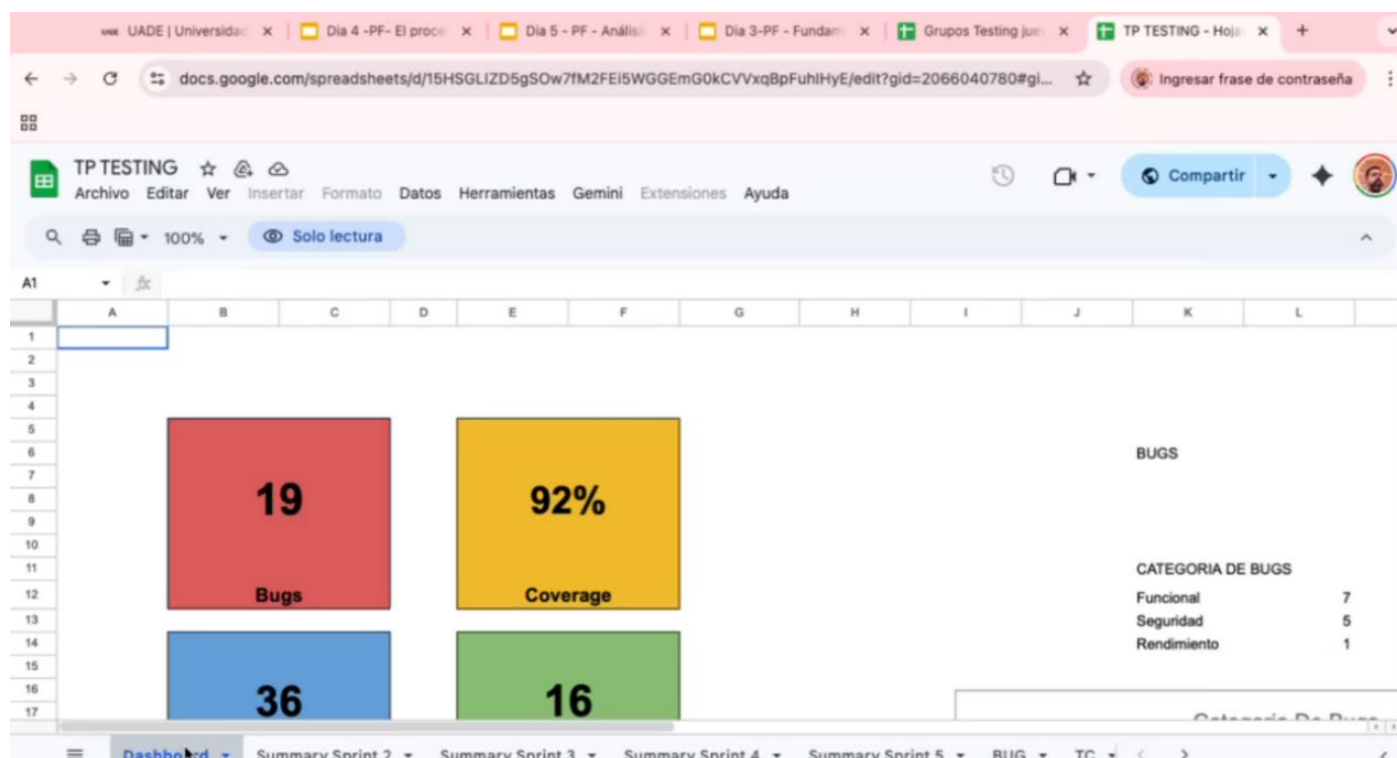
CÓMO identificaré a mis TEST CASES? → Prefijo: "TCXX" → TC01; TC02; TC03...

Ejemplo:

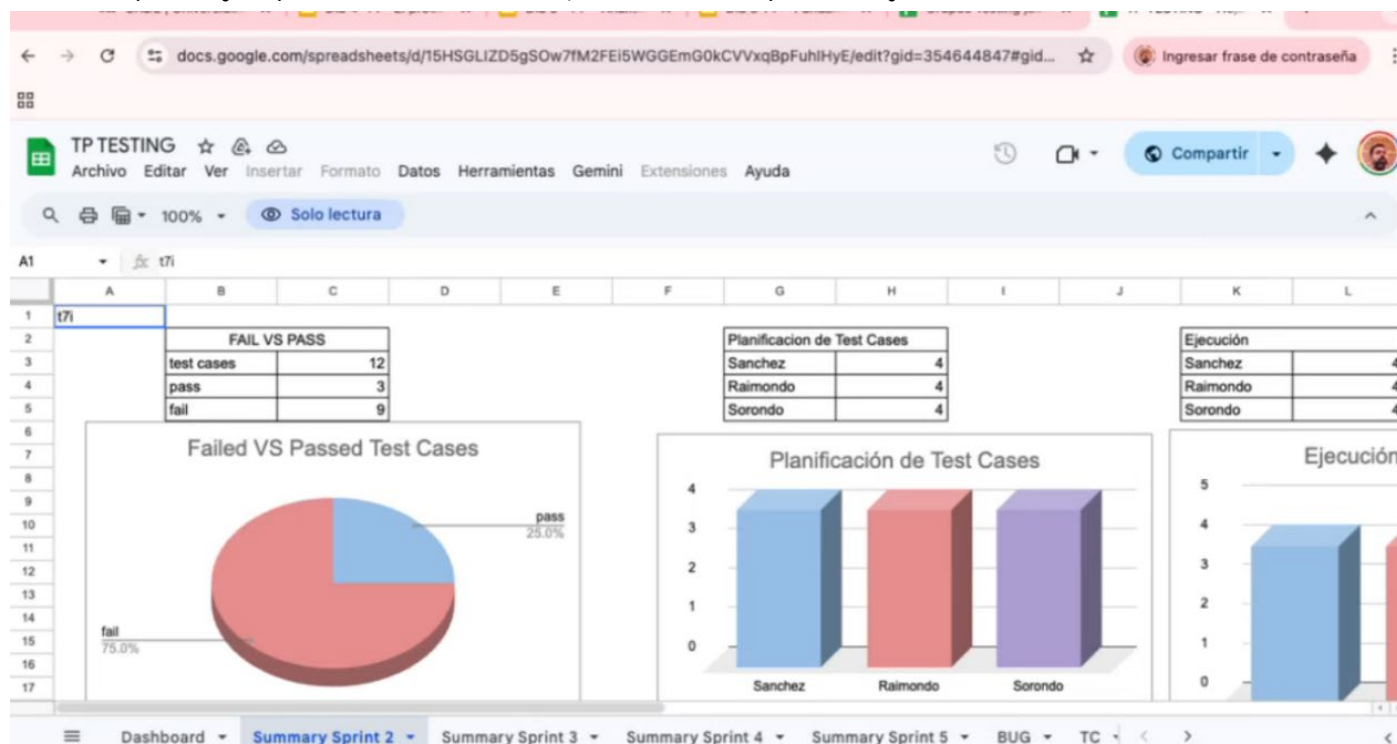


ID de prueba	Nombre	Descripción	Steps	Datos Utilizados	Resultado esperado
	Login				
TC_login_01	Ingreso sin usuario	Ingreso la contraseña sin completar el campo de usuario.	Abrir la app Dejar el campo usuario vacío	User: (vacío) contraseña: 123456	Mensaje de error in de usuario es obliq
TC_login_02	Ingreso sin contraseña	Se ingresa únicamente la contraseña sin completar el campo de usuario.	- Abrir la app - Ingresar un usuario válido - Dejar el campo contraseña vacío - Hacer clic en "Iniciar sesión"	User: juan@gmail.com contraseña: (vacío)	Se debe mostrar ur indicando que el ca obligatorio.
TC_login_03	Ingreso sin contraseña	Se ingresa únicamente la contraseña sin completar el campo de usuario.	- Abrir la app - Ingresar otro usuario válido - Dejar la contraseña vacía - Hacer clic en "Iniciar sesión"	User: sebastian@yahoo.com contraseña: (vacío)	Se debe mostrar ur indicando que el ca obligatorio.
TC_login_04	Ingreso de usuario y contraseña correctos	Se ingresan el nombre de usuario y la contraseña válidos provistos para las pruebas.	- Abrir la app - Ingresar usuario y contraseña válidos - Hacer clic en "Iniciar sesión"	User: roberto@hotmail.com contraseña: qwerty	Debe permitir inicia

Acá hay un ejemplo de un alumno medio sonso que entregó tal cual lo que puso el profe en el pizarrón:



Y acá hay un ejemplo de otro alumno que lo hizo un poco mejor:



Acá hay un alumno que usó muchos prefijos, pero que está bien:

TP TESTING

ID del bug	Descripcion	Categoria	Prioridad	Criticidad	Estado
Login					
BUG_login_01	No valida el campo de usuario vacío: al dejarlo en blanco e intentar iniciar sesión, no se muestra ningún mensaje de advertencia.	Funcional	Media	Alto	Cerrado
BUG_login_02	No valida el campo de contraseña vacío: al dejarlo en blanco e intentar iniciar sesión, no se muestra ningún mensaje de advertencia.	Funcional	Media	Alto	Cerrado
BUG_login_03	No valida el campo de contraseña vacío: al dejarlo en blanco e intentar iniciar sesión, no se muestra ningún mensaje de advertencia.	Funcional	Media	Alto	Cerrado

STEPS

TP TESTING

ID de prueba	Nombre	Descripcion	Steps	Datos Utilizados	Resultado esperado
Login					
TC_login_01	Ingreso sin usuario	Ingreso la contraseña sin completar el campo de usuario.	Abrir la app Dejar el campo usuario vacío - Abrir la app - Ingresar un usuario válido - Dejar el campo contraseña vacío - Hacer clic en "Iniciar sesión"	User: (vacío) contraseña: 123456	Mensaje de error in de usuario es oblig
TC_login_02	Ingreso sin contraseña	Se ingresa únicamente la contraseña sin completar el campo de usuario.	- Abrir la app - Ingresar otro usuario válido - Dejar la contraseña vacía - Hacer clic en "Iniciar sesión"	User: juan@gmail.com contraseña: (vacío)	Se debe mostrar un indicando que el ca obligatorio.
TC_login_03	Ingreso sin contraseña	Se ingresa únicamente la contraseña sin completar el campo de usuario.	- Abrir la app - Ingresar otro usuario válido - Dejar la contraseña vacía - Hacer clic en "Iniciar sesión"	User: sebastian@yahoo.com contraseña: (vacío)	Se debe mostrar un indicando que el ca obligatorio.
TC_login_04	Ingreso de usuario y contraseña correctos	Se ingresan el nombre de usuario y la contraseña válidos provistos para las pruebas.	- Abrir la app - Ingresar usuario y contraseña válidos - Hacer clic en "Iniciar sesión"	User: roberto@hotmail.com contraseña: qwerty	Debe permitir inicia

RUN

The screenshot shows a Google Sheet titled 'TP TESTING' with a menu bar (Archivo, Editar, Ver, Insertar, Formato, Datos, Herramientas, Gemini, Extensiones, Ayuda) and a toolbar with search, print, zoom (100%), and view (Solo lectura) options. The sheet contains a table with columns: Run ID, TC ID, Nombre, Version, Epic, Sprint, Plataforma, Resultado, and Bug ID relacionados. The data is organized into sections: 'Login' (rows 3-8) and 'Columna' (rows 32-37). The 'Login' section shows test runs with results 'Fail' or 'Pass'. The 'Columna' section shows a test run with a 'Fail' result. The bottom navigation bar includes tabs for 'Summary Sprint 2' through 'Summary Sprint 5', 'BUG', 'TC', and 'RUN' (selected).

	A	B	C	D	E	F	G	H	I
1	Run ID	TC ID	Nombre	Version	Epic	Sprint	Plataforma	Resultado	Bug ID relacionados
2	Login								
3	Run_001	TC_login_01	Ingreso sin usuario	1.0.0	Epic_01	Sprint_02	IOS	Fail	BUG_login_01
4	Run_001	TC_login_02	Ingreso sin contraseña	1.0.0	Epic_01	Sprint_02	IOS	Fail	BUG_login_02
5	Run_001	TC_login_03	Ingreso sin contraseña	1.0.0	Epic_01	Sprint_02	IOS	Fail	BUG_login_03
6	Run_001	TC_login_04	Ingreso de usuario y contraseña correctos	1.0.0	Epic_01	Sprint_02	IOS	Pass	
7	Run_001	TC_login_05	Ingreso usuarios intercambiados con las contraseñas	1.0.0	Epic_01	Sprint_02	IOS	Fail	BUG_login_04
8	Run_001	TC_login_06	Ingreso sin @ incluido en el correo	1.0.0	Epic_01	Sprint_02	IOS	Fail	BUG_login_05
32	Run_002	TC_login_09	Funcion de ocultar-mostrar contraseña	1.0.1(3)	Epic_02	Sprint_03	IOS	Pass	
33	Run_002	TC_login_10	Error de ortografia.	1.0.1(3)	Epic_02	Sprint_03	IOS	Pass	
34	Run_002	TC_login_11	Acceso al formulario de registro	1.0.1(3)	Epic_02	Sprint_03	IOS	Pass	
35	Run_002	TC_login_12	Verificar que toda la interfaz esté en el mismo idioma	1.0.1(3)	Epic_02	Sprint_03	IOS	Pass	
36	Columna								
37	Run_003	TC_columna_01	Creación de columnas personalizadas	1.0.1(4)	Epic_03	Sprint_04	IOS	Fail	BUG_columna_01

Se ve que hubo 3 releases

Asi va a ser el TP.

Yo soy el cliente, vienen los producto managers, se lo mandamos

-El profe: Odio tener que entender sus letras horribles en papel. Quiero que diseñen un programa para que se hagan exámenes UADE desde la computadora.

Cómo hacemos?

Un login, una forma de verificar que no haya fraude, que cuando se active no pueda acceder a internet o copypastear, armo el prototipo, lo empiezo a desarrollar. Tengo que esperar al final para desarrollar? NO! Hago el login, pruebo el login. Hago la validación de internet, la pruebo. Y así. No hacemos waterfall, hacemos scrum.

Yo soy el negocio/cliente, viene un grupo de producto manager, me van a tratar de entender, lo van a especificar, lo van a validar, y lo van a mandar al equipo de desarrollo que lo van a armar. El dev team va a tener entregas iterativas, y el equipo de QA las va a ir probando.

Los de DEV van a probar? SI! Caja blanca. Estrategia de testing "pruebas unitarias". Entonces los usuarios no prueban nada? SI: UAT. Que era UAT? User Acceptance Testing.

Entonces, son pruebas que tiene que hacer más el negocio, el equipo de QA, para aceptar que cumplieron con más genperico. Van a haber entonces 3 estrategias de testing: La de los PM UAT, la de los devs caja blanca, y la de QA? magia con todo esto (el profe señala las tablas de Excel).

Y esto lo transforman en una presentación corporativa, y lo presentan frente a un jurado estricto.

Y que voy a hacer para especificar? Usar IA.

Y para prototipar? Y codear? Idem.

Clase 7

Profe: "Quería que esta clase sea para armar los grupos, pero no vinieron los suficientes. Pero en el trabajo, tenemos que aprender a adaptarnos a los imprevistos"

Vamos a hacer un roleplay (simular una interacción): Cliente, Necesidad del negocio.

Etapas 1: Requerimientos.

1er pregunta (fuera del roleplay): Que hay que hacer en el trabajo? Networking

Cómo es el proceso de desarrollo? Iterativo.

Por qué empieza el desarrollo? Porque tengo un problema.



Cientes hablan con Product Owners. De ahí nace una lista de SPECS.

VALIDAMOS que esas specs cumplan con los requerimientos de negocio.

El documento de specs suele ser muy extenso. En nuestro ejemplo, tendrá 89 páginas.

Como es muy extenso, NO se lo doy así nomas a los Developers. Porque si no, predispongo a que no lo lean bien o tenga errores.

(Developers = Programadores, QA, UX, DB...)

Entonces, lo divido en HISTORIAS DE USUARIO.

Estructura de una HISTORIA DE USUARIO:

Como... Quiero... Para... & Criterios de Aceptación

Ej: Si hago una transferencia, mi criterio de aceptación es → Se me achicó el saldo (proporcionalmente) → Se le incrementó el saldo al receptor.

Toda esta estructura que planteamos: A veces se cumplen, a veces no. Cómo se llaman las user stories coloquialmente? Oraciones por slack.

El umbral depende de uno de los Fundamentos del Testing: El Contexto.

No hay slides para este tema, porque cada cuatri el profe lo va cambiando. Pero, por que lo va cambiando?

El equipo de devs desarrolla, y cuando desarrolla lo manda al equipo de QA? NO! Se hablan mutuamente constantemente.

El equipo de QA también está presente en la reunión entre "Cliente-Product Owner", donde se establecen las specs, porque tiene que saber cómo planificar las pruebas.

Aquí, los QA definen tiempo, recursos, técnicas, tipos.
Luego, los QA desarrollan los CASOS DE PRUEBA.

Ej: No voy a hacer pruebas de UX/UI temprano porque generalmente cambian; se hacen al final. No voy a hacer pruebas no funcionales.

Yo necesito que estos equipos sean AUTÓNOMOS y PROFESIONALES. Esto lo define RECURSOS HUMANOS, encargados de traerme GENTE CALIFICADA.

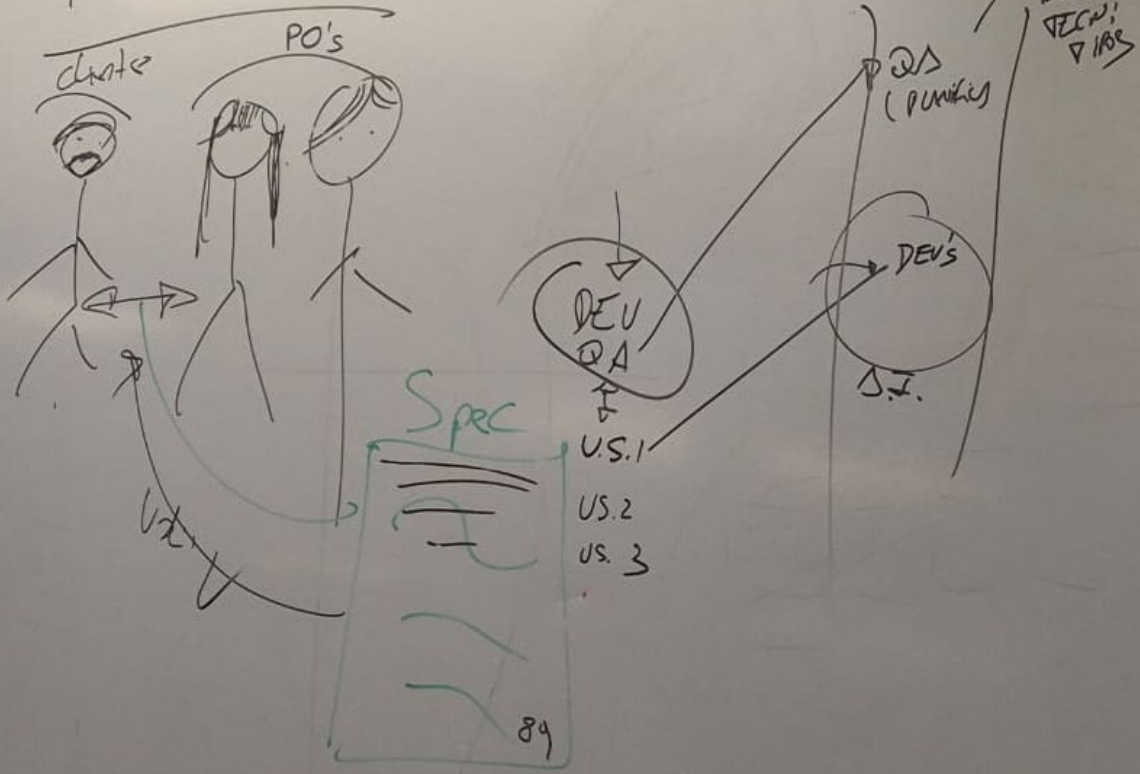
Necesitamos trabajadores PROACTIVOS: Que NO esperen a que le digan qué hacer.

El ciclo será ITERATIVO: No tiene que ser perfecto, sólo empezamos, e iremos mejorando.

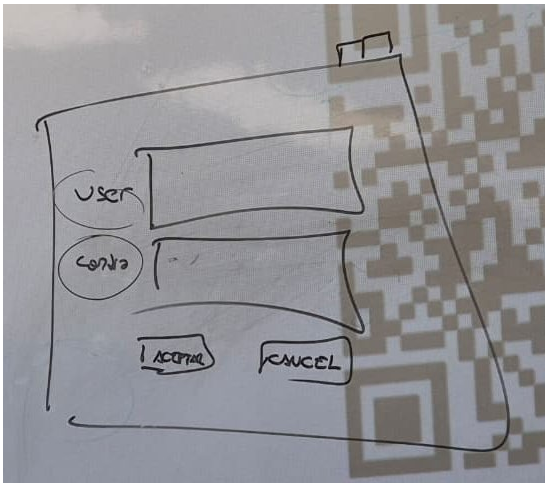
Conviene usar la IA para hacer la hoja de specs, revisarla a fondo e ir corrigiéndola, en lugar de redactarla desde cero.

La consultoría tiene algo de marketing, pero una cosa es saber mostrar lo que hiciste, y otra cosa es adornarlo; mostrar lo que no hiciste, diciendo que no hice.

PROBLEMS



¿Qué casos de prueba elegiríamos?



Qué tenía esto de mal:

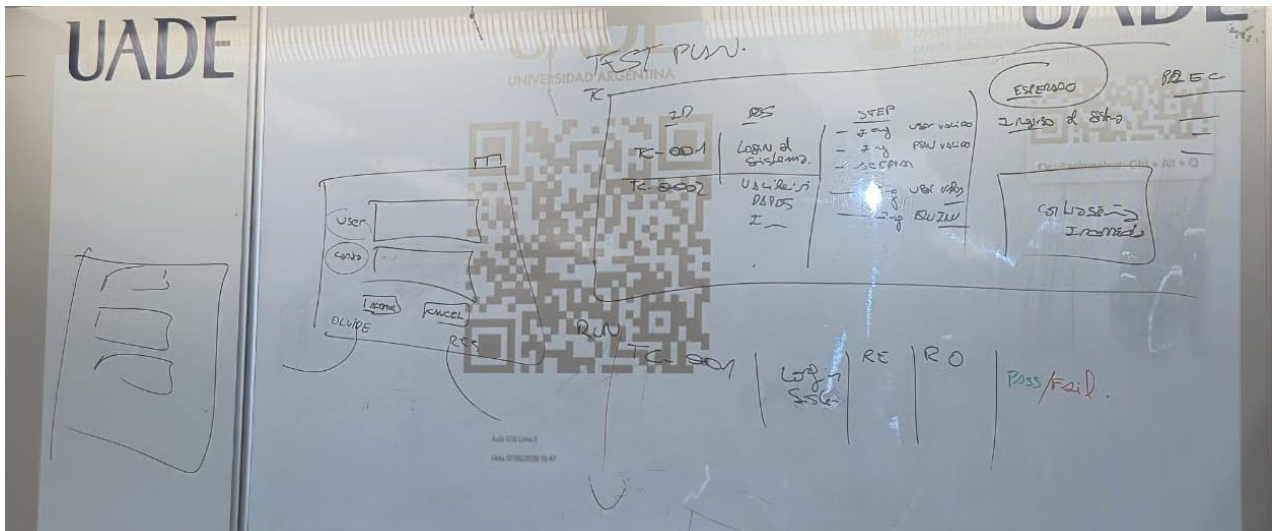
- Botones en idiomas distintos
- El aceptar está a la izquierda

CORAZÓN DE UN CASO DE PRUEBA: El resultado esperado

ID	RS	STEP	ESPERADO
TC-001	Login exitoso al Sistema	- Ingresar user válido - Ingresar password válido - Aceptar	Ingreso al sistema
TC-002	Login fallido al sistema	-Ingresar usuario inválido -Ingresar contraseña inválida -Ingresar usuario válido y contraseña inválida -Ingresar usuario inválido y contraseña válida	Mensaje de error: "Usuario o contraseña inválidos"

Si pongo un usuario, y me dice "Contraseña incorrecta"? No, porque me está diciendo que ese usuario si existe. Debería ser "Usuario o contraseña incorrectos"

Los equipos de QA en este experimento que haremos, deberán definir el PLAN DE PRUEBA o TEST PLAN.



Además nos faltaba "Olvidé mi contraseña" y "Registrarme";
De eso derivan otras 2 pantallas.

OJO: Recordemos PROBAR ABSURDOS! Para eso vimos la tabla de decisión.

Si tengo tiempo → Pruebo todo (full pass)

Si no tengo tiempo → Pruebo lo nuevo y/o lo más completo (Fundamento del Testing: El 80% de los errores viene del 20% del código)

Post-break haremos el roleplay:

Los Product Owner deberán planificar las UAT (User Acceptance Test)

Los devs deberán hacer pruebas unitarias.

GIRA: Ofrece varias soluciones para gerenciamiento de proyectos, sobre todo en el área IT. Podemos gestionar User Stories, Backlog, etc; todo centralizado y en la nube.

TRELLO: De la empresa de GIRA. Plataforma colaborativa en la nube, que me permite subir links de diferentes cosas (Word, Excel, etc).

1er objetivo: Excel de equipos hoy publicado.

Tabla de abajo es la RUN: Los casos de prueba teórico, hechos en la realidad.

RUN	TC	Caso	RE	RO	Pass/Fail
R01	TC-001	Login Sistema			
R01	TC-002				
R01	TC-003				
R02	TC-004 TC-005				
R3	(...)				

HAY QUE AUTOMATIZAR LAS PRUEBAS! (Si somos QA; No aclaró sobre testers)

Si estamos dando el oral, y alguien se come algo, no digas "Che te equivocaste/te faltó esto" decir: "Y complementando a mi compañero..."

Que la presentación no sea "Disney": Decir cual fue el proceso, como lo solucionaron, qué nos quedó pendiente.

Inicio del Roleplay (Roleamos? UwU?)

Propósito: Creación, Administración, Evaluación	PM/PO:
Objetivo: Desarrollar una plataforma que...	DEV:
Milestone 1:	QA:

Como dueño de la UADE y docente, mi problema es que el 1er parcial de Testing, de que manera puedo crear, administrar y evaluar el 1er parcial de testing de aplicaciones en la uade? Corregir hojas en manuscritos es arcaico.

Cual es el centro en la compañía? EL CLIENTE

Y en un hospital? EL PACIENTE

Y en una universidad? EL ALUMNO

Yo creo que si podemos mejorar esos tiempos de corrección desarrollando una plataforma que me permita crear-administrar y evaluar, mejoramos la experiencia del estudiante y de la universidad.

Me gustaría empezar con un MVP con el 1er parcial de Testing de aplicaciones en la UADE.

Camino del profesor:

Diseño mi examen → lo almaceno en la nube → se lo mando el de la fotocopidora → voy a la fotocopidora → imprimo 60 copias → voy a sala de maestros a buscar hojas de colores y un sobre marrón para guardar los exámenes → reparto una copia de parcial + 2 hojas por alumno → los retiro a las 2 horas → los guardo en el sobre marrón → los corrijo en mi casa → los vuelvo a llevar a la clase para que los vean los alumnos → cargo las notas a la plataforma → llevo los exámenes a administración.

Que falta? OPTIMIZAR!!!

2 reglas de negocio que no se negocian:

- Los exámenes los hago yo (el docente), NO la IA
- La plataforma NO tiene la palabra final en la nota, si no el docente

Al profe le gustó que hubiera una corrección tentativa con IA.

Cómo hacemos esta corrección con IA? El profe carga su examen resuelto, y la IA compara si se asemeja o no; si le da el puntaje a cada pregunta de valor de un punto, de:

- 1
- 0,75
- 0,5
- 0,25
- 0

(una compañera dio una descripción detallada sobre cómo armaría su app)

- Que es lo primero que hago una vez que tengo todos los requerimientos?
Se lo mando a la IA

Próxima clase:

- El equipo de PO: Decidir el approach que quieren tomar
- El equipo de dev: staff tecnológico que va a necesita
- El equipo de QA: Decidir que testeos harían (pero si no tienen los requerimientos?)

Hay empresas que piensan:

PM
↓
DEV
↓
QA

Esta jerarquización está MAL: deben estar al MISMO NIVEL o surgen conflicto. El esquema ideal es:

* Dev

*PO SOLUCIÓN *QA

O sea, un CÍRCULO en donde TODOS ESTÉN AL MISMO NIVEL, y LA SOLUCIÓN EN EL CENTRO.

Presentación de 1-2 slides sobre EL STACK TECNOLÓGICO: QUE VAMOS A DESARROLLAR, EN QUÉ, CON QUÉ ESTRATEGIA, ETC.

QA: Armarán un spreadsheet en Google sheets, con:

SUMMARY - TEST CASES - RUN - BUGS - CREADOR - VALOR ESPERADO -
PRIORIDAD - SEVERIDAD - STEPS

En donde RUN y BUGS estarán la clase que viene

Rellenar este primer ejemplo con el happy path de un login.

Y el que se anima para sumar puntos hace una presentación en CANVA con una estrategia de Testing (sin tener en cuenta los requisitos del PO porque no van a haber todavía)

PREGUNTAS:

- Escalabilidad de 1er-2do parcial?
- Escalabilidad a más universidades?
- Escalabilidad a más países? Idiomas?

No hubo apuntes: Sólo pasó a revisar cómo iban los QAs, y expusimos el Canva del Sprint 1 de Devs.

Clase 9

Feedback Oral 1er Sprint

Tuvo demasiada cantidad de texto. La próxima, hagamos más bullet points; más minimalista.

Sprint 2

xd